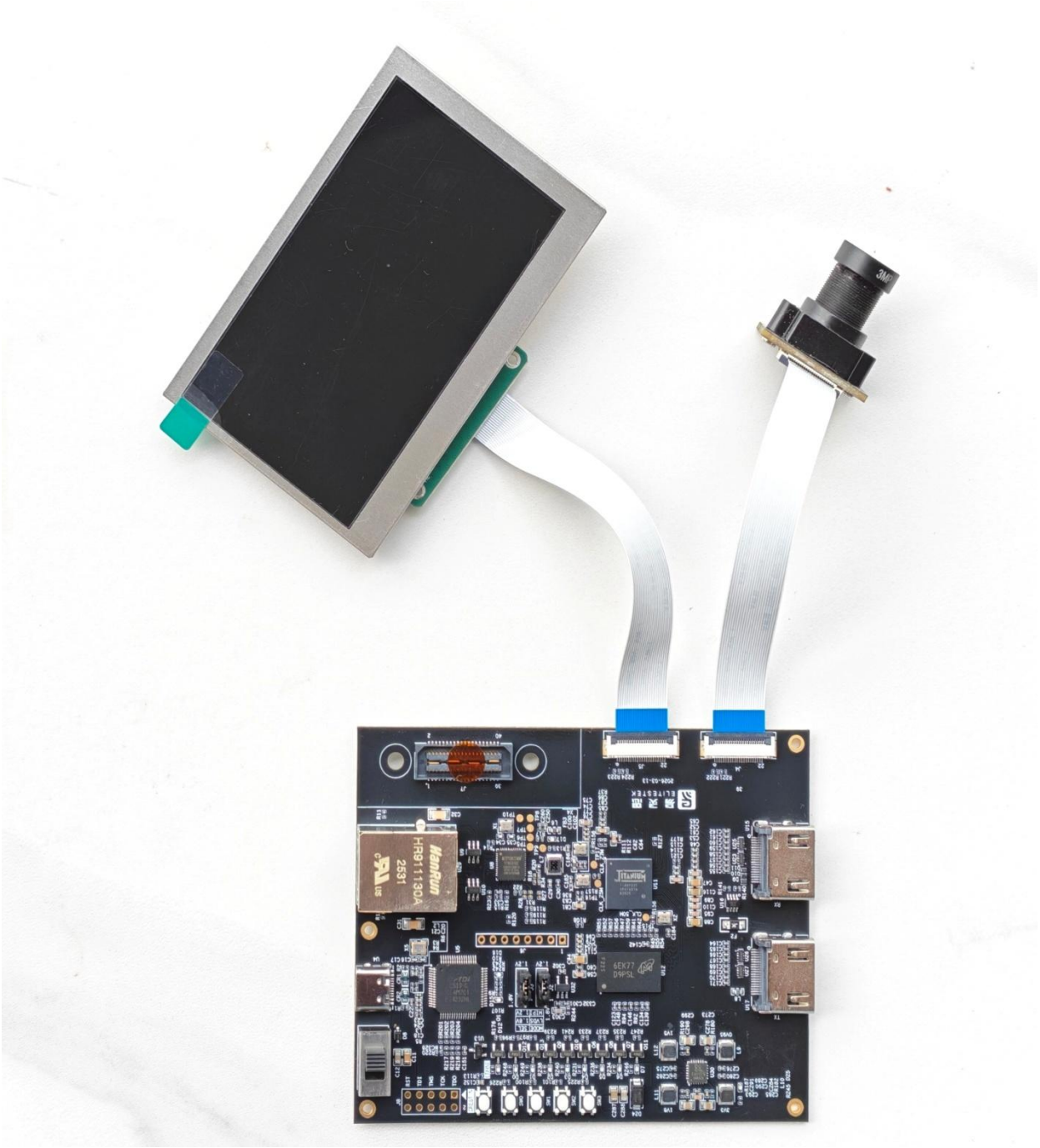
TI60F225I3 DEMO 说明

Edge Wang

[电子邮件地址]


3M
ManRun
HR911130A
2531

目录

硬件篇 5

一、概述 6

二、FPGA 6

2.1 简述 6

2.2 电源部分 7

2.3 bank 划分 7

2.4 Bank 电压的选择 8

2.5 晶振 10

2.6 上电时序及复位 11

2.7 JTAG 下载 13

2.7.1 板载下载器 13

2.7.2 外接下载器 13

2.8 编程方式 14

2.9 加密验证 14

三、GE 14

3.1、RGMII 15

3.2、硬件设计 16

3.3 管脚分配 21

四、HDMI 22

五、DDR3 23

5.1 DDR3 简介 23

1、DDR 的容量大小 24

2、速率等级 24

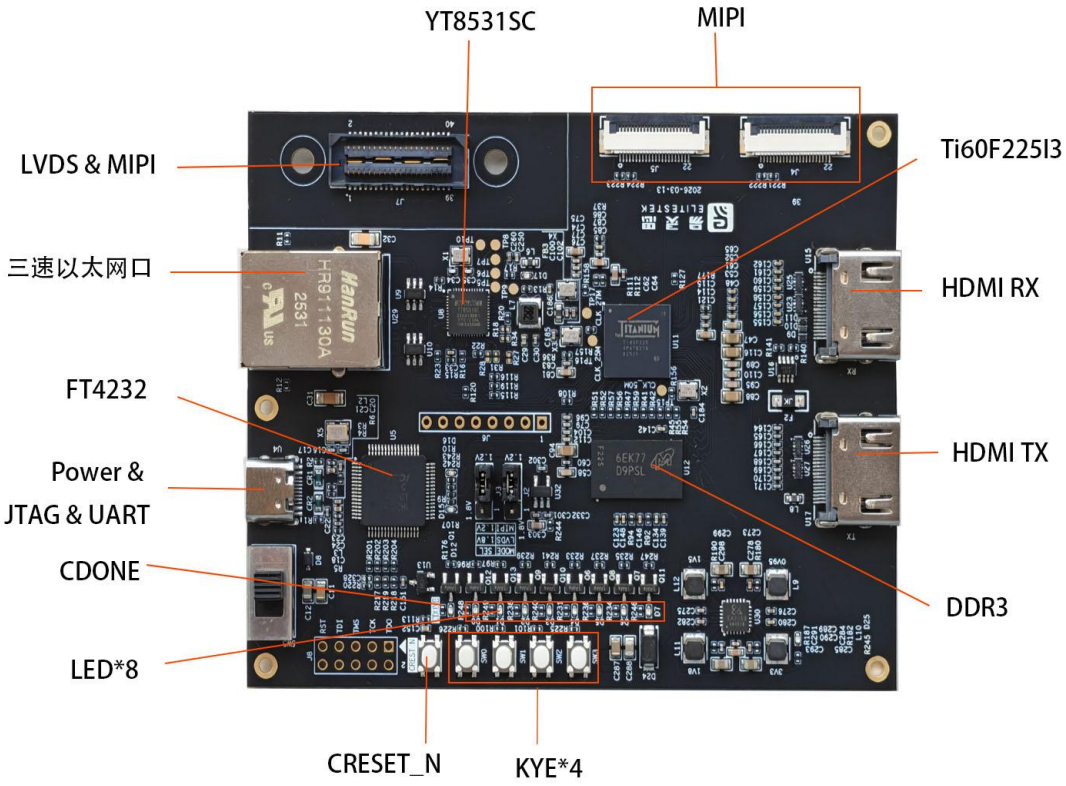
3、DDR3 的芯片地址位宽 25

5.2 Ti60F225 对 DDR3 的支持 25

六、预留 IO 25
6.1 简介 25
6.2 扩展口 J7 引脚分配 26
6.3 拓展子卡 27
七 MIPI 接口 28
7.1 SOFT MIPI 硬件设计原则 28
7.2 Pannel 接口与 FPC 的连接 29
panel 接口与 FPC 对应关系 30
7.3 Sensor 接口与 FPC 的连接 31
sensor 接口与 FPC 对应关系 31
七、Flash 32
八、用户 IO 33
按键管脚分配 33
LED 管脚分配 33
九、加载模式验证管脚 34
十、UART 34
软件篇 34
学习入门 35
01_Ti60F225_Key_led_osc_demo 35
02_Ti60F225_hdmi_demo 43
03_hdmi_tx_demo 49
04_Ti60F225_eth_demo 52
04_Ti60F225_lvds_loop_demo 62
05_Ti60F225_lvds_loop_dpa_demo 67
07_mipi_csi2dsi_SC431AI_demo 74
08_Ti60F225_hardjtag_demo 79

(1)逻辑工程配置 80
(2)打开 RSICV IDE 自带工程。 84
(3)合成固化文件 88
(4)IP 更新注意事项 90
09_Ti60F225_co_debug_demo 91
并发调试（Enable Concurrent Debugging） 91
09_Ti60F225_hdmi2dsi_demo 93
10_Ti60f225_csi2hdmi_demo 96
一、Demo 框图 97
11_mipi_csi_loopback 100
12_mipi_dsi_loop_demo 105
13_carrychain_prj_demo 105
17_Ti60F225_uart_demo 106

硬件篇

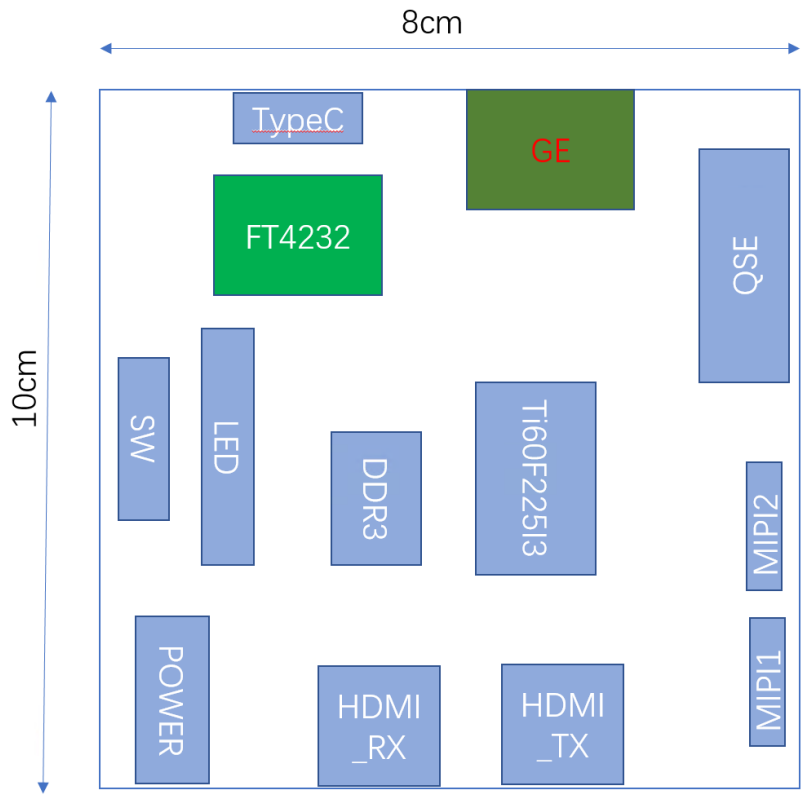

YT8531SC
MIPI
LVDS & MIPI
Ti60F225I3
三速以太网口
HDMI RX
FT4232
HDMI TX
Power &
JTAG & UART
CDONE
DDR3
LED*8
CRESET_N
KYE*4

一、概述

Ti60F255I3-DK 采用的是易灵思的 Ti60F225 FPGA 为核心的开发验证平台。

该验证平台包括 Ti60F225I3 FPGA，一个 GE 口 ,HDMI 接收及 HDMI 发送和 16 位的 DDR3，两路 FPC MIPI 座子，并预留了一个 QSE 座子用于扩展 IO 和验证 MIPI 接口。

其中 GE 口都可以实现 3 速；HDMI 通过 FPGA 高速 IO 实现，可支持到 1080p60Hz；DDR3 采用的是 MT41J128M16JT-125，位宽为 16 位，数据速率最高 800Mbps，可以满足各种高带宽需求。板上的 flash 是 8MB，采用的是 GD25LQ64，用于存储 FPGA 镜像，也可以存储用户数据。下图是各个外设在板上的分布情况。



二、FPGA

2.1 简述

demo 板采用的是易灵思钛金系列的 Ti60F225I3.易灵思钛金系列采用的 16nm 工艺，它具有超低功耗、超小封装和超高性能等特性。

Ti60F22I3 有 60K 逻辑单元，尺寸是 10mm x 10mm，它支持 1.5Gbps　LVDS 和 1.5G MIPI DPHY,可以外挂 800Mbps DDR3.

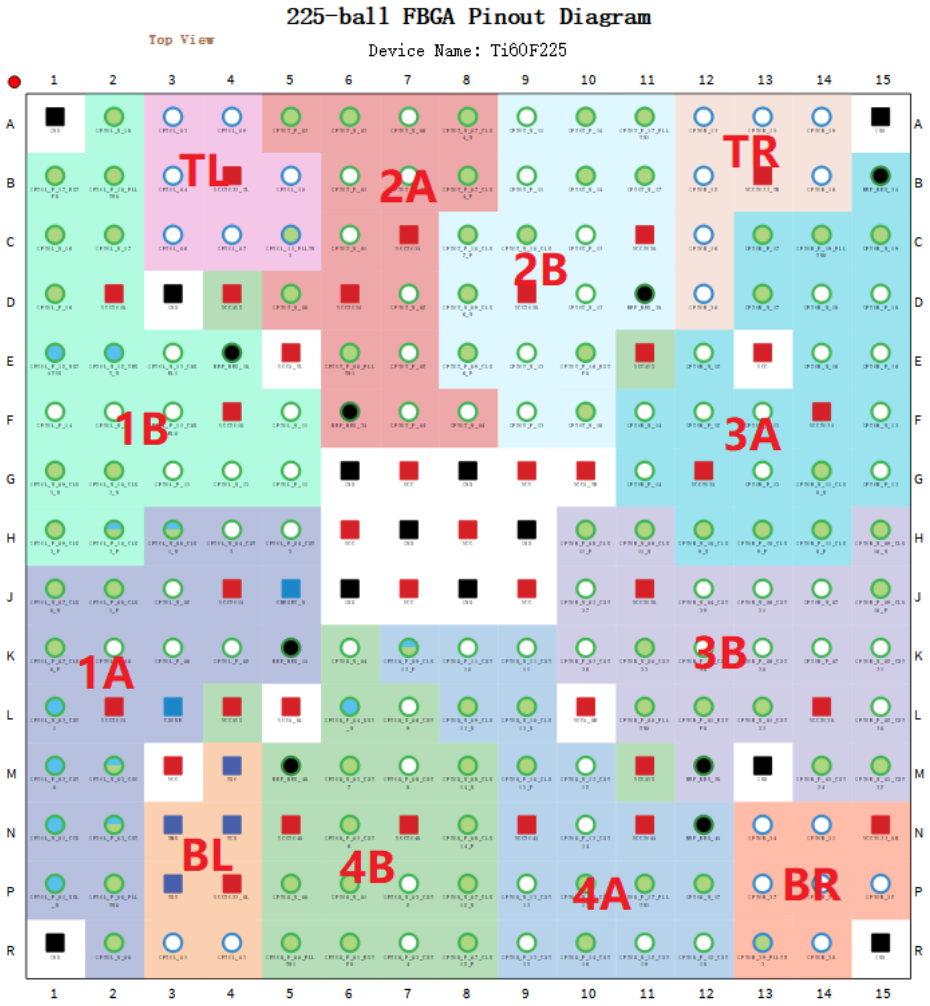
2.2 电源部分

Ti60F225 的电源有表中以下几路。核电压 VCC 是 0.95V。VCCA 是 PLL 的模拟供电电压，为 0.95V；VCCAUX 是辅助供电电压，为 1.8V；VCCIO 是 HSIO 的 IO 供电电压，支持 1.2V，1.5V 和 1.8V；VCCIO33 是 HVIO 的 IO 供电电压，最高支持 3.3V。

Symbol	Description	Min	Max	Units
VCC	Core power supply	-0.5	1.05	V
VCCA	PLL analog power supply	-0.5	1.05	V
VCCAUX	1.8 V auxiliary power supply	-0.5	1.98	V
VCCIO	HSIO bank power supply	-0.5	1.98	V
VCCIO33	HVIO bank power supply	-0.5	3.6	V

2.3 bank 划分

下图是 Ti60F225 的 bank 分配图

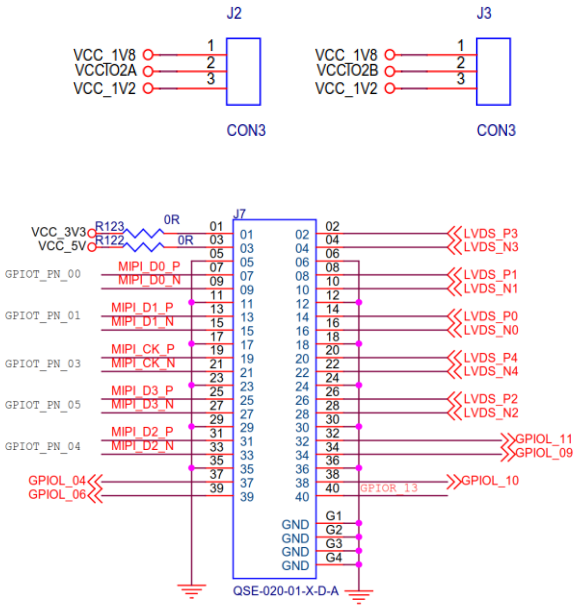


出于硬件设计走线的考虑

（1） DDR3 管脚占用 3B，4A，4B 三个 bank,bank 电压固定为 1.5V，在 FPGA bottom
（2） HDMI 暂用 3A bank，bank 电平固定为 1.8V，在 FPGA 的 right
（3） GE 口走 bank 1A，1B。在 FPGA 的 left,bank 电压固定为 1.8V
（4） 预留的 QSE 座连接到 2A,2B，TR,TL bank 在 FPGA 的 top。其中 2A,2B，支持 1.2V 和 1.8V 电平选择。
（5） BL,BR,TR 和 TL 用于走 LED 和按键。和 GE 口的 3.3V 复位，Bank 电压固定为 3.3V

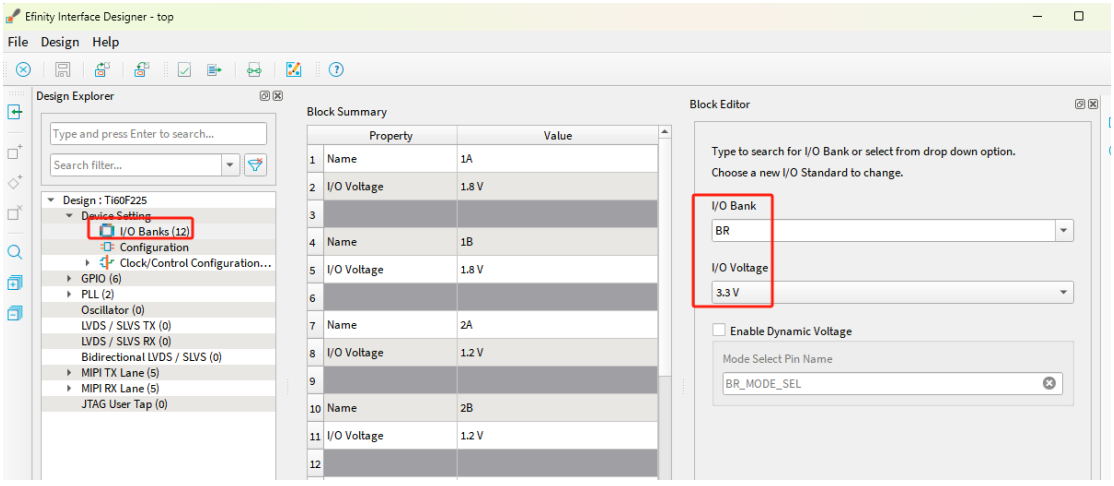
2.4 Bank 电压的选择

(1)扩展座子 J7 可以用作 MIPI 和 LVDS。其中 MIPI IO 在 Bank 2A，电压由 J2 来选择，当用作 MIPI 时选择 1.2V，用作 LVDS 时选择 1.8V; LVDSI IO 在 Bank 1B 在，Bank 电压固定为 1.8V。



BANK 2A 和 BANK 2B 的电源选择用 J2,J3 来控制。当作用 MIPI 时要选择 VCC_1V2.当用途 LVDS 时要选择 VCC_1V8.

（2）需要注意的是在 Interface Desinger 中选择电压时一定要与外部的硬件电压一致。Bank BR,BL,TR,TL 的硬件设计电压为 3.3V，在 interface 中一定要选择 3.3V，如果选择使用为 1.8V 时会烧坏 VCCIO。电压选择如下图。



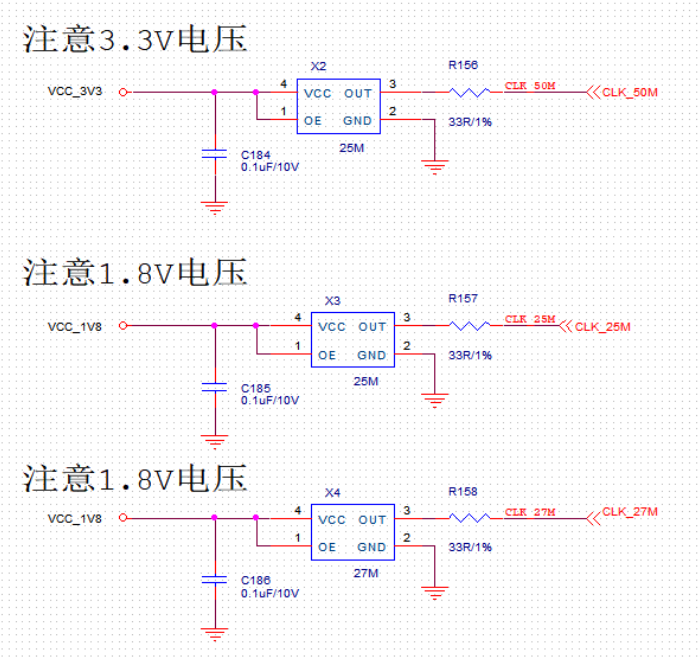
2.5 晶振

Ti60F225 的 PLL 资源

Ti60F225 有 4 个 PLL，分别是 PLL_BL、PLL_TL、PLL_TR 和 PLL_BR，这些 PLL 的参考输入分别为 REFCLK0,REFCLK1,REFCLK2.

PLL	REFCLK0	REFCLK1	REFCLK2	External Feedback I/O
PLL_BL	Single-ended: GPIOL_P_00_PLLIN0 Differential: GPIOL_P_00_PLLIN0, GPIOL_N_00	Single-ended: GPIOB_P_00_PLLIN1 Differential: GPIOB_P_00_PLLIN1, GPIOB_N_00	Unbonded(7)	Single-ended: GPIOB_P_01_EXTFB Differential: GPIOB_P_01_EXTFB, GPIOB_N_01
PLL_TL	Single-ended: GPIOL_P_18_PLLIN0 Differential: GPIOL_P_18_PLLIN0, GPIOL_N_18	Single-ended: GPIOT_P_00_PLLIN1 Differential: GPIOT_P_00_PLLIN1 GPIOT_N_00	GPIOL_11_PLLIN2	Single-ended: GPIOL_P_17_EXTFB Differential: GPIOL_P_17_EXTFB, GPIOL_N_17
PLL_TR	Single-ended: GPIOR_P_19_PLLIN0 Differential: GPIOR_P_19_PLLIN0, GPIOR_N_19	Single-ended: GPIOT_P_17_PLLIN1 Differential: GPIOT_P_17_PLLIN1, GPIOT_N_17	Unbonded(7)	Single-ended: GPIOT_P_16_EXTFB Differential: GPIOT_P_16_EXTFB, GPIOT_N_16
PLL_BR	Single-ended: GPIOR_P_00_PLLIN0 Differential: GPIOR_P_00_PLLIN0, GPIOR_N_00_CDI22	Single-ended: GPIOB_P_17_PLLIN1 Differential: GPIOB_P_17_PLLIN1, GPIOB_N_17	GPIOR_29_PLLIN2	Single-ended: GPIOR_P_01_EXTFB Differential: GPIOR_P_01_EXTFB, GPIOR_N_01_CDI23

Ti60F225-DK 上配有三个有源差分晶振，分别是 25M,27M,50M.其中 25M 走 GCLK 时钟，可以用于 DDR3 参考时钟；27M 走 GCLK 管脚，可以用于常用视频参考时钟；50M 时钟走 3.3V HVIO 所在 bank 的 PLL 输入管脚，建议用作低速信号的时钟。



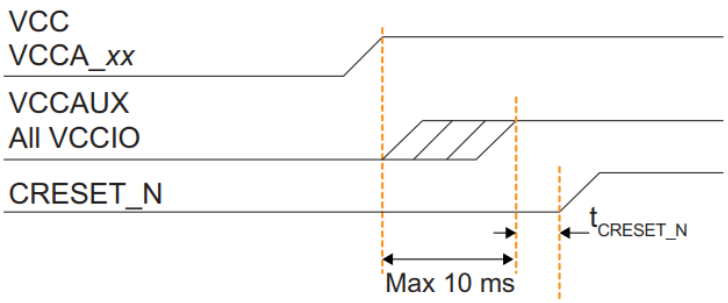
评估板为每个时钟都预留了**测试点**。

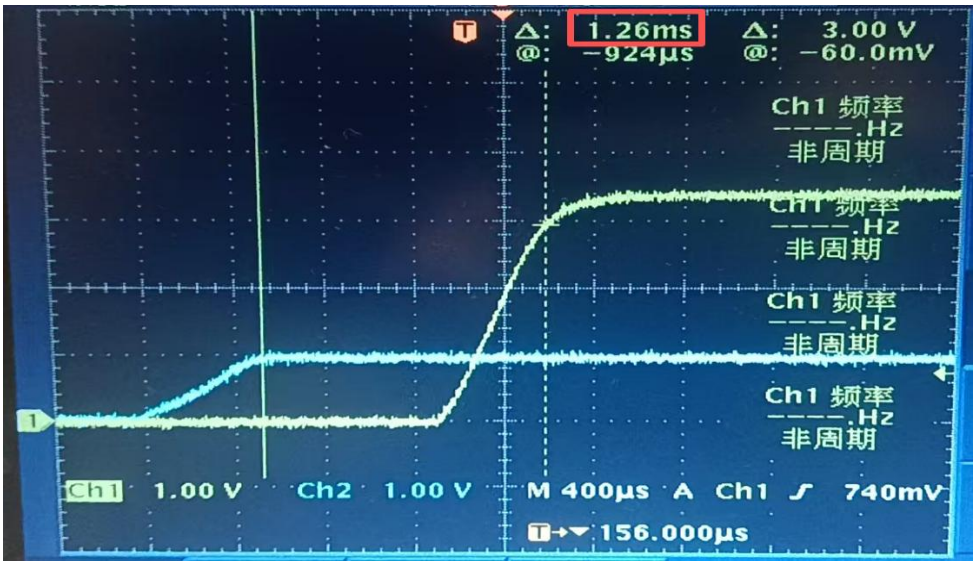
时钟管脚分配

时钟源	TI60F225 管脚名	PIN Num	对应 PLL	说明
25 MHz	GPIOL_P_08_CLK1_P	J2	*	GCLK
27 MHz	GPIOL_P_10_CLK3_P	H2	*	GCLK
50MHz	GPIOR_29_PLLIN2	R13	PLL_BR	3.3V bank IO

2.6 上电时序及复位

Ti60F225I3 对上电时序有较严格的要求。上电时序如下，要保证 VCC 和 VCCA 先上电，然后 VCCAUX 及 VCCIO 再上电，等所有电源上电稳定之后，再经过 t_{CRESET_N} 才可以释放复位。

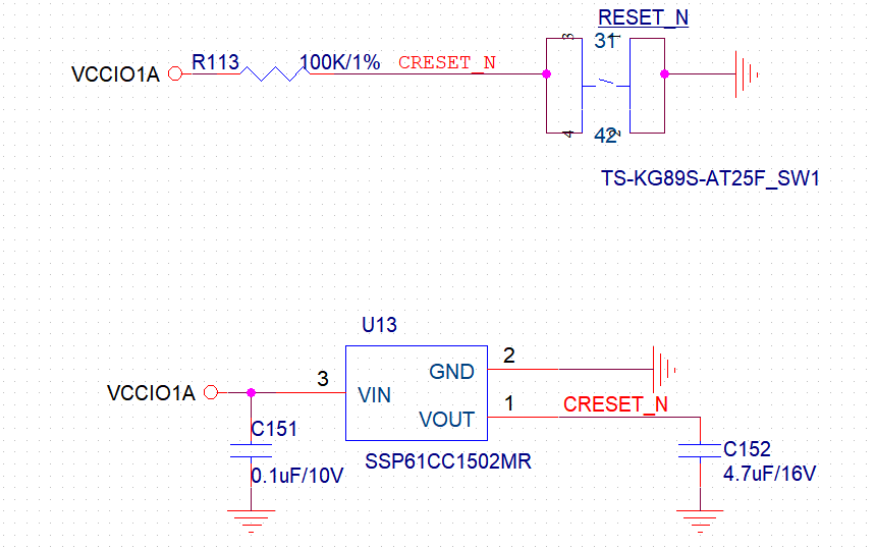




易灵思 FPGA 需要一个专门的复位，在各个电源稳定之后，在上升沿触发 FPGA 采集多功能引脚状态，保证 FPGA 的正确配置。手动复位可以让 FPGA 重新加载 flash 中的程序。

在 Ti60F225-DK 上我们使用一个电压检测芯片，用于检测 IO 电压；为了保证正确的复位时钟，在在电压检测芯片后配上了 RC 复位电源。

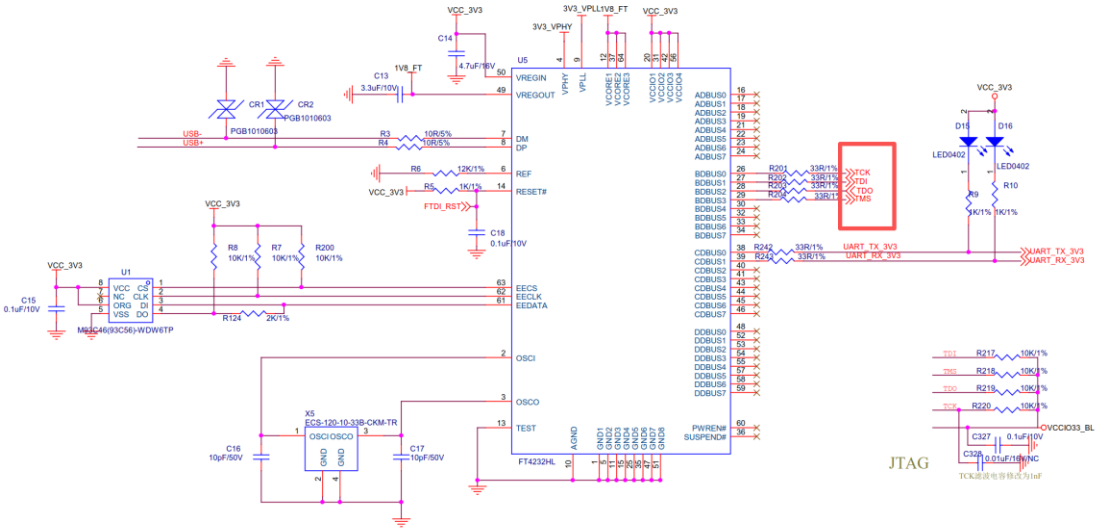
另外还支持手动复位。



2.7 JTAG 下载

2.7.1 板载下载器

Ti60F225-DK 上提供了板载下载器，下载芯片为 FT4232HL，所以客户无需外接下载器。这样既可以保证程序下载的稳定，也可以让连线的简洁：用户只需要一根 TypeC USB 线就可以实现供电、下载和 UART；同时也可以有效的降低客户的采购成本。

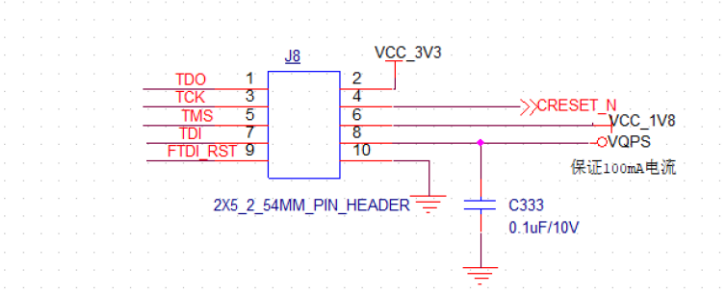


2.7.2 外接下载器

Ti60F225-DK 还预留了外接下载器的座子。操作方法如下：

（1） 把 J8 的 Pin 9(FTDI_RST)与 GND 短接，确认 FT4232DL 的 IO 处于高阻态。

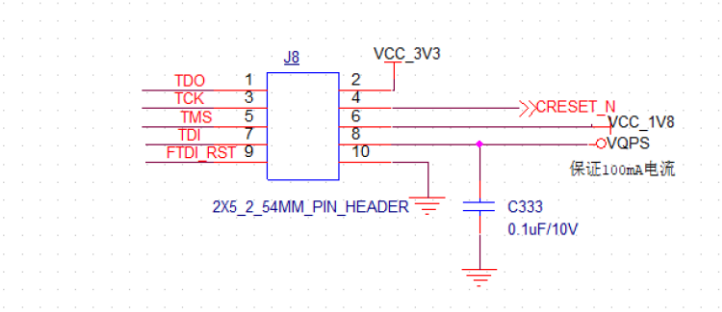
（2） 通过 J8 的 TDO,TCK,TMS,TDI,VCC_3V3 和 GND 与外部下载器连接。

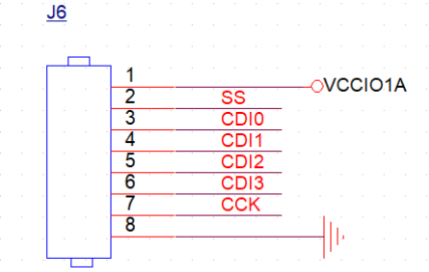


2.8 编程方式

Ti60F225-DK 默认通过连接 USB，然后通过 JTAG Bridge 编程 flash。但其实也可以去验证 AS 模式。操作如下：

（1） 把 J8 的 CRESET_N 连接 GND 或者手动按住 CRESET_N 按键；
（2） 通过外部下载器连接 J6 连接座了的 VCCIO1A,SS,CDI0,CDI1，CCK 和 GND；
（3） Programmer 中选择 Active mode；
（4） 如果上电时通过下拉 SS，确保其实为低电平，则跳过（3）选择 Passive Mode 验证 PS 模式。





2.9 加密验证

J8 座子还连接了 VQPS，如果需要验证加密可以查阅相关资料或者联系相关技术支持。

三、GE

Ti60F225-DK 使用的三速以太网芯片为 YT8531SC。

首先先简单介绍下 RGMII.

3.1、RGMII

RGMII(Reduced Gigabit Media Independant Interface),精简 GMII 接口。相对于 GMII 相比，RGMII 具有如下特征:

TX_ER 和 TX_EN 复用，通过 TX_CTL 传送

RX_ER 与 RX_DV 复用，通过 RX_CTL 传送

1 Gbit/s 速率下，时钟频率为 125MHz

100 Mbit/s 速率下，时钟频率为 25MHz

10 Mbit/s 速率下，时钟频率为 2.5MHz

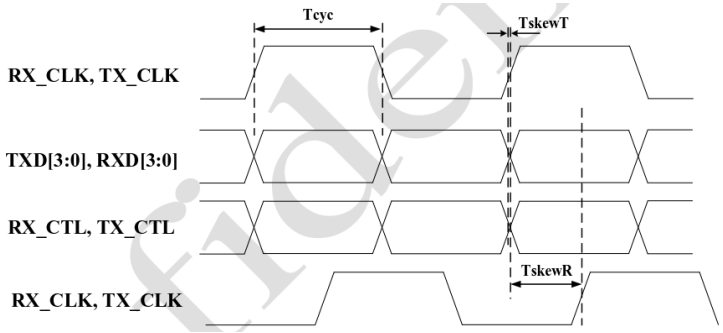
信号定义如下:

信号名称	描述	方向
TXC	发送时钟	MAC→PHY
TX_CTL	发送数据控制	MAC → PHY
TXD[3:0]	发送数据 4bit	MAC → PHY
RXC	接收时钟	PHY → MAC
RX_CTL	接收数据控制	PHY → MAC
RXD[3:0]	接收数据 4bit	PHY → MAC
MDIO	管理数据	双向
MDC	管理数据时钟	MAC → PHY

RGMII 的 TXC/RXC 时钟为 125Mhz，为了达到 1000Mbit 的传输速率，TXD/RXD 信号线在时钟上升沿发送接收 GMII 接口中的 TXD[3:0]/RXD[3:0]，在时钟下降沿发送接收 TXD[7:4]/RXD[7:4],并且信号 TX_CTL 反应了 TX_EN 和 TX_ER 状态，即在 TXC 上升沿发送 TX_EN,下降沿发送 TX_ER，同样的道理试用于 RX_CTL,

TX_ER（transmit coding error）：TX_ER 同步于 TX_CLK，在数据传输过程中，如果 TX_ER 有效超过一个时钟周期，并且此时 TX_EN 是有效的，则数据通道中传输的数据是无效的，没用的。注：当 TX_ER 有效并不影响工作在 10Mb/s 的 PHY 或者 TX_EN 无效时的数据传输。在 MII 接口的连线中，如果 TX_ER 信号线没有用到，必须将它下拉接地。

下图为发送接收的时序:



3.2、硬件设计

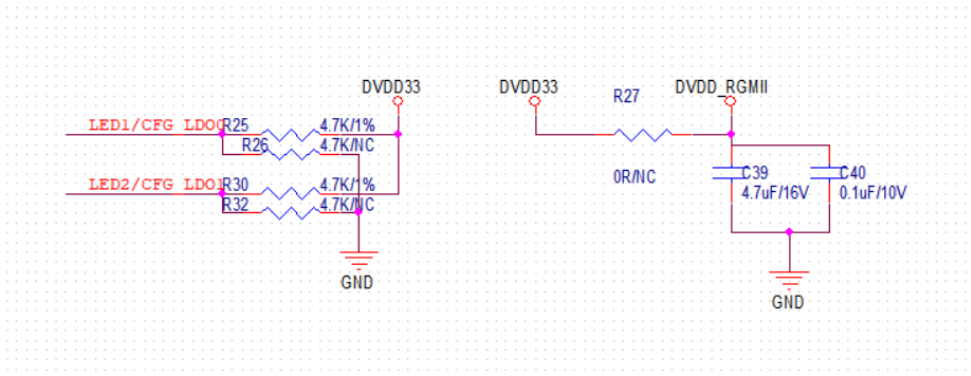
(1)CFG_LDO 处理

根据文档描述，如果 CFG_LDO[1:0]配置成 2'b00，DVDD_RGMII 的电源是由外部供电，其它则选择从内部 LDO 供电。并且配置为 2'b01 时 LDO 输出 2.5V，若配置为 2'b10 或者 2'b11 时 LDO 输出为 1.8V。

DVDD_RGMII 是用于 RGMII I/O, MDC/MDIO 的电源。

Table 14. CFG_LDO[1:0] Configuration

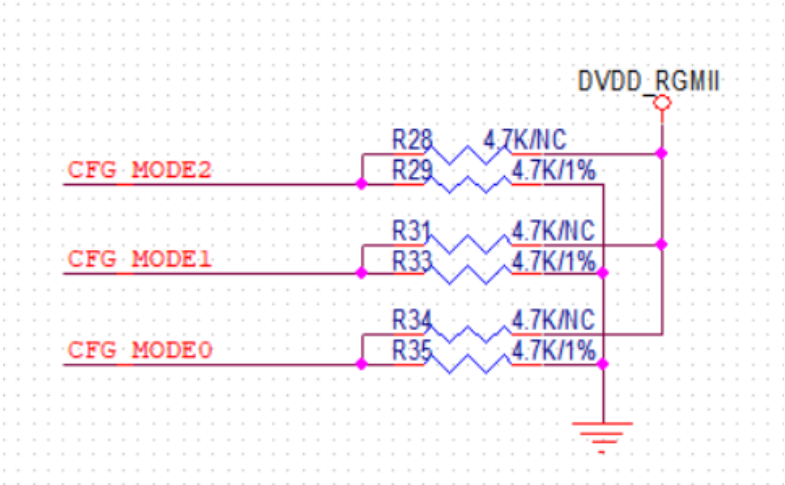
Configuration	Description
2'b01	LDO is set to 2.5V
2'b10 or 2'b11	LDO is set to 1.8V
2'b00	Use external 3.3V to supply to DVDD_RGMII pin. LDO is disabled



(2) CFG_MODE[2:0]处理

CFG_MODE0	O/LI/PD	CFG_MODE[2:0]: Operation Mode Configuration. 3'b000: UTP <-> RGMII 3'b001: FIBER <->RGMII 3'b010: UTP/FIBER <-> RGMII (Media Auto Detection) 3'b011: UTP <-> SGMII 3'b100: SGMII (PHY side) <-> RGMII (MAC side), 3'b101: SGMII (MAC side) <->RGMII (PHY side) 3'b110: UTP <->FIBER (Media Conversion auto mode) 3'b111: UTP <-> FIBER (Media Conversion force mode)
CFG_MODE1	O/LI/PD	
CFG_MODE2	O/LI/PD	

目前我们配置的是 UTP <-> RGMII 模式。



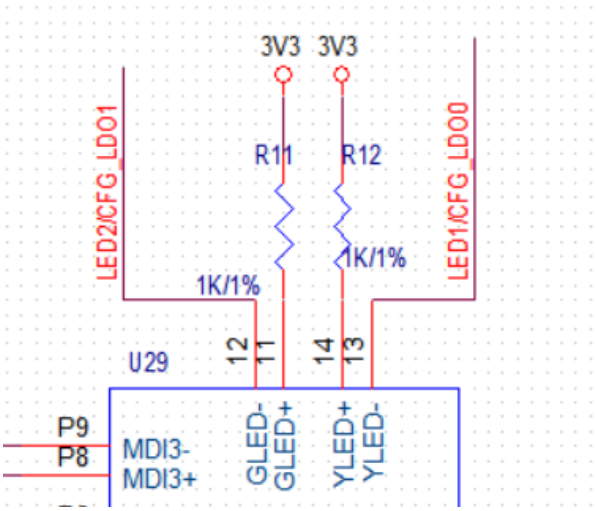
(2)LED 处理

按照文档，LED0,LED1 和 LED2 分别对应 10M,100M，1000Mbsp 数据速率。

Table 8. LED Default Settings

No.	Pin Name	Type	Description
35	LED0	O/LI/PU	Light = Link up at 10Mbps Blinking = Transiting or Receiving
36	LED1	O/LI/PU	Light = Link up at 100Mbps Blinking = Transiting or Receiving
37	LED2	O/LI/PD	Light = Link up at 1000Mbps Blinking = Transiting or Receiving

那在设计原则上，只连接了 100M 和 1000M，也就是在 100M 和 1000M 分别有对应的 LED 灯闪烁，两个灯都不闪烁则认为是 10M.



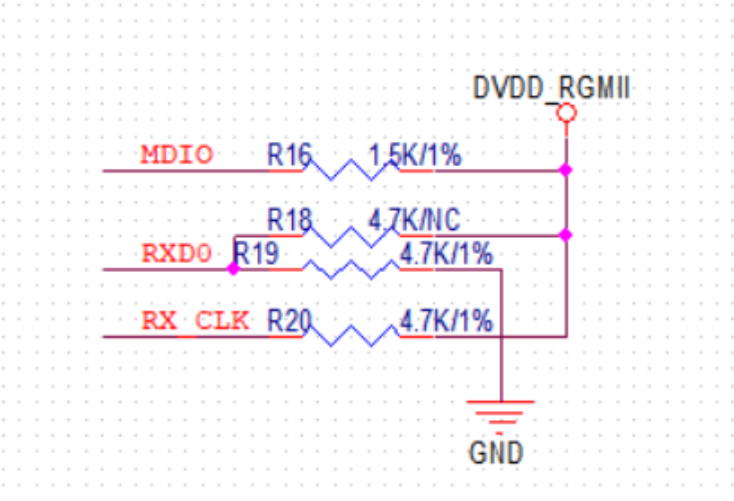
（4）PHYAD[2:0]处理

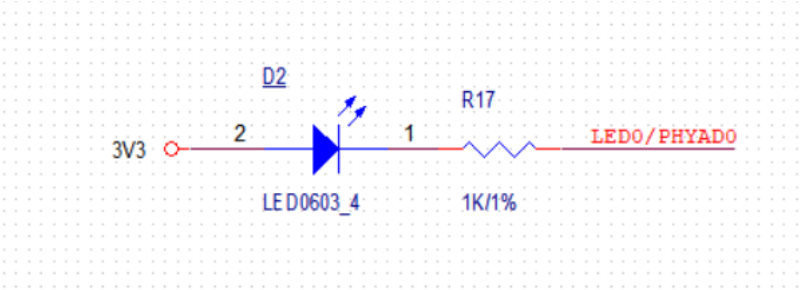
上电时通过 PHYAD[2:0]的电平来确定 PHY 地址。PHYAD 应该是用于 MDO。

PHYAD[2:0] 设置

这里我们把 PHYAD0 和 PHYAD1 设置成了 1，所以 PHY 地址是 00011b

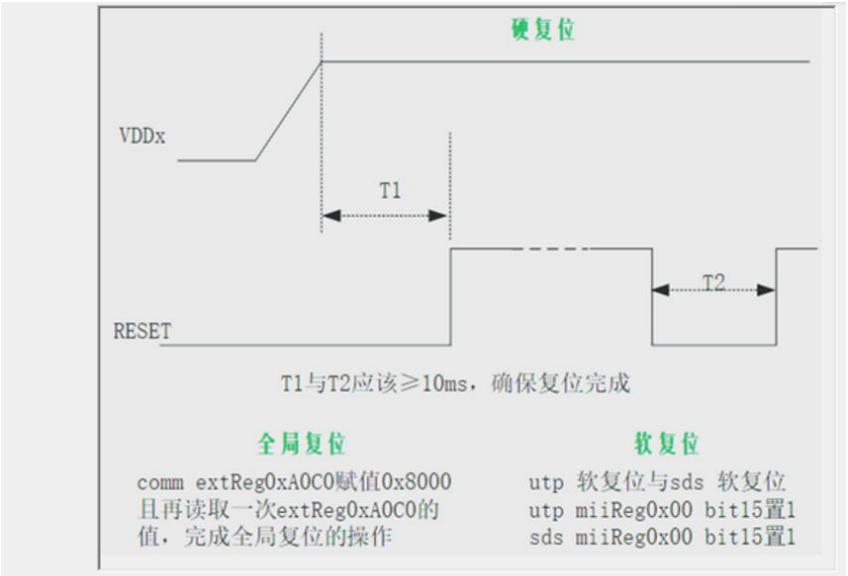
PHYAD0	O/LI/PU	PHYAD[2:0]. PHY address config
PHYAD1	O/LI/PD	
PHYAD2	O/LI/PD	





（5）复位

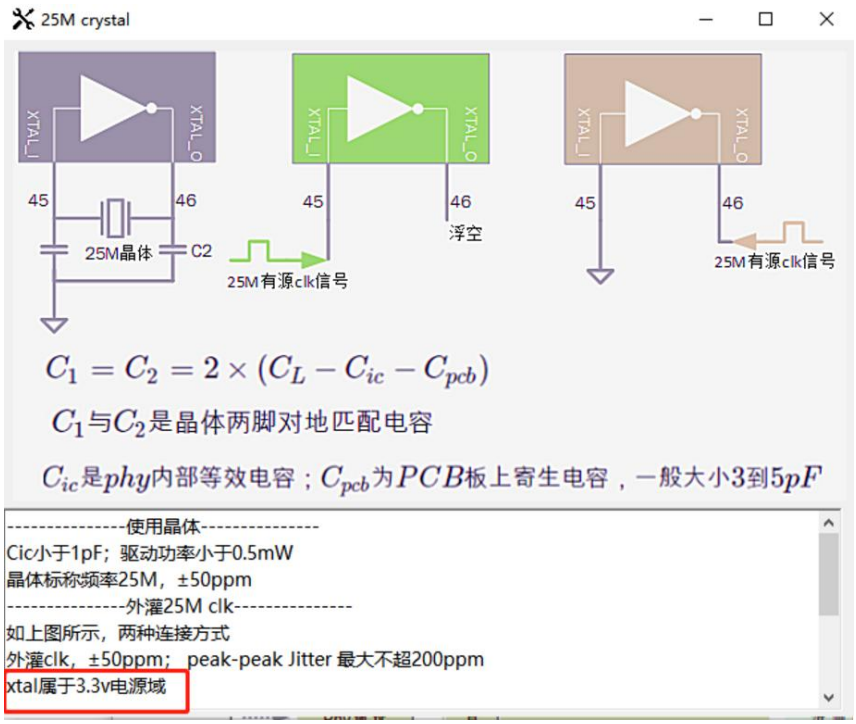
Pin 13 Reset 管脚属于 3.3V 电源域，这是需要注意的。




```
----------硬复位----------
Reset低有效，至少拉低10ms进行复位
Reset由低变高后15ms，再进行MDIO操作寄存器
pin13 Reset管脚属于3.3v电源域
----------全局复位----------
extReg0xA0C0赋值0x8000,且再读取一次0xA0C0，完成全局复位操作
全局复位相当于做了一次硬复位，仅仅是全局复位时不做POS，引用硬复位时记录的POS值
```

（6）晶振

xtal 属于 3.3V 电源域。所以选择 3.3V 无源晶振



(7) MDIO

MDIO 需要接上拉电阻，3.3V 上拉到 4.7k,1.8V 上拉 1.5k.

MDC 时钟频率最大是 12.5M

Idle->Start of Frame->OP Code->PHY Address->Register Address->Turnaround->Register Data-> Idle

Pin Name	Description
Start of Frame	01
OP Code	Read = 10 Write = 01
PHY Address	5-bit PHY Address
Register Address	5-bit Target Register Address
Turnaround	Read = z0 Write = 10
Register Data	16-bit Target Data

Figure 1 MDC/MDIO Read Timing

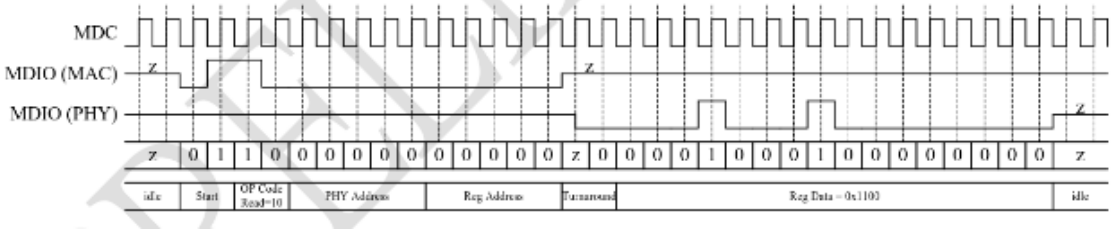
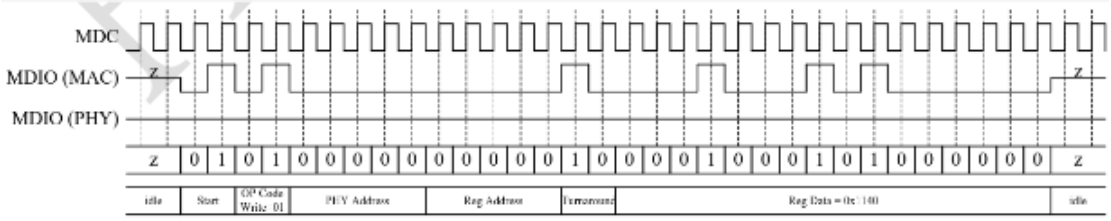


Figure 2 MDC/MDIO Write Timing





3.3 管脚分配

Net Name	FPGA Pin Name	FPGA Pin Num.	Bank 电压
MDIO	GPIOL_P_11	G5	1.8V
MDC	GPIOL_N_11	F5	
RX_CLK	GPIOL_P_09_CLK2_P	H1	
RX_CTL	GPIOL_N_08_CLK1_N	H3	
RXD0	GPIOL_N_07_CLK0_N	J1	
RXD1	GPIOL_N_06	K2	
RXD2	GPIOL_P_00_PLLIN0	P2	
RXD3	GPIOL_N_05	J3	
TX_CLK	GPIOL_P_07_CLK0_P	K1	

TX_CTL	GPIOL_N_00	R2	
TXD0	GPIOL_P_06	K3	
TXD1	GPIOL_P_05	K4	
TXD2	GPIOL_N_09_CLK2_N	G1	
TXD3	GPIOL_N_10_CLK3_N	G2	
PHY_RST_N	GPIOL_01	R3	3.3V

四、HDMI

xilinx 和 altera 都有通过 IO 驱动 HDMI 的方案，支持的分辨率各有不同，这跟不同系列的 FPGA 性能是相关的。

易灵思的钛金系列 FPGA 是采用 16nm 工艺的，属于高性能 FPGA，它支持 1.5G LVDS 及高速的 IO，这成为了实现 HDMI 自环的基础.

HDMI 接收过程

HDMI 接收端通过 50R 电阻上拉，再通过电容耦合连接到 FPGA。

HDMI 发送处理

与 LVDS 转 CML 的电平转换相似，只不过这里都使用了交流耦合。有时间再尝试 DC 耦合方式。

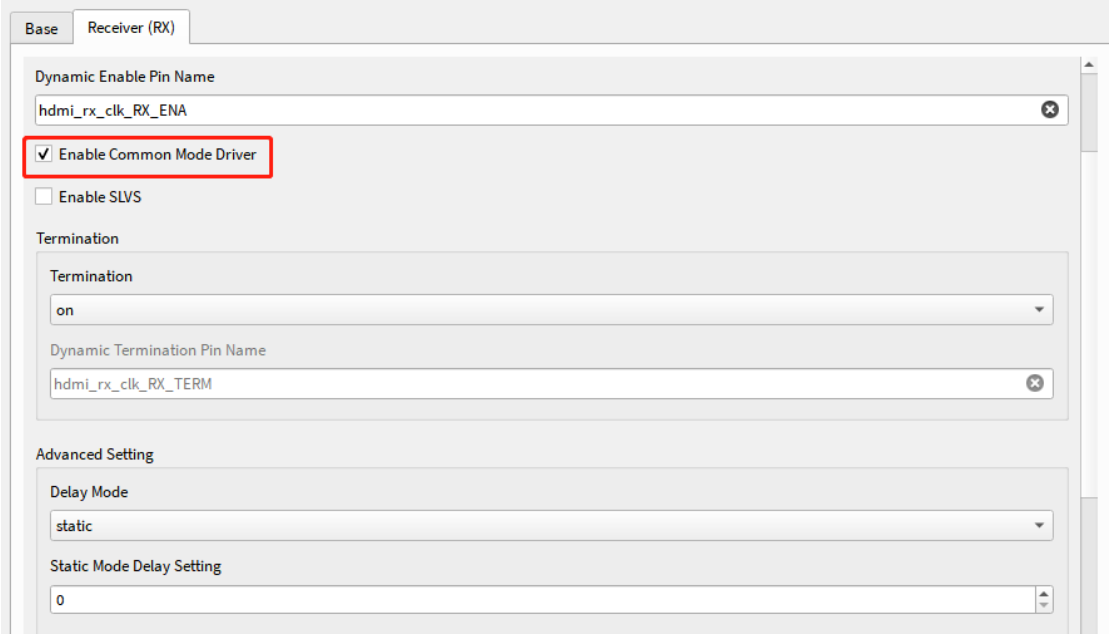
管脚分配

Net Name	PIN name	Pin num.	
HDMI_IN1_CP	GPIOR_N_19	C15	
HDMI_IN1_CN	GPIOR_P_19_PLLIN0	C14	
HDMI_IN1_P1	GPIOR_N_18	D14	
HDMI_IN1_N1	GPIOR_P_18	D15	
HDMI_IN1_P2	GPIOR_N_17	D13	
HDMI_IN1_N2	GPIOR_P_17	C13	
HDMI_IN1_P0	GPIOR_N_16	E14	
HDMI_IN1_N0	GPIOR_P_16	E15	
HDMI_RX_HPD_N0	GPIOR_N_15	E12	
HDMI_RX_5V_N0	GPIOR_P_15	F12	
HDMI_RX_SDA0	GPIOR_N_14	F11	
HDMI_RX_SCL0	GPIOR_P_14	G11	
TX_TMDS_DATA_P0	GPIOR_N_13	F15	

TX_TMDS_DATA_N0	GPIOR_P_13	G15	
TX_TMDS_DATA_P2	GPIOR_N_12	F13	
TX_TMDS_DATA_N2	GPIOR_P_12	G13	
TX_TMDS_DATA_P1	GPIOR_N_11_CLK8_N	G14	
TX_TMDS_DATA_N1	GPIOR_P_11_CLK8_P	H14	
TX_TMDS_CLK_P	GPIOR_N_10_CLK9_N	H12	
TX_TMDS_CLK_N	GPIOR_P_10_CLK9_P	H13	

注意：FPGA 差分对 P 与 N 脚对调。

HDMI 设置要勾选 Enable Common Mode Driver



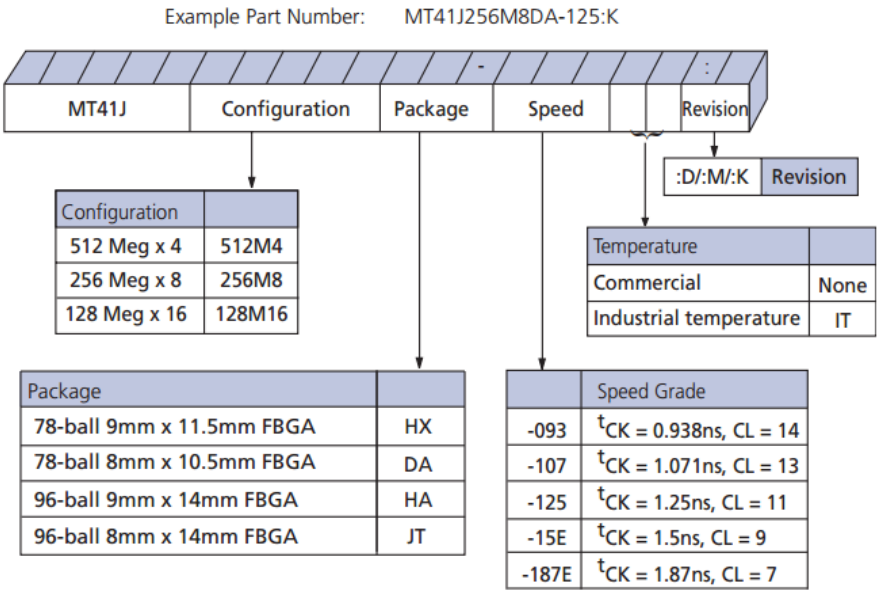
五、DDR3

Ti60F225-DK 板上配有一片 DDR3 采用的是 MT41J128M16JT-125，位宽为 16 位，数据速率最高 1600Mbps，容量为 2Gb，一般可以满足各种高带宽和大存储容量需求。下面我们先简单介绍下该 DDR3。

5.1 DDR3 简介

1、DDR 的容量大小

先看下 micron 公司对 DDR3 命名的规则：



1、meg 的含义：

内存中 Meg 的含义：Meg 就是兆的含义，即 1000,000.

MT41J128M16 – 16 Meg x 16 x 8 Banks 的含义：

表示方法是：每个逻辑 BANK 的单元格数×每个单元格的位数(芯片数据位宽)×逻辑 BANK 数量，即每个逻辑 BANK 的单元格数为 16 兆，每个单元格的数据位是 16bit，逻辑 BANK 的数量为 8 个。这样 MT41J128M16 就是 256MB(2Gb)的容量。

2、速率等级

下面是镁光器件的速率等级划分：

Table 1: Key Timing Parameters

Speed Grade	Data Rate (MT/s)	Target ^{t}RCD-^{t}RP-CL	^{t}RCD (ns)	^{t}RP (ns)	CL (ns)
-093[1, 2, 3, 4]	2133	14-14-14	13.09	13.09	13.09
-107[1, 2, 3]	1866	13-13-13	13.91	13.91	13.91
-125[1, 2,]	1600	11-11-11	13.75	13.75	13.75
-15E[1,]	1333	9-9-9	13.5	13.5	13.5
-187E	1066	7-7-7	13.1	13.1	13.1

Notes:
1. Backward compatible to 1066, CL = 7 (-187E).
2. Backward compatible to 1333, CL = 9 (-15E).
3. Backward compatible to 1600, CL = 11 (-125).
4. Backward compatible to 1866, CL = 13 (-107).

Speed 等级高的可以兼容等级低的芯片。

3、DDR3 的芯片地址位宽

Parameter	512 Meg x 4	256 Meg x 8	128 Meg x 16
Configuration	64 Meg x 4 x 8 banks	32 Meg x 8 x 8 banks	16 Meg x 16 x 8 banks
Refresh count	8K	8K	8K
Row addressing	32K (A[14:0])	32K (A[14:0])	16K (A[13:0])
Bank addressing	8 (BA[2:0])	8 (BA[2:0])	8 (BA[2:0])
Column addressing	2K (A[11, 9:0])	1K (A[9:0])	1K (A[9:0])
Page size	1KB	1KB	2KB

上面的三个不同地址位宽的芯片内存容量都是 256MB，当选定了芯片的数据位宽和容量大小之后，地址的位宽在 DATASHEET 上是可以找到的。从上图可以看到 MT41J128M16JT-125 的 Row 地址是 14 位，Column 地址位宽是 10 位，bank 地址位宽是 3 位。

DDR3 的 PCB 的硬件设计需要考虑数据线和地址控制线的等长，保证 DDR3 的高速稳定的工作。

5.2 Ti60F225 对 DDR3 的支持

易灵思的 IO 是支持 SSTL 和 Differential SSTL 电平标准的，这是能支持 DDR3 的硬件基础。另外易灵思在 Ti60F225 的 DDR3 控制器是软核的，数据速率最高是 800Mbps。由前面 2.3 的介绍可知，该 DDR3 直接连到了 FPGA 的 3B,4A,4B bank 上。

六、预留 IO

6.1 简介

预留 IO 用于用户扩展，当然还有一个主要的用处是用于 MIPI 和 LVDS 的接收发送。预留的两个座子分别是 J7.

（1） 板上预留的 HSIO 最多支持三组 MIPI，每组支持 4lane。
（2） HSIO 还可以用于 LVDS 传输，最大 15 对差分对。
（3）每个座子上预留了 5V 和 3V 电源，方便外接子卡供电。

（4）TR 和 TL bank 的 HVIO 连接到扩展座子上，可以根据外设需要设置成 3.3V 和 1.8V。

6.2 扩展口 J7 引脚分配

CONN Num.	Net Name	FPGA Pin Name	FPGA pin Num.	Bank 电压
01	VCC_3V3			
02	LVDS	GPIOL_P_14	F1	1.8V
03	VCC_5V			
04		GPIOL_N_14	F2	1.8V
05	GND			
06	GND			
07	MIPI_D0_P	GPIOT_P_00_PLLIN1	E6	1.8V/1.2V
08		GPIOL_P_13	F3	1.8V
09	MIPI_D0_N	GPIOT_N_00	D5	1.8V/1.2V
10	GPIOL_N_13	GPIOL_03	E3	1.8V
11	GND			
12	GND			
13	MIPI_D1_P	GPIOT_P_01	B6	1.8V/1.2V
14	LVDS_P4	GPIOL_P_16	D1	1.8V
15	MIPI_D1_N	GPIOT_N_01	C6	1.8V/1.2V
16	LVDS_N4	GPIOL_N_16	C1	1.8V
17	GND			
18	GND			
19	MIPI_CK_P	GPIOT_P_03	A5	1.8V/1.2V
20	LVDS_P3	GPIOL_P_17	B1	1.8V
21	MIPI_CK_N	GPIOT_N_03	A6	1.8V/1.2V
22	LVDS_N3	GPIOL_N_17	C2	1.8V
23	GND			
24	GND			
25	MIPI_D3_P	GPIOT_P_05	E7	1.8V/1.2V
26	LVDS_P2	GPIOL_P_18	B2	1.8V
27	MIPI_D3_N	GPIOT_N_05	D7	1.8V/1.2V
28	LVDS_N2	GPIOL_N_18	A2	1.8V
29	GND			
30	GND			
31	MIPI_D2_P	GPIOT_P_04	F7	1.8V/1.2V
32	GPIOL_11	GPIOL_11	C5	3.3V
33	MIPI_D2_N	GPIOT_N_04	F8	1.8V/1.2V
34	GPIOL_09	GPIOL_09	A4	3.3V

35	GND			
36	GND			
37	GPIOL_04			3.3V
38	GPIOL_10			3.3V
39	GPIOL_06			3.3V
40	GPIOR_13			3.3V

6.3 拓展子卡

目前 XL 提供了两种子卡，一种转 FPC，一种转 2.54 排针。方便 MIPI LVDS 使用。

正面



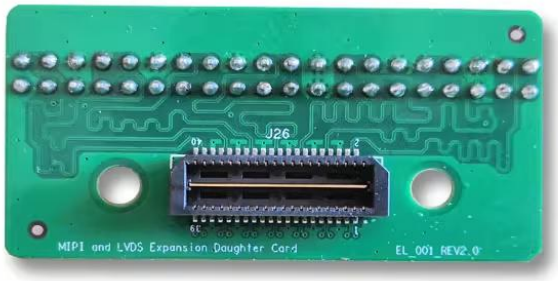
反面



正面

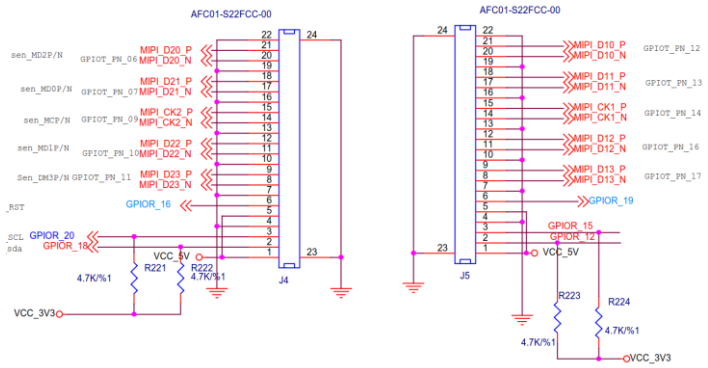


反面



七 MIPI 接口

Demo 板有两个 FPC 座子，用于连接 MIPI 屏和 sensor



7.1 SOFT MIPI 硬件设计原则

初识易灵思可能有些点要注意。这里我们说明下钛金系列 MIPI 的设计规则 。

钛金系列 FPGA 支持 CSI RX/TX 和 DSI TX,速率最高能达到 1.5Gbps(不同速率等级的器件是有差异的要具体确认。)

HSIO	MIPI lane	C4, I3, Q3	–	1,500
		C3	–	1,300
		I3L, C4L	–	1,250
		C3L	–	1,100

（1）MIPI 的 bank 电压是 1.2V 的。

（2）一定要参考 pin out 文件。这里以 Ti60F225 为例。如下图。

pin out 文件 MIPI RX Function 一栏就是 MIPI RX 可以使用的情况。

对于 MIPI RX 信号，数据定义为 RX_DATA_P/Nx_Iy，x 是指每个组中的数据线信号编号。y 是指 MIPI 组，每个组是 4lane 或者 8lane 的，如下图第 8 组就是 4lane MIPI 组，包括 5 个差分对；第 7 组就是 8lane MIPI 组，包括 9 个差分对。

每组 MIPI 中的时钟叫 RX_CLK_Iy，它的位置是固定的。但数据是可以交换的。

（3）对于 MIPI TX 则没有太多要求，只要放在 MIPI bank 即可。

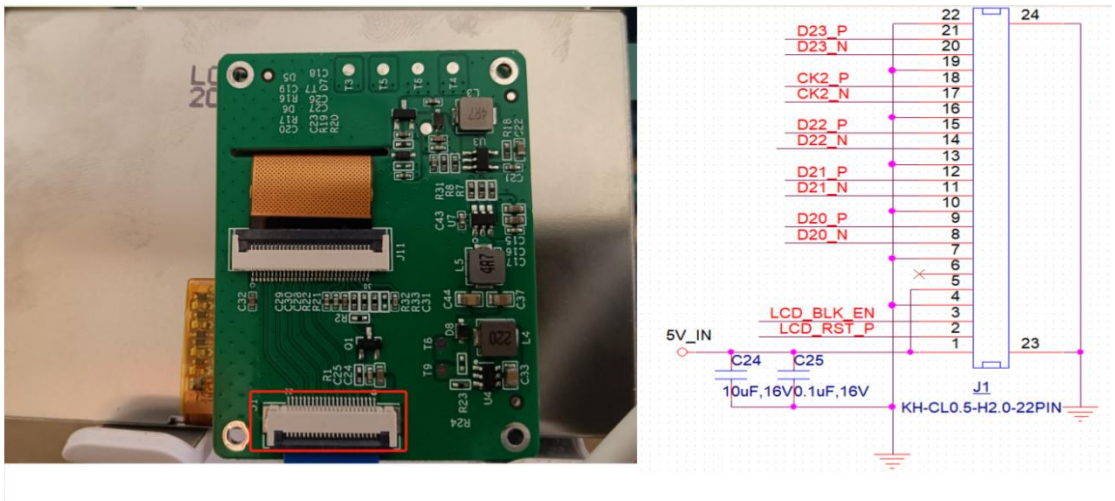
(4)当使用 HSIO 为 GPIO,LVDS 或者 MIPI 时，要确保 GPIO，LVDS 和 MIPI 之间要隔出一对未分配的 IO，这样可以减少开关噪声，如果没有隔开的话软件也会提示错误。

Bank Number	Pin Names	User Function	Config Function	Emulated MIPI RX Functi	VREF Function for HSTL/SSTL
BANK3A	REF_RES_3A				
BANK3A	GPIOR_N_45			RX_DATA_N3_I6	
BANK3A	GPIOR_P_45_PLLIN0	PLL_TR_IN0		RX_DATA_P3_I6	
BANK3A	GPIOR_N_44			RX_DATA_N2_I6	
BANK3A	GPIOR_P_44_EXTFB	PLL_TR_EXTFB		RX_DATA_P2_I6	
BANK3A	GPIOR_N_43			RX_CLK_N_I6	
BANK3A	GPIOR_P_43			RX_CLK_P_I6	
BANK3A	GPIOR_P_42			RX_DATA_P1_I6	
BANK3A	GPIOR_N_42			RX_DATA_N1_I6	
BANK3A	GPIOR_N_41			RX_DATA_N0_I6	
BANK3A	GPIOR_P_41			RX_DATA_P0_I6	VREF3A
BANK3A	GPIOR_N_40			RX_DATA_N7_I7	
BANK3A	GPIOR_P_40			RX_DATA_P7_I7	
BANK3A	GPIOR_N_39			RX_DATA_N6_I7	
BANK3A	GPIOR_P_39			RX_DATA_P6_I7	
BANK3A	GPIOR_P_38			RX_DATA_P5_I7	
BANK3A	GPIOR_N_38			RX_DATA_N5_I7	
BANK3A	GPIOR_N_37			RX_DATA_N4_I7	
BANK3A	GPIOR_P_37			RX_DATA_P4_I7	
BANK3A	GPIOR_N_36			RX_CLK_N_I7	
BANK3A	GPIOR_P_36			RX_CLK_P_I7	
BANK3A	GPIOR_P_35			RX_DATA_P3_I7	
BANK3A	GPIOR_N_35			RX_DATA_N3_I7	
BANK3A	GPIOR_N_34			RX_DATA_N2_I7	
BANK3A	GPIOR_P_34			RX_DATA_P2_I7	
BANK3A	GPIOR_N_33			RX_DATA_N1_I7	
BANK3A	GPIOR_P_33			RX_DATA_P1_I7	
BANK3A	GPIOR_P_32			RX_DATA_P0_I7	
BANK3A	GPIOR_N_32			RX_DATA_N0_I7	

7.2 Pannel 接口与 FPC 的连接

另外我们也提供了相应的 MIPI 屏。

MIPI 屏背板的 FPC 连接

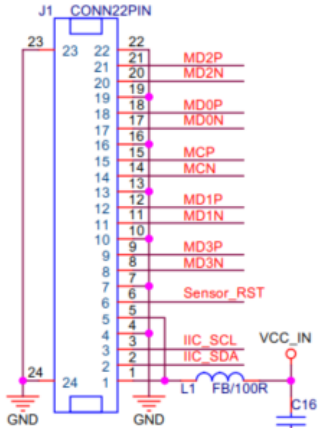


panel 接口与 FPC 对应关系

panel	FPC.Pin	J4	J5
GND	22		
D23P	21	GPIOT_P_06	GPIOT_P_12
D23N	20	GPIOT_N_06	GPIOT_N_12
GND	19		
CK2P	18	GPIOT_P_07	GPIOT_P_13
CK2N	17	GPIOT_N_07	GPIOT_N_13
GND	16		
D22_P	15	GPIOT_P_09	GPIOT_P_14
D22_N	14	GPIOT_N_09	GPIOT_N_14
GND	13		
D21_P	12	GPIOT_P_10	GPIOT_P_16
D21_N	11	GPIOT_N_10	GPIOT_N_16
GND	10		
D20_P	9	GPIOT_P_11	GPIOT_P_17
D20_N	8	GPIOT_N_11	GPIOT_N_17
GDN	7		
NC	6	GPIOR_16(3.3V)	GPIOR_19(3.3V)
5V	5		
	4		
EN	3	GPIOR_20(3.3V)	GPIOR_15(3.3V)
RST_P	2	GPIOR_18(3.3V)	GPIOR_12(3.3V)
5V	1		

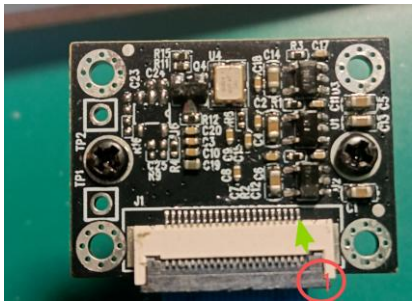
7.3 Sensor 接口与 FPC 的连接

Sensor 接口与 QSE 接口的对应关系



问：Sensor 的 1 脚是那个?

答：如下图



sensor 接口与 FPC 对应关系

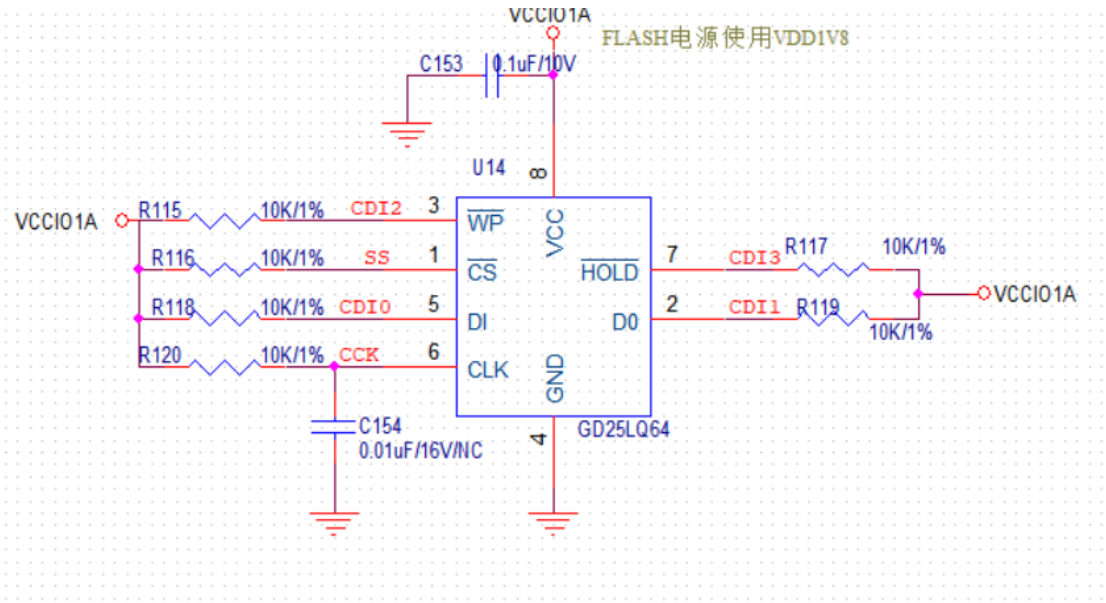
sensor	FPC.Pin	J4	J5
	22		
MD2P	21	GPIOT_P_06	GPIOT_P_12
MD2N	20	GPIOT_N_06	GPIOT_N_12
	19		
MD0P	18	GPIOT_P_07	GPIOT_P_13
MD0N	17	GPIOT_N_07	GPIOT_N_13
	16		
MCP	15	GPIOT_P_09	GPIOT_P_14

MCN	14	GPIOT_N_09	GPIOT_N_14
	13		
MD1P	12	GPIOT_P_10	GPIOT_P_16
MD1N	11	GPIOT_N_10	GPIOT_N_16
	10		
MD3P	9	GPIOT_P_11	GPIOT_P_17
MD3N	8	GPIOT_N_11	GPIOT_N_17
	7		
RST	6	GPIOR_16(3.3V)	GPIOR_19(3.3V)
5V	5		
	4		
IIC_SCL	3	GPIOR_20(3.3V)	GPIOR_15(3.3V)
IIC_SDA	2	GPIOR_18(3.3V)	GPIOR_12(3.3V)
5V	1		

七、Flash

Flash 用于存储 FPGA 的程序，也可以用于用户数据存储。Ti60F225-DK 配置了一片 GD25LQ64 的 flash 芯片。

注意 FLASH 的第 6 脚 CLK 上一般不需要预留电容。



八、用户 IO

Ti60F225-DK 上配置了 4 个 LED 和两个按键。

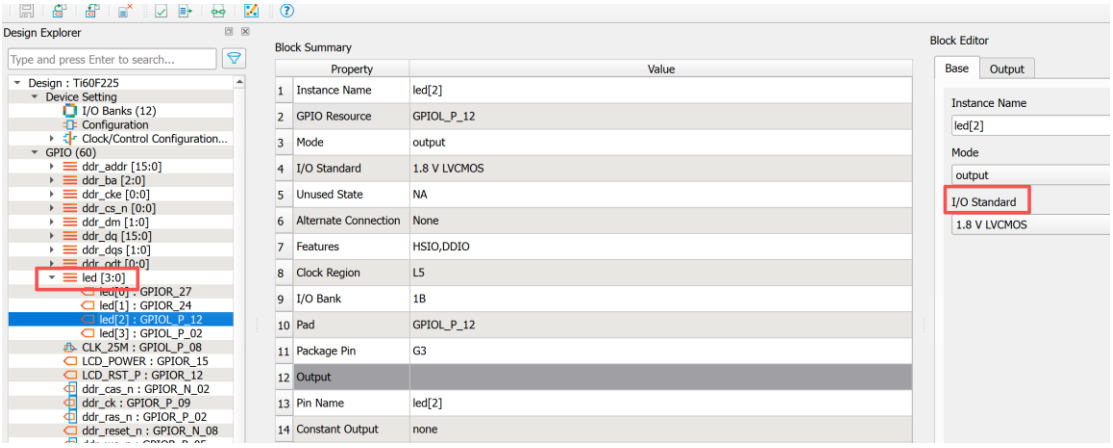
按键管脚分配

按键名	FPGA Pin Name	FPGA Pin Num.	Bank	Bank 电压
KEY0	GPIOL_07	C4	TL	3.3V
KEY1	GPIOR_22	P14	BR	3.3V
KEY2	GPIOR_21	N14	BR	3.3V
KEY3	GPIOL_03	A3	TL	3.3V

LED 管脚分配

LED 名	FPGA Pin Name	FPGA Pin Num.	Bank	Bank 电压
LED0	GPIOR_27	P13	BR	3.3V
LED1	GPIOR_24	N13	BR	3.3V
LED2	GPIOL_P_12	G3	1B	1.8V
LED3	GPIOL_P_02_CSI	N2	1A	1.8V
LED4	GPIOL_N_12	G4	1B	1.8V
LED5	GPIOL_P_15	E1	1B	1.8V
LED6	GPIOL_N_15	E2	1B	1.8V
LED7	GPIOL_N_02	M2	1A	1.8V

注意 LED0~1 为 3.3V，而其余 LED 为 1.8V，修改电平方法如下：



九、加载模式验证管脚

Demo 板预留了 J6 排针座。

J6	FPGA Pin Name	FPGA Pin Num.	FPGA P	Bank 电压
J6.1	VCCIO1A	-	-	1.8V
J6.2	GPIOL_P_01_SSL_N	P1	1A	1.8V
J6.3	GPIOL_P_03_CDI0	M1	1A	1.8V
J6.4	GPIOL_N_03_CDI1	L1	1A	1.8V
J6.5	GPIOL_P_04_CDI2	H5	1A	1.8V
J6.6	GPIOL_N_04_CDI3	H4	1A	1.8V
J6.7	GPIOL_N_01_CCK	N1	1A	1.8V
J6.8	GND	-	-	-

十、UART

FT4232

SIGNAL	FPGA Name	FPGA Num	说明
UART_RX	GPIOL_02	R4	UART_RX：FPGA 为 input；
UART_TX	GPIOR_28	R14	UART_TX：FPGA 为 output；

表 8

软件篇

学习入门

我们在微信公众号上提供很多关于易灵思的学习资料，按住 Ctrl 并点击即可打开链接。

Efinity 软件安装-v5

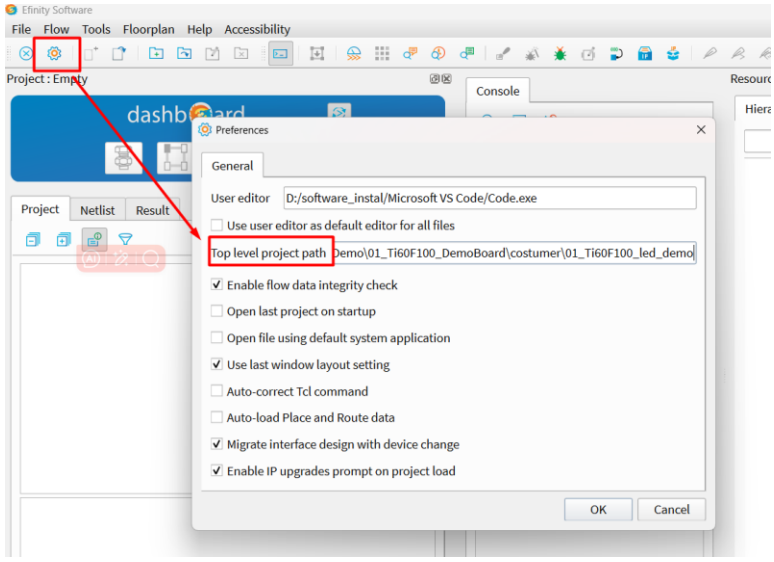
易灵思下载器驱动安装说明

Efinity 入门使用-v8

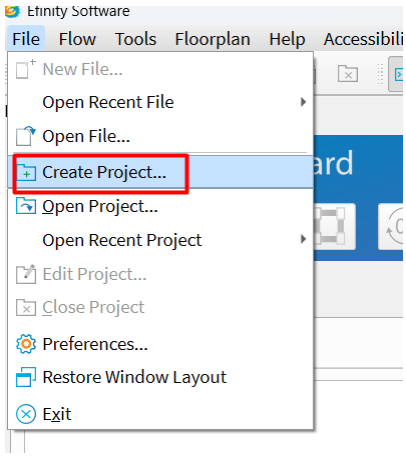
InterfaceDesinger 使用案例-v4

01_Ti60F225_Key_led_osc_demo

打开 Efinity，设置 Preferences。Top Level project path 指向(1)新建的工程目录。



点击 File-> Create Project...



在 Create New Project 面板中设置相关参数，器件由芯片来决定。

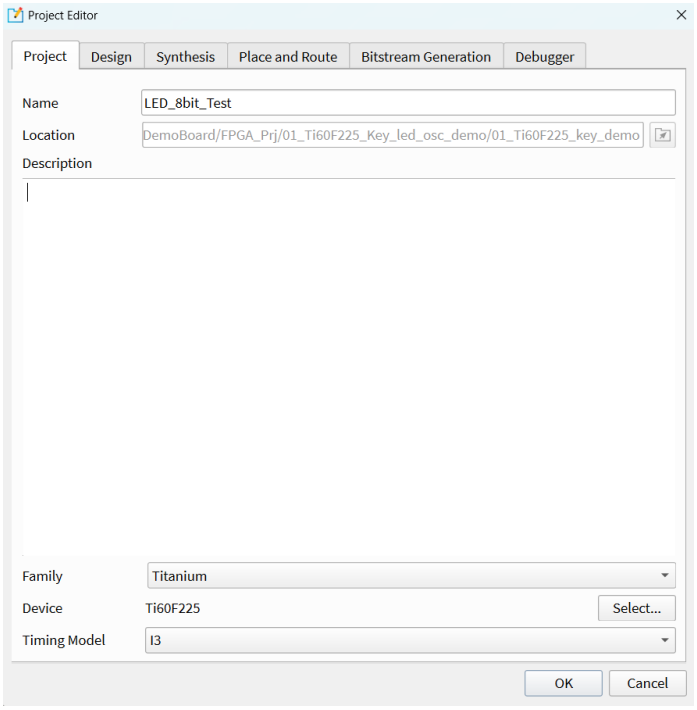
（1）工程名“Name”,

（2）指定工程所在的位置“Location”,

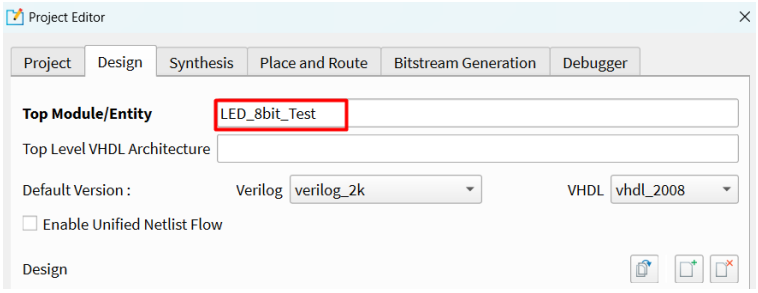
（3）“Family”设置器件家族“Tianium”

（4）Device 设置为“Ti60F225”

（5）Timing Mode 设置为 I3

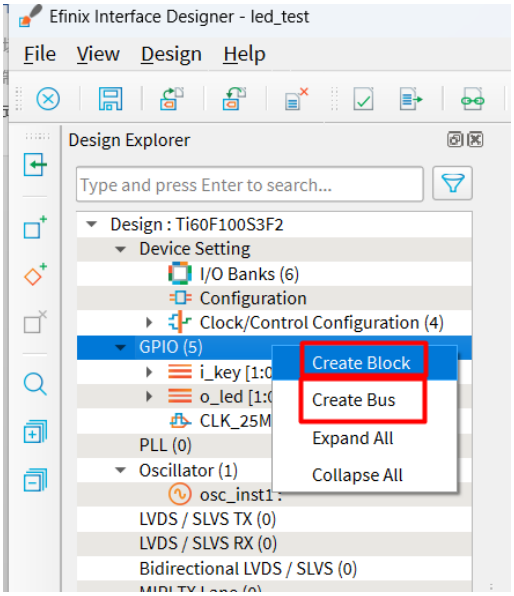


在 Design 中设置工程的 Top Module 名字。这里指定为 LED_8bit_Test,注意，必须要指定 Top Module/Entity，不然软件可能自动推断 top Module 以至程序编译出现不可预知的现象。

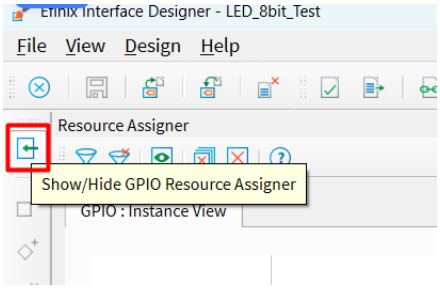


(2)添加 IO

关于 IO 的添加请参考公众号 *Efinity 入门使用-v8*

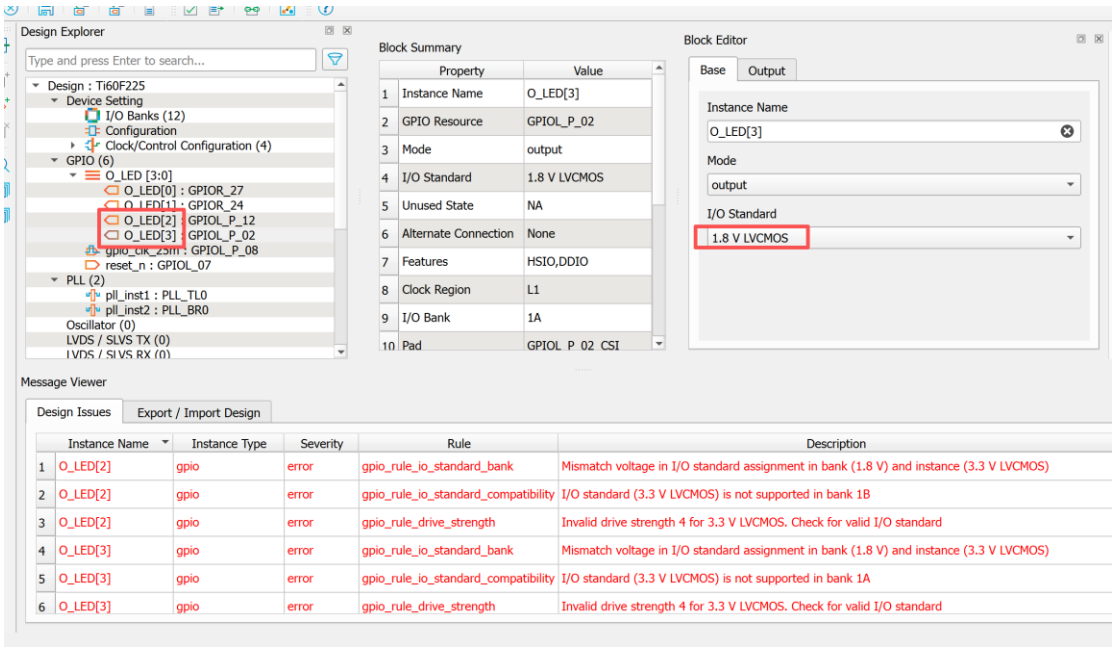


点击 show/Hide GPIO Resource Assigner 可以查看 IO 的分配。



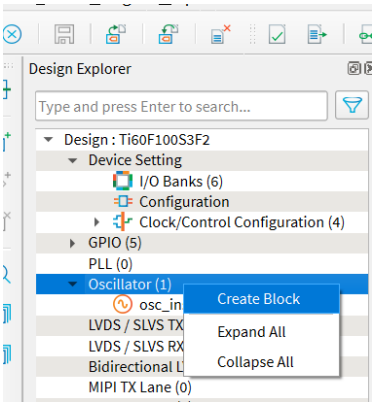
GPIO : Instance View

			all	all	all	all	
Instance	Package Pin	Resource	I/O Bank	Alt Conn	Features	Clock Region	Pad
clk_sel	A2	GPIOL_N_18	1B	None	HSIO,DDIO	L7	GPIOL_N_18
gpio_clk_25m	J2	GPIOL_P_08	1A	GCLK,RCLK	HSIO,DDIO	L3	GPIOL_P_08_CLK1_P
gpio_clk_27m	B2	GPIOL_P_18	1B	PLL_CLKIN	HSIO,DDIO	L7	GPIOL_P_18_PLLIN0
gpio_clk_50m	R13	GPIOR_29	BR	PLL_CLKIN	GPIO,DDIO	B1	GPIOR_29_PLLIN2
key_i[0]	P13	GPIOR_27	BR	None	GPIO,DDIO	B1	GPIOR_27
key_i[1]	N13	GPIOR_24	BR	None	GPIO,DDIO	B1	GPIOR_24
led_data[0]	N14	GPIOR_21	BR	None	GPIO,DDIO	B1	GPIOR_21
led_data[1]	P14	GPIOR_22	BR	None	GPIO,DDIO	B1	GPIOR_22
led_data[2]	R14	GPIOR_28	BR	None	GPIO,DDIO	B1	GPIOR_28
led_data[3]	P15	GPIOR_25	BR	None	GPIO,DDIO	B1	GPIOR_25

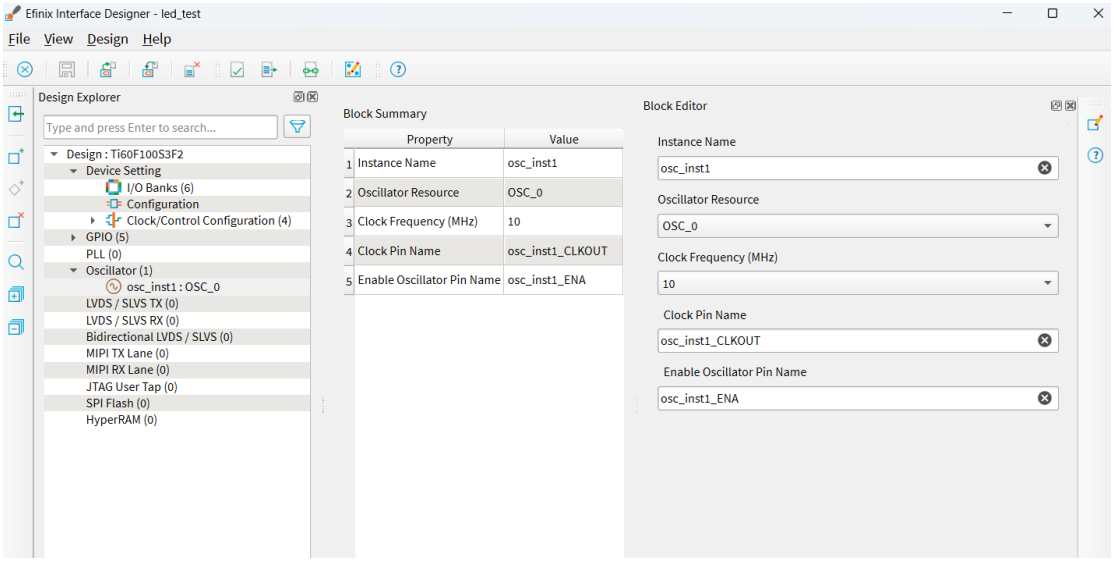


(3)添加片上晶振

钛金系列 FPGA 有一颗片上晶振，可以在 interface 中进行添加。在 interface 中选择 Osillator，右击选择 Create Block，即可添加一个片上晶振,并且在右侧同时弹出对应的 Block Editor 界面。

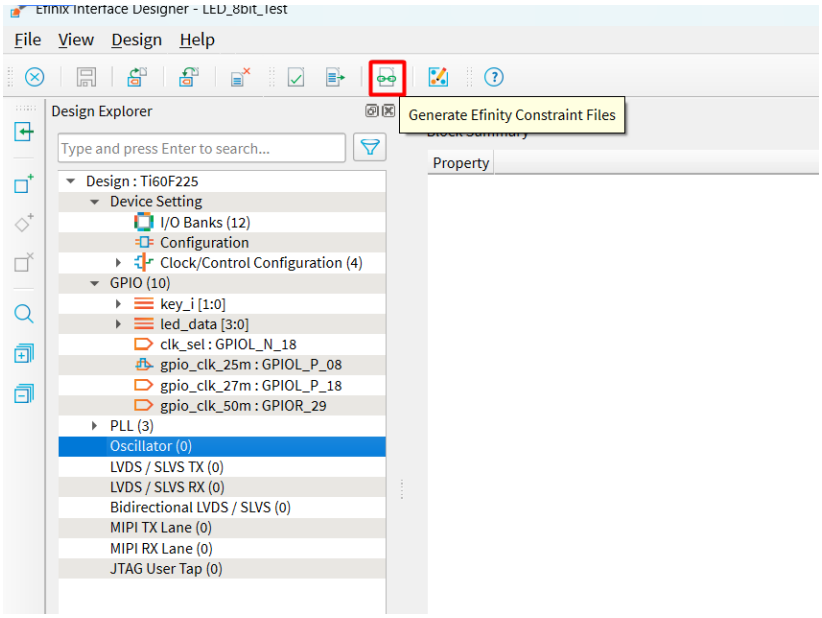


下图右侧为 Osillator 的 Block Editor。可以根据需要修改 Instance Name，当然也可以选择默认的例化名；Oscillator Resource 选择 OSC_0；Clock Frequency(MHz)为片上晶振的时钟频率选择，支持 10，20，40，80MHz 共 4 种频率，下面默认选择为 10MHz；Clock Pin Name 和 Enable Oscillator Pin Name 也可以根据需要来修改，下图选择默认。

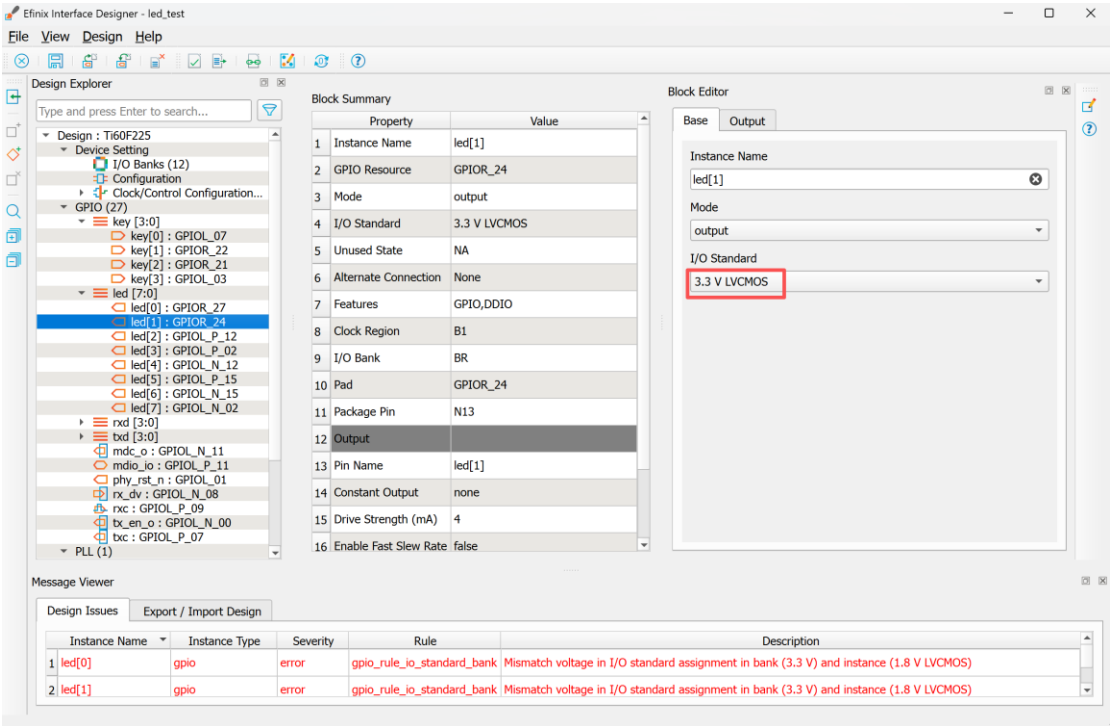


（4）生成约束。

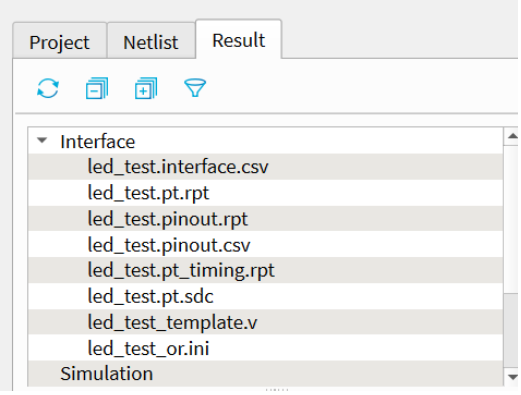
在 interface 中一切配置好之后，点击 Generate Efinity Constraint Files 生成相关约束文件。在控制面板的 Result 目录下会显示生成的相关文件。



注意 LED[1:0]是 3.3V 的，而其它 IO 是 1.8V 的，需要在下面位置单独修改。



或者在控制面板的 Result 中查看 interface 生成的文件。这里有常用到的文件是<prj>.pt.sdc 和<prj>_template.v 文件。其中<prj>.pt.sdc 文件用于给工程添加约束；而<prj>_template.v 文件提供了接口例化信号，用于实现 interface 与 core 逻辑之间的互连的，用户只要把 template.v 文件中的信号复制到 Top Module 即可。



（5） 添加顶层信号

把上面 led_test_template.v 文件中的信号复制到顶模块。信号如下：

```
module LED_8bit_Test
(
  (* syn_peri_port = 0 *) input gpio_clk_50m,
  (* syn_peri_port = 0 *) input [3:0] key_i,
  (* syn_peri_port = 0 *) input sys_pll_LOCKED,
  (* syn_peri_port = 0 *) input clk_50m,
  (* syn_peri_port = 0 *) input gpio_clk_27m,
  (* syn_peri_port = 0 *) input clk_54m,
  (* syn_peri_port = 0 *) input gpio_clk_25m,
  (* syn_peri_port = 0 *) input clk_25m_x2,
  (* syn_peri_port = 0 *) input osc_clk,
  (* syn_peri_port = 0 *) output [7:0] led_data,
  (* syn_peri_port = 0 *) output sys_pll_RSTN,
  (* syn_peri_port = 0 *) output osc_inst1_ENA
);
```

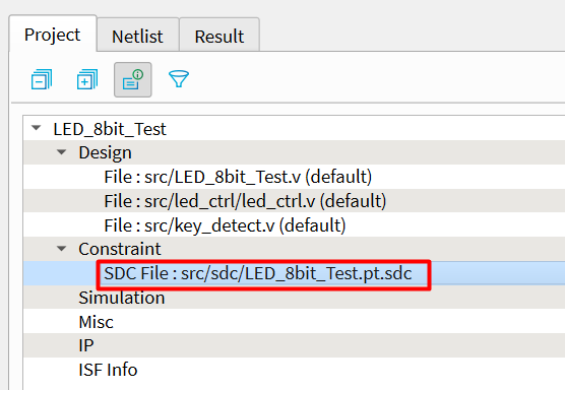
（5）添加约束

在 outflow 文件夹下，会生成 xxx.pt.sdc 文件，内容如下，我们需要把 sdc 文件复制出来放到别的位置，这里我们放在了工程目录下。我们可以看到 osc_inst1_CLKOUT 的约束软件是可以自动识别的，但 **gpio_clk_25m** 由于是外部时钟输入所以不能自动识别频率，但是也提供了相应的约束参考，所以我们进行了手动约束。

```
# PLL Constraints
##################
create_clock -period 10.0000 clk_50m
```

```
create_clock -period 9.2593 clk_54m
create_clock -period 10.0000 clk_25m_x2
create_clock -period 40.000 [get_ports {gpio_clk_25m}]
```

在控制面板中右击 Constraint，选择 add 添加 sdc 文件到工程。



(6) 编译工程

关于编译操作可以查看公众号 *Efinity 入门使用-v8*

(7) 程序测试

一、LED 及晶振测试

LED 名	FPGA Pin Name	FPGA Pin Num.	Bank	Bank 电压
LED0	GPIOR_27	P13	BR	54M
LED1	GPIOR_24	N13	BR	50M
LED2	GPIOL_P_12	G3	1B	25M
LED3	GPIOL_P_02_CSI	N2	1A	25M
LED4	GPIOL_N_12	G4	1B	片上晶振
LED5	GPIOL_P_15	E1	1B	片上晶振
LED6	GPIOL_N_15	E2	1B	片上晶振
LED7	GPIOL_N_02	M2	1B	片上晶振

程序配置之后，5 个灯同时按 1s/次进行闪烁。如果那个灯没有闪烁可以按照上面的对应关系来查找是否对应晶振存在问题。

二、按键测试

按键名	FPGA Pin Name	FPGA Pin Num.	Bank	说明
KEY0	GPIOL_07	C4	TL	控制 LED0 的闪烁频率

KEY1	GPIOR_22	P14	BR	控制 LED1 的闪烁频率
KEY2	GPIOR_21	N14	BR	控制 LED2-3 的闪烁频率
KEY3	GPIOL_03	A3	TL	控制 LED4-7 的闪烁频率

三、添加 debug 信号。

02_Ti60F225_hdmi_demo

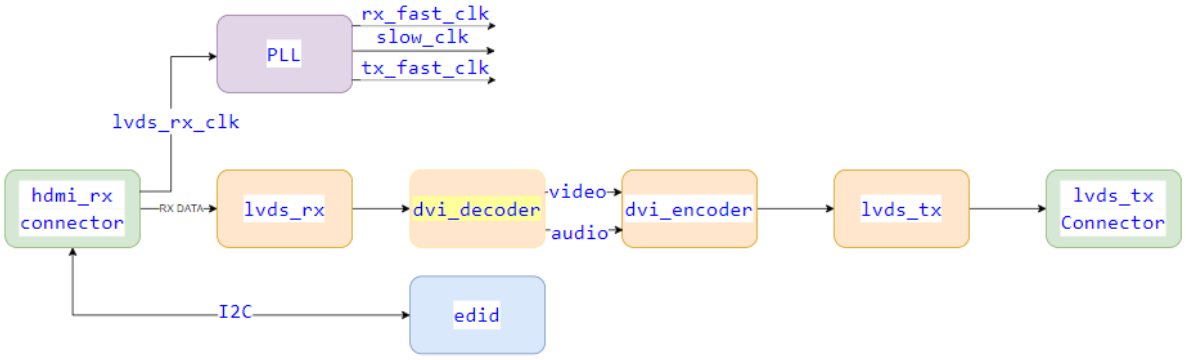
该 demo 通过 LVDS 模拟 HDMI 输出。该 demo 实现 hdmi rx 到 hdmi tx 回。知识补充

audio_max_word_length	:32	0:最大是 20 位; 1：最大位宽 24 位;	
audio_samp_word_len	Bits[35:33]	Bit3[32] = 1，0 000b：字长没有说明 001：20 位，16 位 010：22 位，18 位 100：23 位，19 位 101：24 位，20 位 110：21 位，17 位	
audio_samp_freq	Bits[27:24]	采样频率和时钟精度 Bits[27:24] 0100: 20.05Khz 0000:44.1kHz 1000：88.2KHz 0110:24KHz 0010:48KHz 1010:96KHz 0011：32KHz 0001：采样频率没有指定 1001：768KHz	
Source number	Bits[19:16]	0000: 不用考虑	

		0001：1 0010：2 0011：3 。。。。。。 1111：15 ，没有使用	
Channel number	Bits[23:20]	Bits[23:0] 0000:不用考虑 0001：左声道 0010：右声道	
Audio sample word mode	Bits[0]	0:线性 PCM 采样 1：	
Original sampling frequency	Bits[39:36]	没有使用，为 0	
Clock accuracy	Bit[29:28]	没有使用，0	

```
Parameter AUDIO_CHANNEL = 0; //=0:two channel ;=1: 8 channel
```

（1） Demo 框图说明



（1） HDMI 通过 FPGA 的高速 IO，并用 LVDS 实现 HDMI 接收；
（2） HDMI 接收时钟经过 PLL 产生内部处理和发送的多路径时钟
（3） HDMI decoder 解出的数据直接输入到 dvi encoder
（4） 输出通过 LVDS。

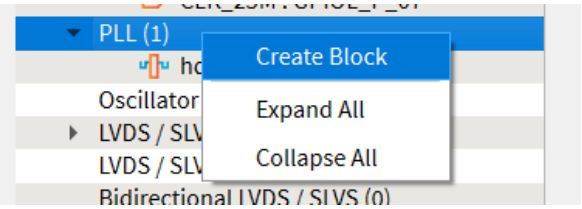
LED 指示灯说明

LED	说明
D3	FPGA 系统运行灯。上电之后开始闪烁，闪烁时间为 0.5s
D4	HDMI 接收指示点，当输入 HDMI 有时钟之后就会闪烁。闪烁时间由 HDMI 的接收分辨率来确定，当分辨率为 1080p60 时，闪烁时间间隔为 0.5s

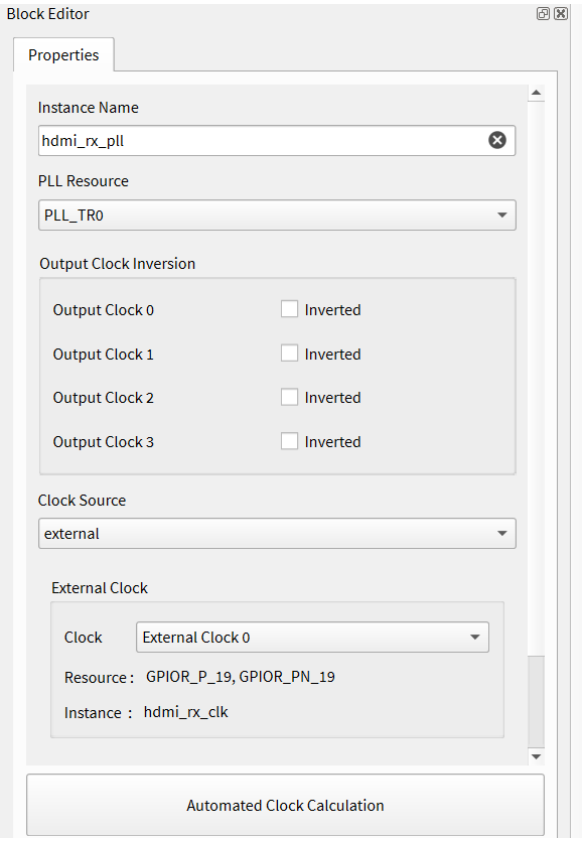
（2）添加相应的管脚。

（3）PLL 的设置。

在 PLL 中右键 PLL,选择"Create Block"添加一个 PLL.

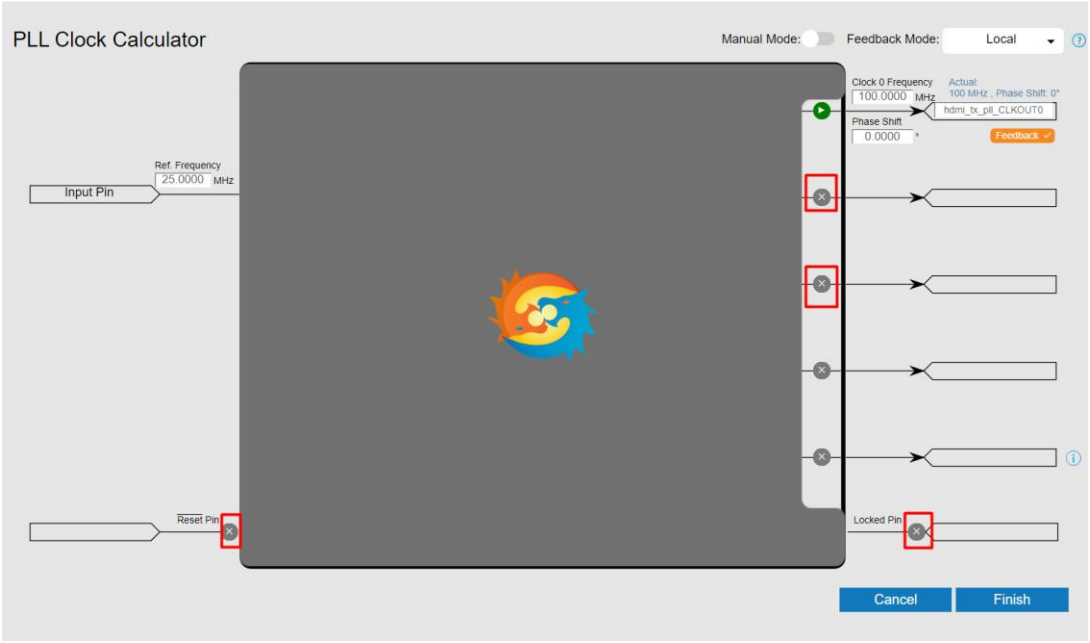


在 PLL 的 Block Editor 中,设置 Instance Name,为"hdmi_rx_pll";设置 PLL_Resource 为"PLL_TR0";Clock source 设置为"external refclk0";**时钟源即为 hdmi_rx_clk**



点击上图中的 Aucomated Clock Calculation，打开 PLL 的设置界面。设置 Ref Frequency 为 148.5MHz，关于晶振可以参考“TI60F225-DK 硬件设计_v1.4 “的时钟部分。点击 Reset Pin，Locked Pin 及 CLKOUT1,CLKOUT2 及 CLKOUT3 位置的”x”，打开相应的管脚。

设置工程需要的两个时钟，分别为 hdmi_tx_slow_clk 和 hdmi_tx_fast_clk；频率分别为 148.5MHz 和 742.5MHz，并且 hdmi_tx_fast_clk 的移相 90 度,hdmi_rx_fast_clk 移相 45 度。

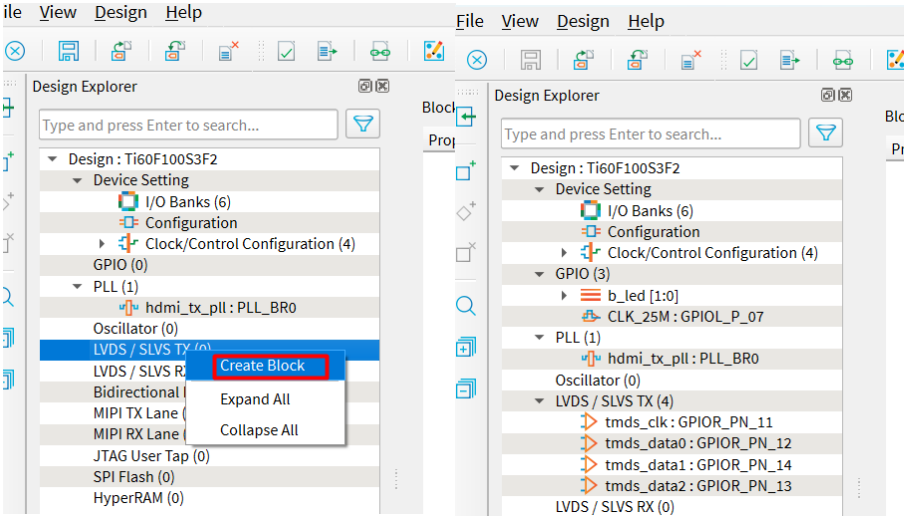


效果如下图：



（4）添加 LVDS

在 interface 中选择 LVDS /SLVS TX,右键选择 Create Block。添加 4 对 LVDS。关于 LVDS 的参数设置如下表



表

（1） Hdmi rx

Instance Name	hdmi_rx_clk	hdmi_rx_d0/1/2/3
Connection Type	Pll_in	nornal
Input Pin/Bus Name	hdmi_rx_clk_RX_DATA	lvds_rx_dn_RX_DATA
Dynamic Enable Pin Name	hdmi_rx_clk_RX_ENA	lvds_rx_dn_RX_ENA
Enable Common Mode Driver	Yes	Yes
Enable SLVS	NO	NO
Termination	on	on
Delay Mode	Dynamic	static
Static mode delay setting		16
Enable Deserialization	--	YES
Enable Half Rate deserialization	--	YES
Serial Clock	--	Hdmi_rx_fast_clk
Parallel Clock	--	Hdmi_rx_slow_clk
Reset Pin Name	--	lvds_rx_d0_RX_RST

（2） Hdmi tx

Instance Name	tmds_tx_clk	Tmds_tx_datan
LVDS Resource	GPIOR_PN_11	GPIOR_PN_12
Differental Type	lvds	lvds
VOD	large	large
Pre-Emphsis	Medium low	Medium low
Mode	Serial data	Serial data

Output pin/bus name	Tmds_clk_o	tmds_data0_o
Enable pin name	tmds_clk_TX_OE	tmds_data0_TX_OE
Enable Serialization	Yes	Yes
Enable Half Rate	Yes	Yes
Serialization Width	10	10
Serial Clock	hdmi_tx_fast_clk	hdmi_tx_fast_clk
Parallel Clock	hdmi_tx_slow_clk	hdmi_tx_slow_clk
Reset Pin Name	tmds_clk_TX_RST	tmds_data0_TX_RST
Advanced Setting	0	0

关于 LVDS 的信号控制如下:

```
assign hdmi_rx_clk_RX_ENA = 1'b1;

assign hdmi_rx_d0_RX_ENA = 1'b1;
assign hdmi_rx_d1_RX_ENA = 1'b1;
assign hdmi_rx_d2_RX_ENA = 1'b1;

assign hdmi_rx_d0_RX_RST = 1'b0;
assign hdmi_rx_d1_RX_RST = 1'b0;
assign hdmi_rx_d2_RX_RST = 1'b0;

assign tmds_tx_data0_TX_OE = 1'b1;
assign tmds_tx_data1_TX_OE = 1'b1;
assign tmds_tx_data2_TX_OE = 1'b1;
assign tmds_tx_clk_TX_OE   = 1'b1;

assign tmds_tx_data0_TX_RST = 1'b0;
assign tmds_tx_data1_TX_RST = 1'b0;
assign tmds_tx_data2_TX_RST = 1'b0;
assign tmds_tx_clk_TX_RST   = 1'b0;
```

(5) Demo 演示效果

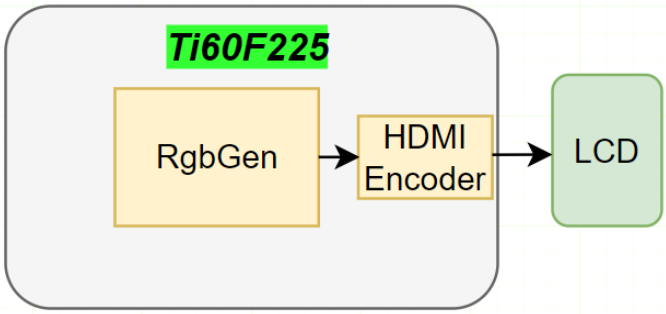


03_hdmi_tx_demo

版本	更新说明	Efinity 版本
hdmi_tx_demo_v2	（1）color_bar 更新为 v3.0	2025.2.288.4.15
hdmi_tx_demo_v3	(1) 更新了 PLL 的参考时钟为 27M。以保证输出标准的 1080p60 (2) 更新软件版本为	2026.1.132.1.2

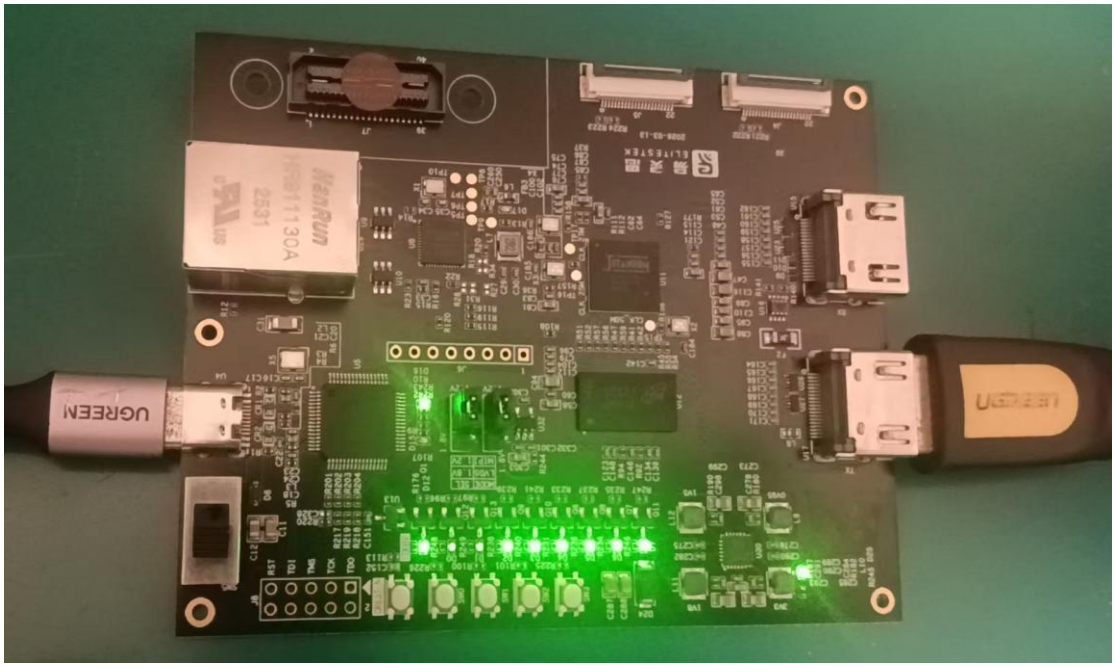
hdmi_tx_demo 是一个单独测试输出的工程。

工程框图



硬件连接

只需要连接 HDMI TX 到电脑即可。



测试输出结果

彩条测试，从左到右依次为
EBU 检测图从左至右依次为 白色、黄色、靛色、绿色 紫色、红色、蓝色、黑色



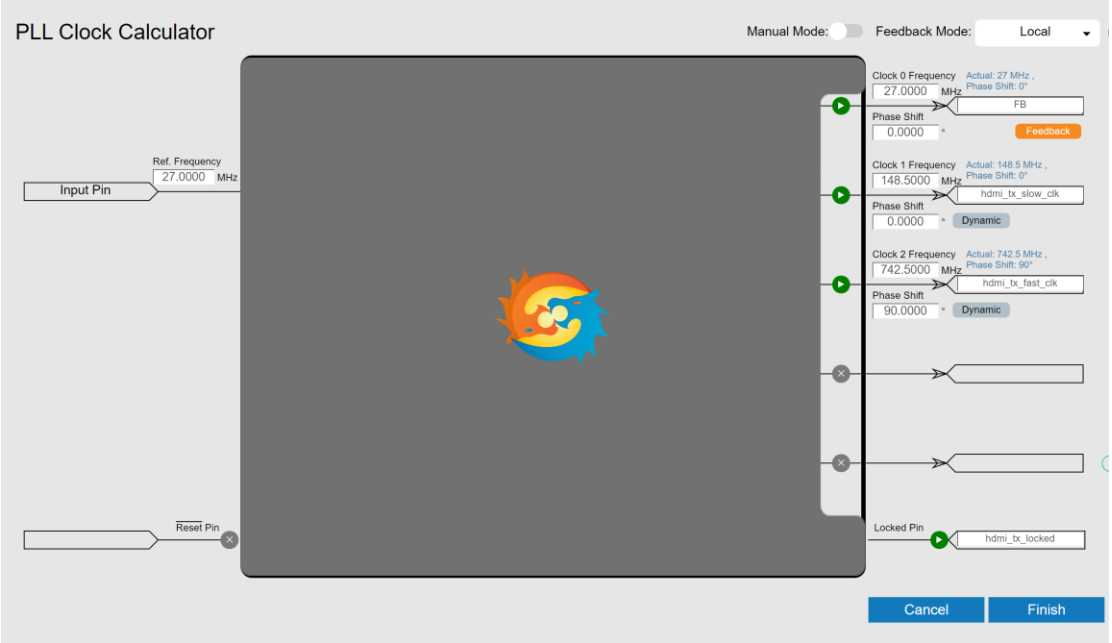
Timing 设置

根据需要设置输出分辨率参考 VESA-Monitor-Timing-Standard.pdf,以下是 1080p60 的参数

Name	1920x1080p60
Pixel Clock	148.5 MHz
TMDS Clock	1,485.0 MHz
Active Pixels	1920
Front Porch	88
Sync Width	44
Back Porch	148
Active Lines	1080
Front Porch	4
Sync Width	5
Back Porch	36

下面是 1080p60 的参数在工程的配置。PLL 的的 hdmi_tx_slow_clk 对应上面的 pixel Clock,而 hdmi_tx_fast_clk 为 TMDS Clock 的二分之一,因为 LVDS 是双沿发送。

```
parameter   HACT            = 12'd1920;
parameter   VACT            = 12'd1080;
parameter   HSP             = 8'd44;
parameter   HBP             = 8'd148;
parameter   HFP             = 8'd88;
parameter   VSP             = 8'd5;
parameter   VBP             = 8'd36;
parameter   VFP             = 8'd4;
```

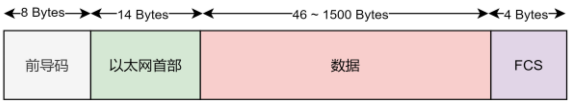


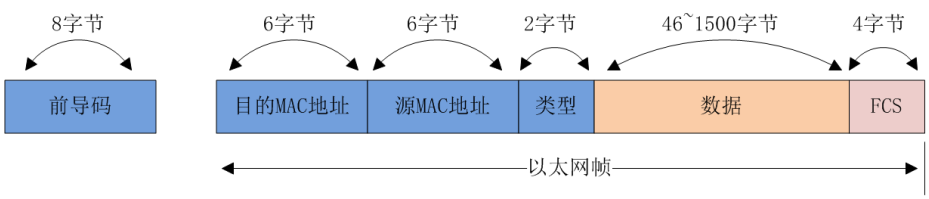
04_Ti60F225_eth_demo

版本	说明	Efinity 版本
04_Ti60F225_tse_hj_demo_v5		2025.2.288.4.15

以太网帧

下图为以太网的帧格式。

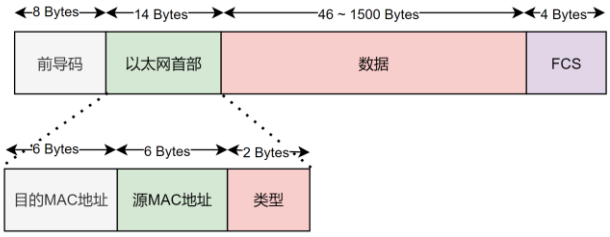




前导码（Preamble）：8 字节，连续 7 个 8'h55 加 1 个 8'hd5，表示一个帧的开始，用于双方设备数据的同步。

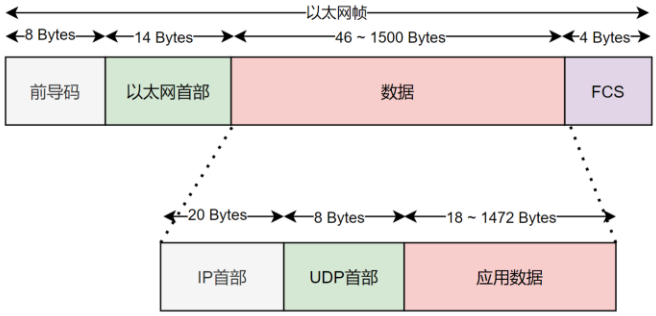
以太网首部

以太网首部包括三部分：目的 MAC 地址，源 MAC 地址和类型。



目的 MAC 地址：6 字节，存放目的设备的物理地址，即 MAC 地址
源 MAC 地址：6 字节，存放发送端设备的物理地址
类型：2 字节，用于指定协议类型，常用的有 0800（hex）表示 IP 协议；0806 表示 ARP 协议；8035 表示 RARP 协议 。数据先传输高字节,再传输低字节.
数据：46 到 1500 字节，最少 46 字节，不足需要补全 46 字节，例如 IP 协议层就包含在数据 部分，包括其 IP 头及数据。
FCS：帧尾，4 字节，称为帧校验序列，采用 32 位 CRC 校验，对目的 MAC 地址字段到数据字段进行校验。

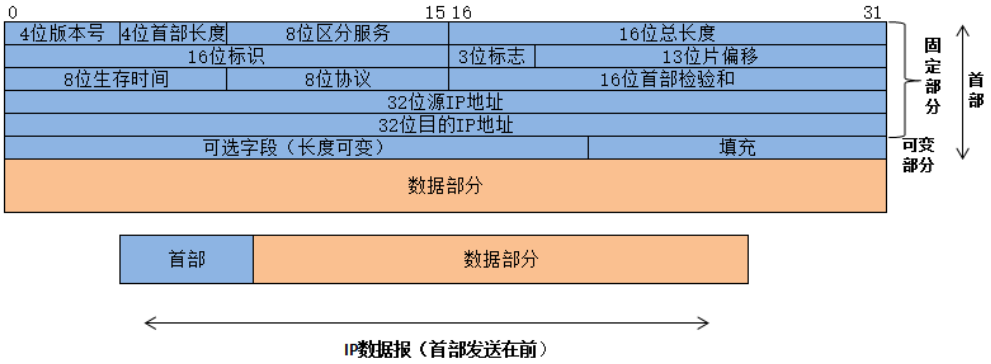
进一步扩展，以 UDP 协议为例，可以看到其结构如下，除了以太网首部的 14 字节，数据部分包含 IP 首部，UDP 首部和应用数据共 46~1500 字节。



下面再介绍下 IP 首部

IP 数据报（IP 首部）格式

下图为 IP 分组的报文头格式，报文头的前 20 个字节是固定的，后面的可变。



版本:占 4 位,指 IP 协议的版本目前的 IP 协议版本号为 4 (即 IPv4)。

首部长度:占 4 位,可表示的最大数值是 15 个单位(一个单位为 4 字节)因此 IP 的首部长度的最大值是 60 字节。

区分服务:占 8 位,用来获得更好的服务,在旧标准中叫做服务类型,但实际上一直未被使用过.1998 年这个字段改名为区分服务.只有在使用区分服务(DiffServ)时,这个字段才起作用.一般的情况下都不使用这个字段。

总长度:占 16 位,指**首部**和**数据部分**之和的长度,单位为字节,因此数据报的最大长度为 65535 字节.总长度必须不超过最大传送单元 MTU(Maximum Transmission Unit)。

标识:占 16 位,它是一个计数器,用来产生数据报的标识。

标志(flag): 占 3 位,目前只有后两位有意义

位	名称	含义
1	保留位	必须为 0，目前没有使用，保留给将来扩展使用。
2	不分片位（DF，Don't Fragment）	用于控制 IP 数据报是否允许分片。如果 DF 位被设置为 1，表示不允许对该数据报进行分片。当路由器需要转发一个超过出接口 MTU 的数据包，且 DF 位为 1 时，路由器会丢弃该数据包，并向源主机发送一个 ICMP 差错报告报文，告知源主机数据包因为不能分片而被丢弃。如果 DF 位为 0，则表示允许对数据报进行分片。
3	更多分片位（MF，More Fragments）	用于标识当前分片是否是最后一个分片。当 MF 位为 1 时，表示该分片后面还有更多的分片；当 MF 位为 0 时，表示这是最后一个分片或者该数据报没有进行分片。例如，一个大的 IP 数据报被分成了多个分片进行传输，除了最后一个分片的 MF 位为 0 外，其他分片的 MF 位都为 1。

片偏移:占 13 位,指较长的分组在分片后某片在原分组中的相对位置.片偏移以 8 个字节为偏移单位。

生存时间:占 8 位,记为 TTL(Time To Live) 数据报在网络中可通过的路由器数的最大值,TTL 字段是由发送端初始设置一个 8 bit 字段.推荐的初始值由分配数字 RFC 指定,当前值为 64. 发送 ICMP 回显应答时经常把 TTL 设为最大值 255。

协议:占 8 位,指出此数据报携带的数据使用何种协议以便目的主机的 IP 层将数据部分上交给哪个处理过程, 1 表示为 ICMP 协议, 2 表示为 IGMP 协议, 6 表示为 TCP 协议, 17（dec）表示为 UDP 协议。

首部检验和:占 16 位,只检验数据报的首部不检验数据部分，采用二进制反码求和，即将 16 位数据相加后，再将迸位不低 16 位相加，直到进位为 0，最后将 16 位取反。

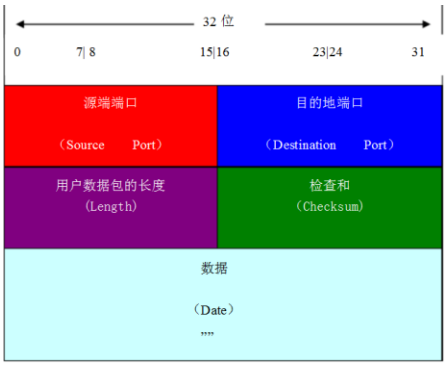
源地址和目的地址:都各占 4 字节,分别记录源地址和目的地址

UDP 协议

UDP 是 User Datagram Protocol（用户数据报协议）的英文缩写。UDP 只提供一种基本的、低延迟的被称为数据报的通讯。所谓数据报，就是一种自带寻址信息，从发送端走到接收端的数据包。UDP 协议经常用于图像传输、网络监控数据交换等数据传输速度要求比较高的场合。

UDP 协议的报头(**UDP 首部**)格式:

UDP 报头由 4 个域组成，其中每个域各占用 2 个字节，具体如下:

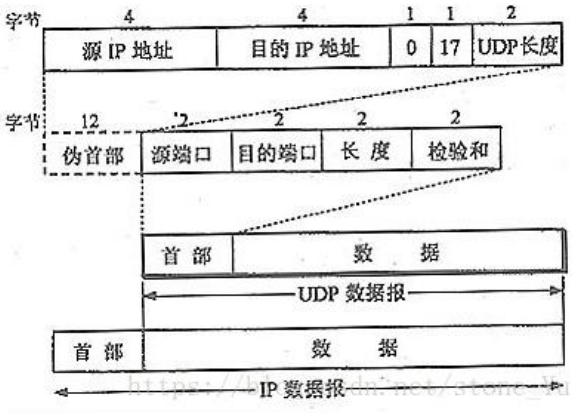


源端口：源端口号。在需要对方回信时选用。不需要时可用全 0。

目的端口：目的端口号。这在终点交付报文时必须要使用到。

长度：UDP 用户数据报的长度，对应 **UDP 首部**及**应用数据**的字节长度，其最小值是 8（仅首部）。

校验和：检测 UDP 用户数据报在传输中是否有错。有错就丢弃。



三、UDP 校验和计算

UDP 计算校验和的方法和 IP 数据报首部校验和的方法相似。不同的是：**IP 首部**校验和只校验 **IP 首部**，但 UDP 的校验和是把首部和数据部分一起都检验。

UDP 的校验和需要计算 **UDP 首部**加**应用数据**部分，并且还需要加上 UDP 伪首部。这个伪首部指源地址、目的地址、UDP 数据长度、协议类型（0x11），协议类型就一个字节，但需要补一个字节的 0x0，构成 12 个字节。伪首部+UDP 首部+数据一起计算校验和。

UDP 检验和的计算方法是：

1. 从“伪头部”开始，按每 16 位当作一个数，逐次求和，最终得出一个 32 位的数；
2. 如果这个 32 位的数的高 16 位不为 0，则进行“回卷”操作。也就是将这个 32 位数据的高 16 位与低 16 位相加，得到一个 32 位的数；
3. 重复第 2 步直到高 16 位为 0。
4. 最终，将低 16 位取反，得到校验和，填入 checksum 字段中

差错检验

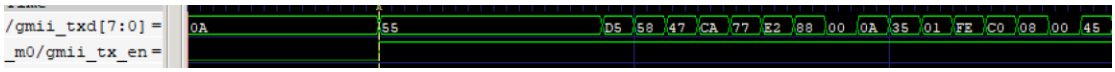
当接收到 UDP 报文时，需要如何检验其正确性？方法就是将 UDP 报文中包括校验和在内的，所有的 16 位的数相加，如果低 16 位全为 1，则没有出错。否则表明该分组中出现了错误。

需要注意，UDP 对差错具有一定的校验能力，但缺少差错恢复的能力

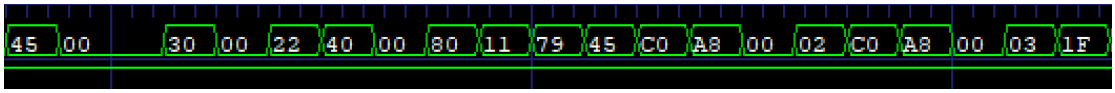
位	0 – 7	8 – 15	16 – 23	24 – 31
0	来源地址			
32	目的地址			
64	全零	协议名	UDP报文长度	
96	来源连接端口		目的连接端口	
128	报文长度		检验和	
160+	数据			

为何称之为“伪头部”？原因是，UDP 协议只使用它来辅助计算校验和，它并不是发送 IP 数据包时使用的 IP 数据包的头部。

下图是抓取发送的一个数据包。55 55 55 55 55 55 55 D5 是前导码。58 47 CA 77 E2 88 是目的 MAC 地址，00 0A 35 01 FE C0 08 为源 MAC 地址，0800 为 IP 协议，45 为版本号及首部长度，首部长度为 5 说明，IP 首部长度为 20 个字节；



00 为 8 位区分服务；00 30 为总长度 48，【45 00 00~30 30 0A】;40 00 是偏移；80 是指生成时间；11 是指 UDP 协议；79 45 是校验和；C0 A8 00 02 是源地址 192.168.0.2；C0 A8 00 03 是目的地址 192.168.0.3；

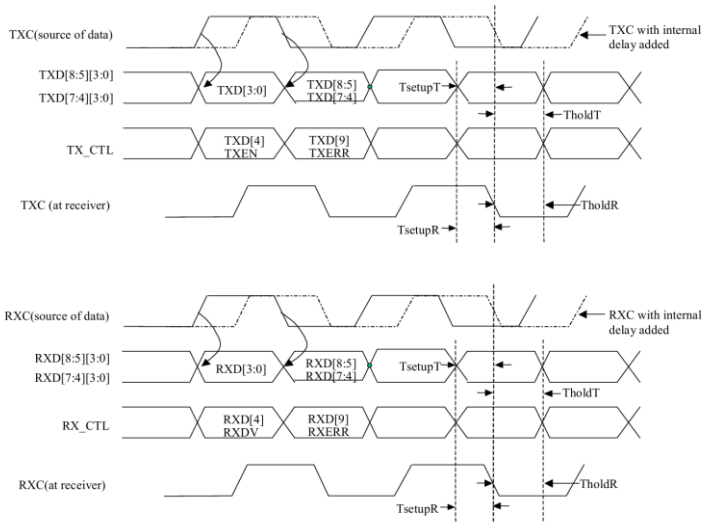


1F 90 为源端口 8080；1F 90 为目的端口 8080；00 1C 为数据长度；4A 63 是校验和；从 48 65~30 0A 为“Hello From Ti60F100“+“回车“;DD 8F 35 78 为 FCS.



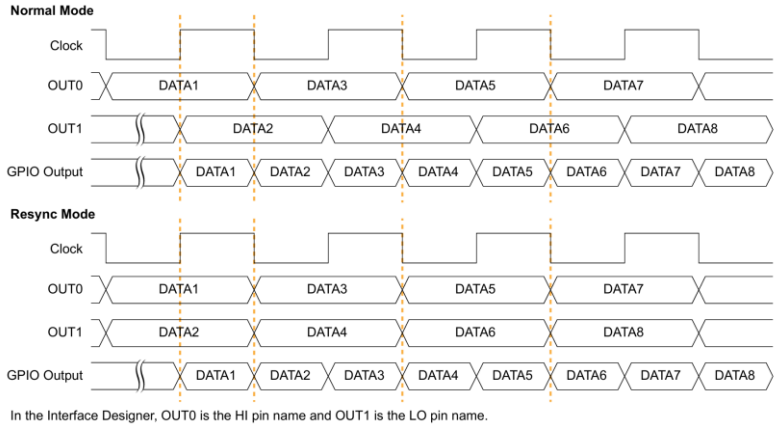
接口处理

首先来看 RGMII 规范(参考 RGMII_V2.0)对接口的时序要求。无论是接收还是发送，在时钟的上升沿发送低 4 位，TXEN 及 RXDV，时钟的下降沿发送高 4 位，TXERR,RXERR。



由下图为 FPGA 的 DDIO 发送波形，时钟的上升沿打出去的是 HI 数据，反之，下降沿打出去的是 LO 数据。

Figure 17: DDIO Output Timing Waveform



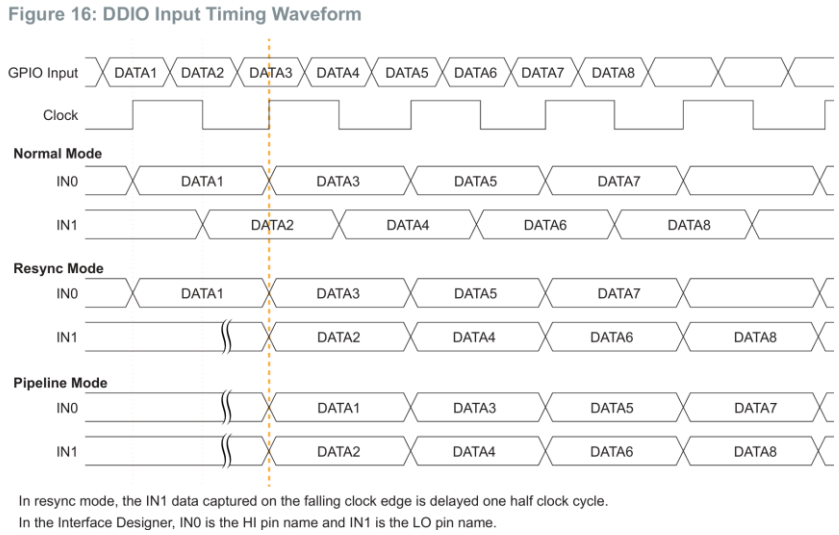
In the Interface Designer, OUT0 is the HI pin name and OUT1 is the LO pin name.

综上，就要求发送时就要把 gmii_txd[3:0]赋值给 HI,gmii_txd[7:4]赋值给 LO, tx_en 赋值给 Hi,tx_err 赋值给 LO,所以在程序上处理如下：

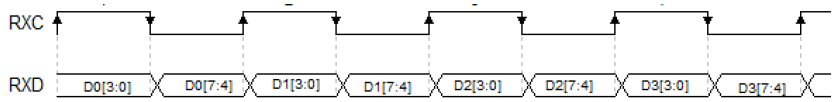
```
always @(posedge gmii_tx_clk or negedge rst_n)
begin
    if (!rst_n) begin
        rgmii_tx_ctrl_lo      <= 1'b0 ;
        rgmii_tx_ctrl_hi      <= 1'b0 ;
        rgmii_txd_lo          <= 4'd0 ;
        rgmii_txd_hi          <= 4'd0 ;
    end else begin
```

```
        rgmii_tx_ctrl_lo     <= gmii_tx_en_r ^ gmii_tx_er_r;          //TX_ER
        rgmii_tx_ctrl_hi     <= gmii_tx_en_r ;                        //TX_EN
        rgmii_txd_lo         <= gmii_txd_r[7:4] ;
        rgmii_txd_hi         <= gmii_txd_r[3:0] ;
    end
end
```

下图为 DDIO 的接收波形，从波形上看，只要是正常的接收，时钟的上升沿数据对应的是 HI，下降沿数据对应的是 LO.

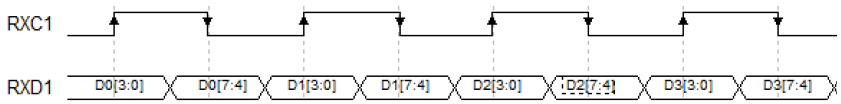


如果 PHY 没有设置 RXC 的延时，所以 PHY 端发出的数据波形如下：



但是时钟经过 FPGA 的 gclk 会有 1ns 多的延时，所以波形会变成下面的图，这时我们可以按中心对齐直接采样。当然根据需要可以调整 IO 的延时，另外由上面的图可以看出在 Resync Mode 下，LO 会比 HI 延时一拍，所以要延时一拍进行对齐。

Clock Pin	Max (ns)	Min (ns)
clk_125m	1.903	1.231
osc_inst1_CLKOUT	1.852	1.199
rxc1	1.847	1.161



```
always @(posedge rxc)begin
          align_rxd_hi     <= rxd_hi;
          align_rx_dv_hi <= rx_dv_hi;
end
assign align_rxd_lo = rxd_lo;
assign align_rx_dv_lo = rx_dv_lo;


util_gmii_to_rgmii util_gmii_to_rgmii_m0
(
 /*|*/.rst_n                    (rst_n              ),
 //interface RXD
 /*i*/.rgmii_rxc                (rxc                ),
 /*i*/.rgmii_rx_hi              (align_rxd_lo), //hi low 要取反
 /*i*/.rgmii_rx_lo              (align_rxd_hi), //hi low 要取反
 /*i*/.rgmii_rx_dv              (align_rx_dv_hi),
 /*i*/.rgmii_rx_er              (align_rx_dv_lo),
```

udp_total_data_length 就是 udp 发送有效数据的总长度。因为 UDP 的“伪首部”和首部共 20 个字节，而数据长度最小要求 46 个字节，所以 udp_total_data_length 最小是 46 – 20;
udp_data_length 是除去“伪首部”的 udp 数据长度。
Upd_send_data_length 是 upd 的有效数据字节数。

```
always @(posedge clk or negedge rst_n)
  begin
    if(rst_n == 1'b0)
       begin
          udp_total_data_length <= 16'd0 ;
          udp_data_length <= 16'd0 ;
       end
    else
       begin
          udp_data_length <= udp_send_data_length + 8;
          if (udp_send_data_length < 18)
             udp_total_data_length <= 26 ;
```

```
        else
            udp_total_data_length <= udp_send_data_length + 8 ;
    end
end
```

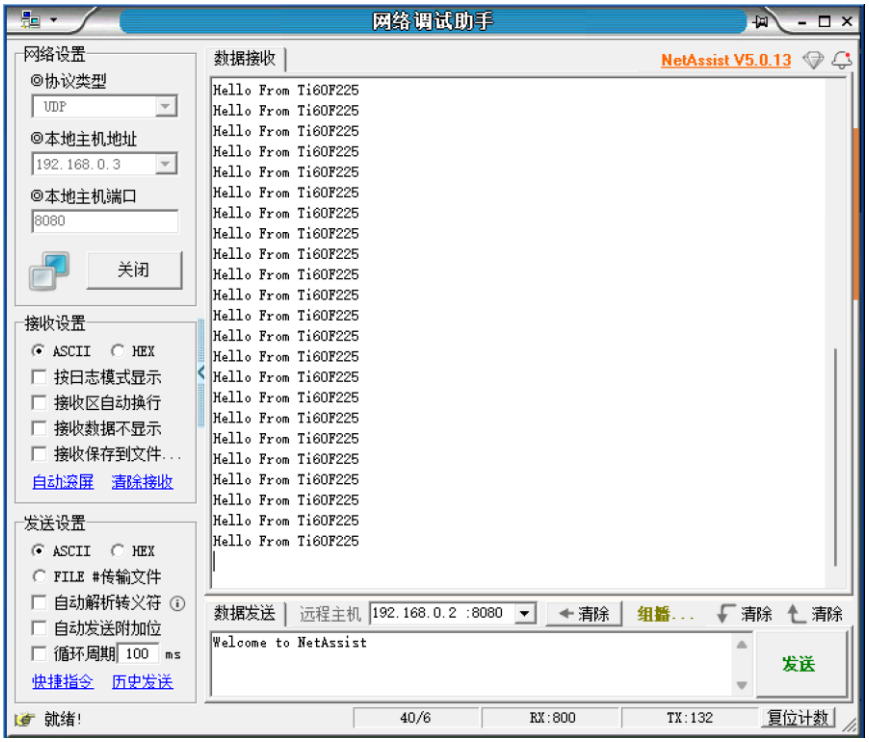
Eth speed	Speed(Mbps)	
2'b00	10	
2'b01	100	
2'b10	1000	

黄灯闪烁是千兆网。

测试

把 IP 地址修改成 192.168.0.3



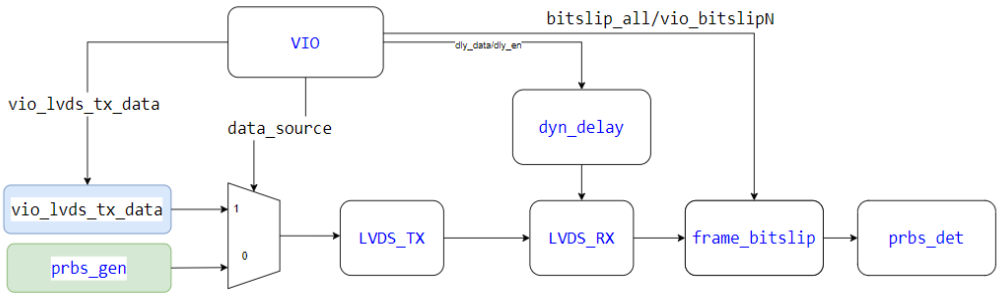


04_Ti60F225_lvds_loop_demo

软件	版本
Efinity	2025.2.288.3.8
工程名	03_Ti60F100_LVDS_demo_v1.1 (1)把 LVDS 速率修改成 1.5Gbsp

该工程旨在指导客户使用 LVDS，该程序实现 1.5Gbps LVDS 自环。在 02 demo 中我们已经添加了 LVDS 的输出，关于 LVDS 可以预先学习公众号 *LVDS-V1*。

（1） Demo 框图



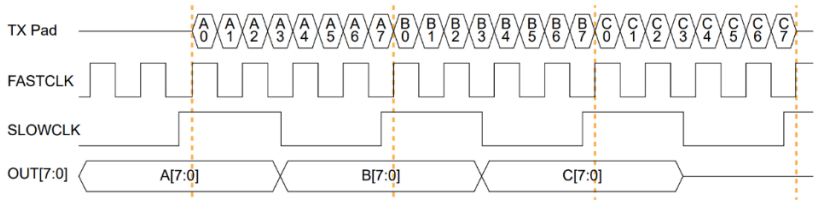
1） prbs_gen 模块。产生 8 位 prbs 数据，用于 LVDS 输出，通过 data_source 选择。

2） vio_lvds_tx_data。由 VIO 产生的数据，用于 LVDS 输出，通过 data_source 选择。

```
always @(posedge tx_slow_clk or negedge tx_rstn)
    if (!tx_rstn) begin
        lvds_tx_d0_TX_DATA <= 8'h0;
        lvds_tx_d1_TX_DATA <= 8'h0;
        lvds_tx_d1_TX_DATA <= 8'h0;
        lvds_tx_d1_TX_DATA <= 8'h0;
    end else begin
        if (data_source) begin
            lvds_tx_d0_TX_DATA <= prbs_data;
            lvds_tx_d1_TX_DATA <= prbs_data;
            lvds_tx_d2_TX_DATA <= prbs_data;
            lvds_tx_d3_TX_DATA <= prbs_data;
        end else begin
            lvds_tx_d0_TX_DATA <= vio_lvds_tx_data;
            lvds_tx_d1_TX_DATA <= vio_lvds_tx_data;
            lvds_tx_d2_TX_DATA <= vio_lvds_tx_data;
            lvds_tx_d3_TX_DATA <= vio_lvds_tx_data;
        end
    end
```

LVDS_TX.在 interface 中设置。LVDS 的数据传输是低位先传

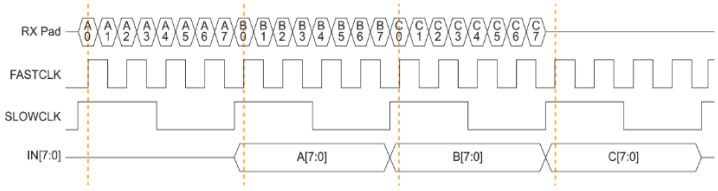
Figure 31: LVDS Timing Example Serialization Width of 8 (Half Rate)



OUT is byte-aligned data passed from the core on the rising edge of SLOWCLK.

3） LVDS_RX。在 interface 中设置。

Figure 29: LVDS RX Timing Example Serialization Width of 8 (Half Rate)



IN is byte-aligned data passed to the core on the rising edge of SLOWCLK.

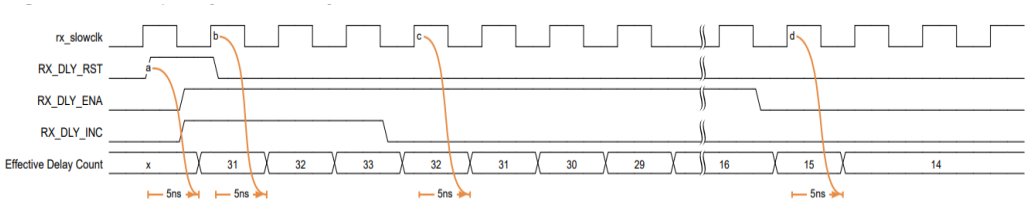
4） Dly_delay 模块。用于 LVDS_RX 的 IO 延时调整。

LVDS 动态校准

钛金提供了 64 个延时 step，每个 step 延时约 20ps,动态校准使用的是 slowclk,在 RX_DLY_RST 生效后会把延时调整到 step 31。并且延时的生效在时钟上升沿 5ns 之后，所以注意 slow_clk 在大于 200MHz 和小于 200M 时处理上的区别。

在 pll_slowclk 的上 posedge 断言 RX_DLY_RST

在 pll_slowclk 的 negedge 断言 RX_DLY_INC 和 RX_DLY_ENA



5） frme_bitslip 模块。用于 LVDS_RX_DATA 的数据位移。

6） prbs_det 模块。用于 PRBS 数据的校验。

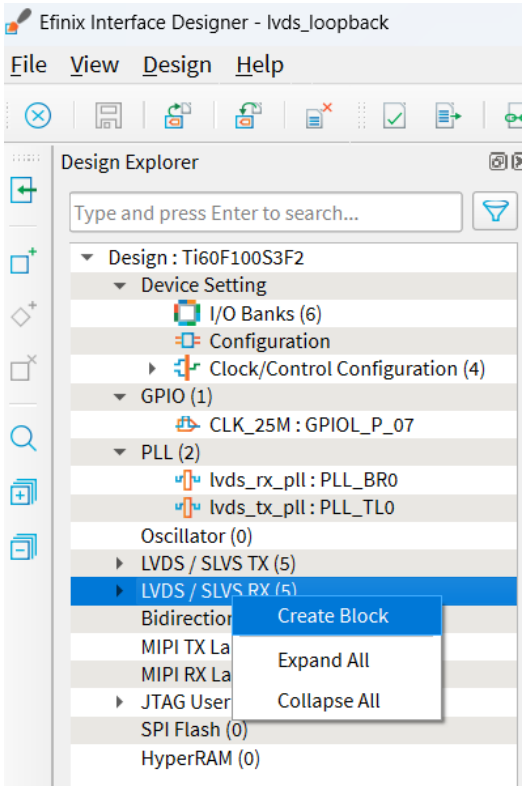
7） VIO 模块。通过 JTAG 产生控制信号。

（2）添加 VIO

VIO 的添加请参考培训视频。

(3) LVDS RX 添加

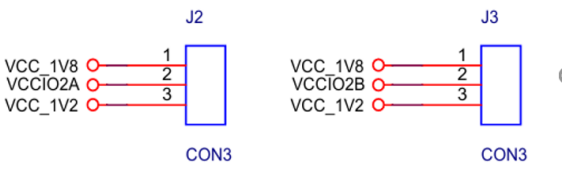
在 demo 02 中已经添加了 LVDS TX，这时来学习添加 LVDS RX.在 Interface 中选择 LVDS/SLVS RX，右击选择 Crate Block;要注意 LVDS_RX_CLK 是 PLL_IN 的属性。

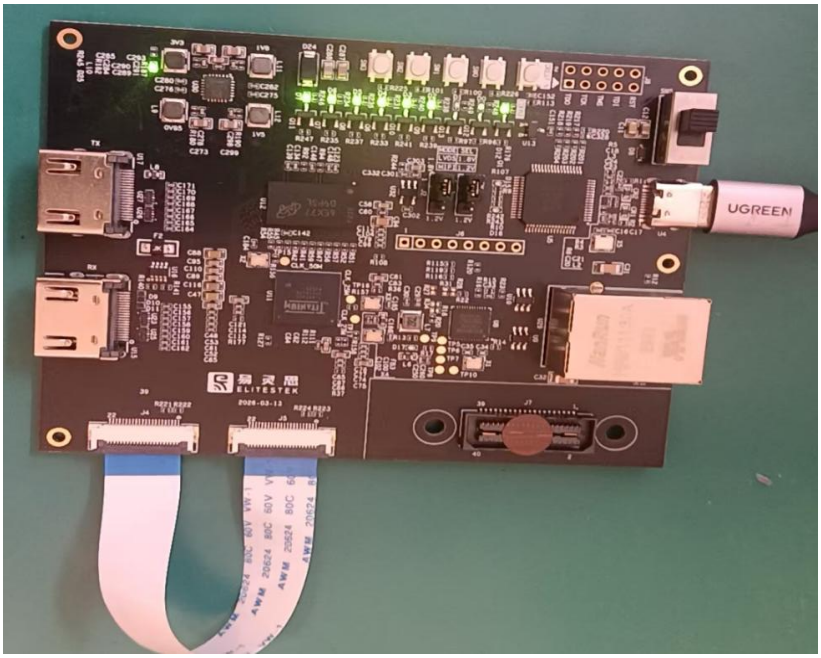


Instance Name	lvds_rx_clk	lvds_rx_d0/1/2/3
Connection Type	Pll_in	nornal
Input Pin/Bus Name	lvds_rx_clk_RX_DATA	lvds_rx_d0_RX_DATA
Dynamic Enable Pin Name	lvds_rx_clk_RX_ENA	lvds_rx_d0_RX_ENA
Enable Common Mode Driver	NO	NO
Enable SLVS	NO	NO
Termination	NO	NO
Delay Mode	Dynamic	Dynamic
Enable Deserialization	--	YES
Enable Half Rate deserialization	--	YES
Serial Clock	--	rx_fast_clk
Parallel Clock	--	rx_slow_clk
Reset Pin Name	--	lvds_rx_d0_RX_RST

(4) 硬件连接设置

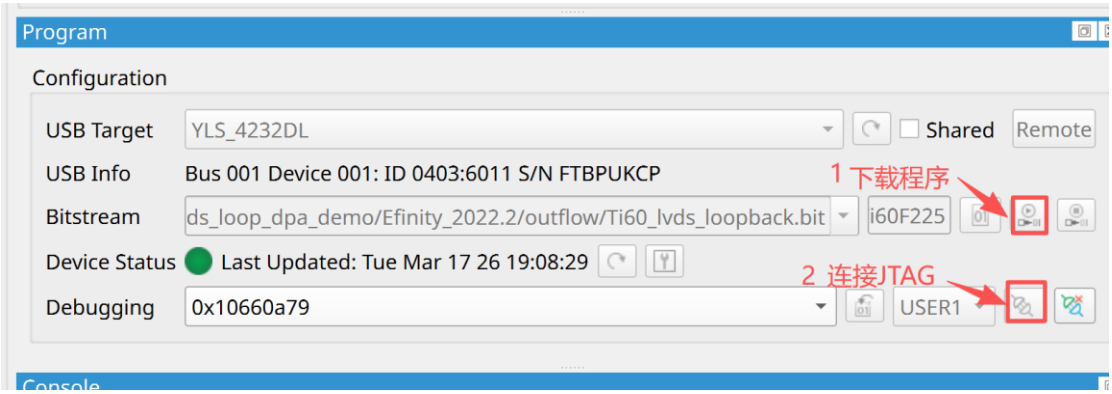
注意 LVDS 管脚所在的 bank 电压要设置成 1.8V。该 LVDS 所在的 bank 为 2B，所以要把 J3 的电压选择为 1.8V



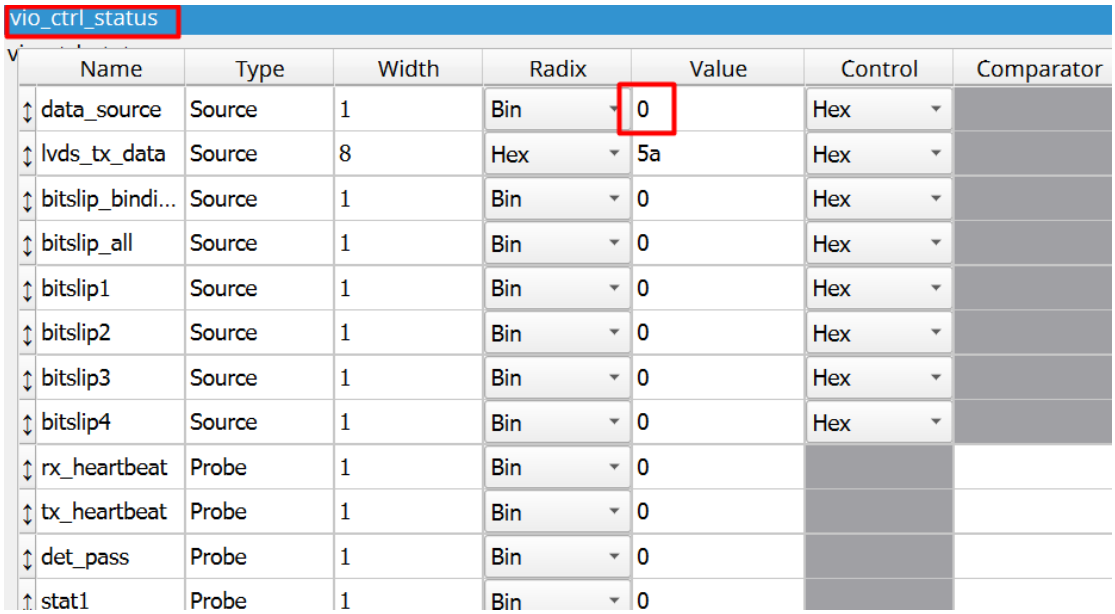


(3)演示设置

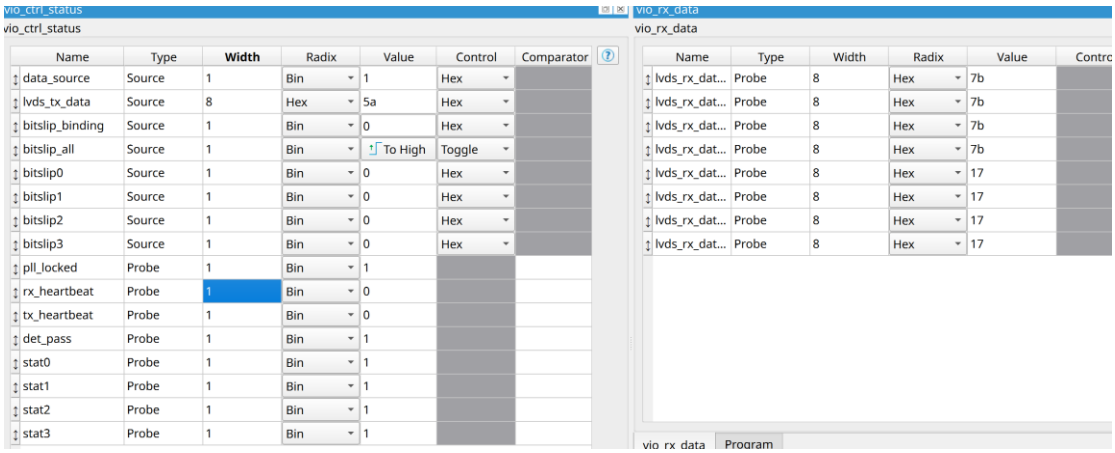
a) 烧写程序，连接 debug，



2）在 vio_ctrl_status 界面，把 data_source 设置为 0，这是 LVDS TX 的数据由 VIO 的 lvds_tx_data 来发送的。



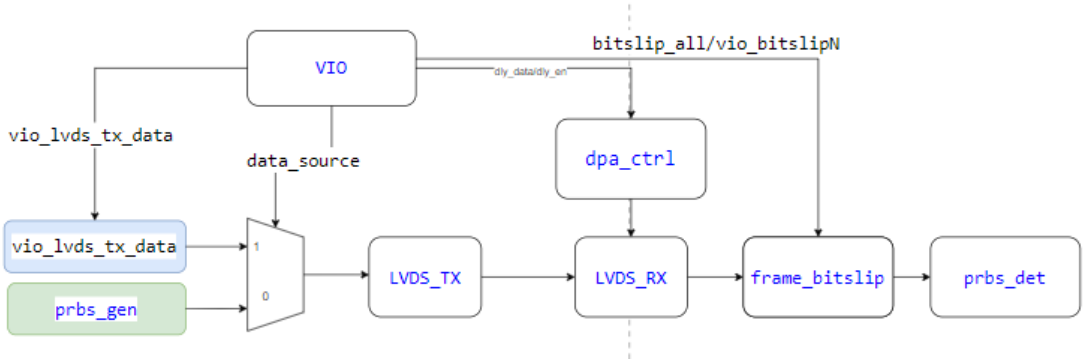
3）主要观察 statx，如果稳定为 1，说明 1.5G 自环正确。



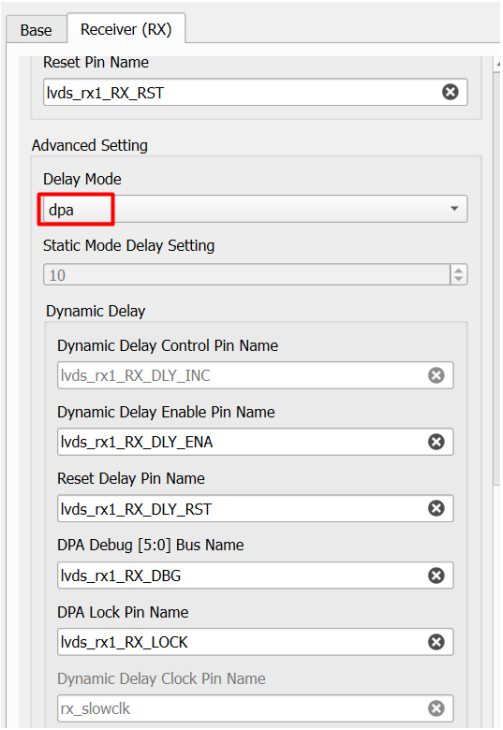
05_Ti60F225_lvds_loop_dpa_demo

Dpa demo 是把动态校准模块换成了 dpa_ctrl 模块。

(1)Demo 框图说明



把 LVDS RX 的 Delay Mode 设置为 dpa,会添加 4 个信号 DLY_ENA,DLY_RST,DBG,LOCK.




```
input [5:0]   lvds_rx1_RX_DBG,
input         lvds_rx1_RX_LOCK,
input [7:0]   lvds_rx1_RX_DATA,
output        lvds_rx1_RX_DLY_ENA,
output        lvds_rx1_RX_DLY_RST,
output        lvds_rx1_RX_RST,
output        lvds_rx1_RX_ENA,

input [5:0]   lvds_rx2_RX_DBG,
input         lvds_rx2_RX_LOCK,
input [7:0]   lvds_rx2_RX_DATA,
output        lvds_rx2_RX_DLY_ENA,
output        lvds_rx2_RX_DLY_RST,
output        lvds_rx2_RX_RST,
output        lvds_rx2_RX_ENA,
```

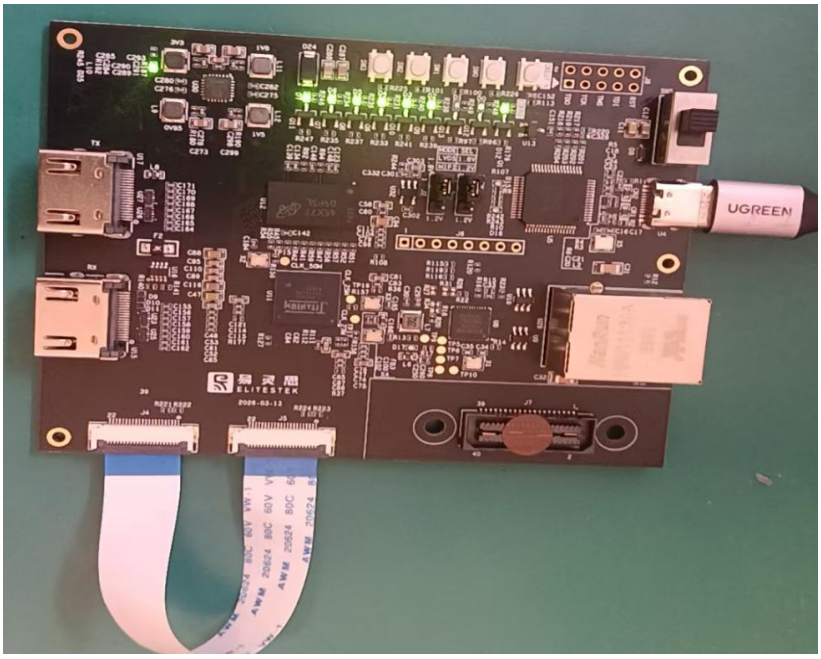
关于 DPA 的控制代码文档也有提供，如下:

```
module dpa_ctrl #(
parameter LANE_NUM = 4
)(
 input i_clk,
 input i_arstn,
 input i_pll_locked,
 input [LANE_NUM*6-1:0] i_dly_dbg,
 input [LANE_NUM-1:0] i_dly_lock,
 output [LANE_NUM-1:0] o_dly_rst,
 output [LANE_NUM-1:0] o_dly_ena,
 output o_all_locked,
 output [LANE_NUM-1:0] o_rx_rst
);

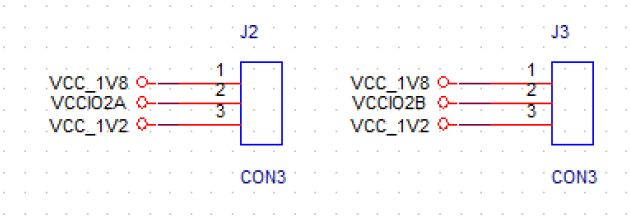
reg [LANE_NUM-1:0] r_ena, r_lock;
reg [1:0] r_rx_rst;
reg r_all_locked;
wire [LANE_NUM-1:0] w_all0, w_all1;
wire w_rst;
genvar i;
assign o_dly_rst = {LANE_NUM{~i_pll_locked | ~i_arstn}};
assign o_dly_ena = r_ena;
assign o_all_locked = r_all_locked;
assign o_rx_rst = {LANE_NUM{w_rst}};
assign w_rst = ~r_rx_rst[1] & r_rx_rst[0];
generate
 for(i=0; i<LANE_NUM; i=i+1)begin
 always@(negedge i_clk or negedge i_arstn)begin
      if(!i_arstn)begin
            r_ena[i] <= 1'b0;
            r_lock[i] <= 1'b0;
      end else begin
            if(w_all0[i] | w_all1[i] | i_dly_lock[i])begin // stop DPA calibration by putting ENA to
LOW,
                  r_ena[i] <= 1'b0; // when DBG stepped on 0 or 63, or DPA locked by itself
                  r_lock[i] <= 1'b1;
            end else begin
                  r_ena[i] <= 1'b1;
                  r_lock[i] <= r_lock[i];
            end
      end
 end
 assign w_all0[i] = ~|(i_dly_dbg[6*(i+1)-1:6*i]);
 assign w_all1[i] = &(i_dly_dbg[6*(i+1)-1:6*i]);
 end
endgenerate
always@(negedge i_clk or negedge i_arstn)begin
      if(!i_arstn)begin
            r_rx_rst <= 2'b00;
            r_all_locked<= 1'b0;
      end else begin
            if(&r_lock)begin
            r_rx_rst[0] <= 1'b1;
            r_rx_rst[1] <= r_rx_rst[0];
            end
```

```
    if(w_rst)
        r_all_locked <= 1'b1; // all DPA locked signal
    end
end
endmodule
```

（2）硬件连接

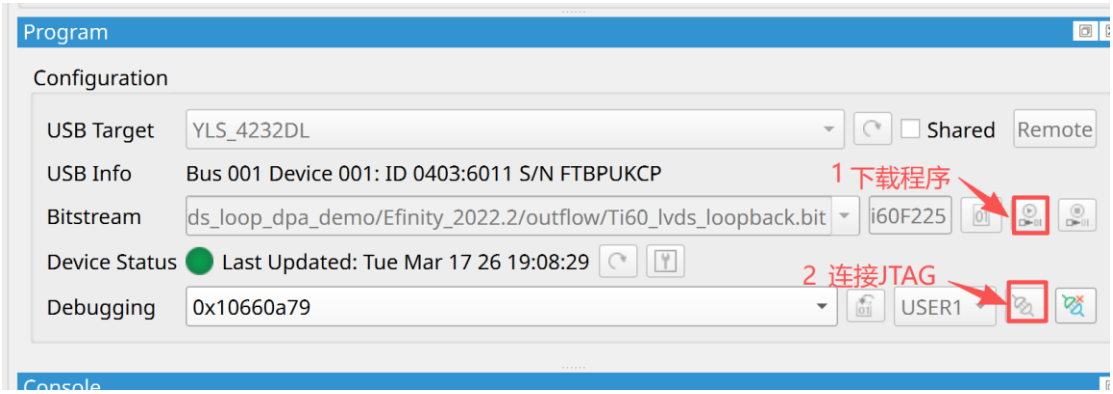


注意 LVDS 管脚所在的 bank 电压要设置成 1.8V。该 LVDS 所在的 bank 为 2B，所以要把 J3 的电压选择为 1.8V

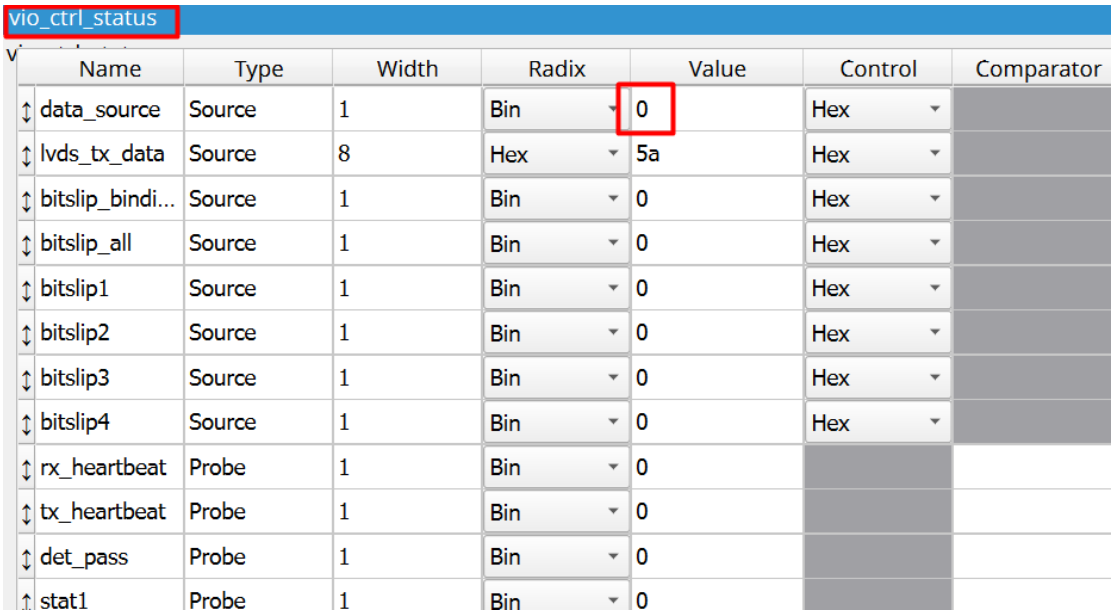


（3）演示

(1) 烧写程序，连接 debug，

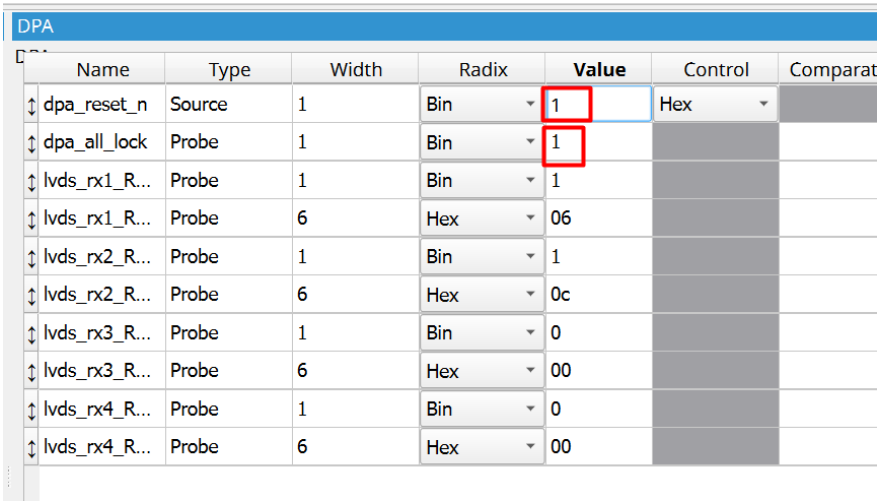


（2）在 vio_ctrl_status 界面，把 data_source 设置为 0，这是 LVDS TX 的数据由 VIO 的 lvds_tx_data 来发送的。

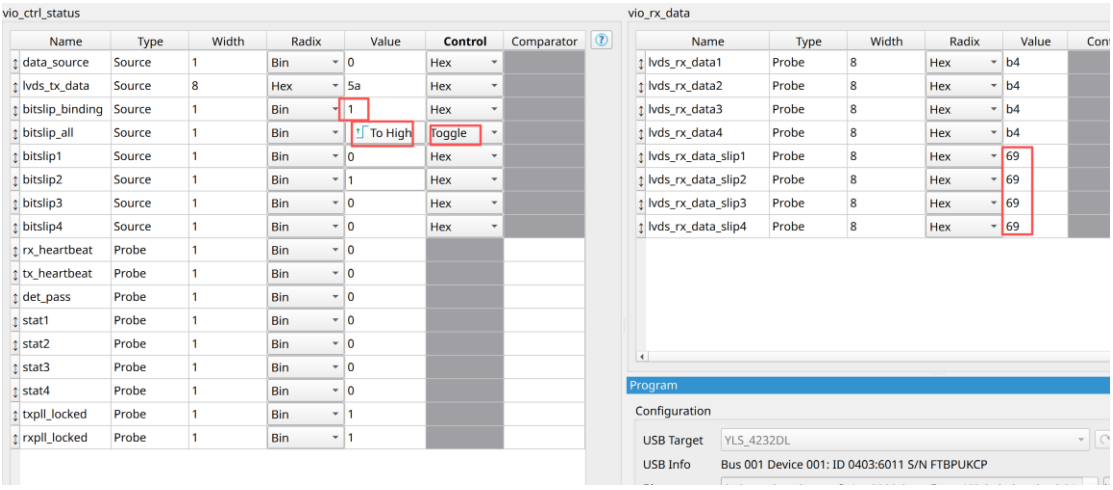


Name	Type	Width	Radix	Value	Control	Comparator
data_source	Source	1	Bin	0	Hex	
lvds_tx_data	Source	8	Hex	5a	Hex	
bitslip_bindi...	Source	1	Bin	0	Hex	
bitslip_all	Source	1	Bin	0	Hex	
bitslip1	Source	1	Bin	0	Hex	
bitslip2	Source	1	Bin	0	Hex	
bitslip3	Source	1	Bin	0	Hex	
bitslip4	Source	1	Bin	0	Hex	
rx_heartbeat	Probe	1	Bin	0		
tx_heartbeat	Probe	1	Bin	0		
det_pass	Probe	1	Bin	0		
stat1	Probe	1	Bin	0		

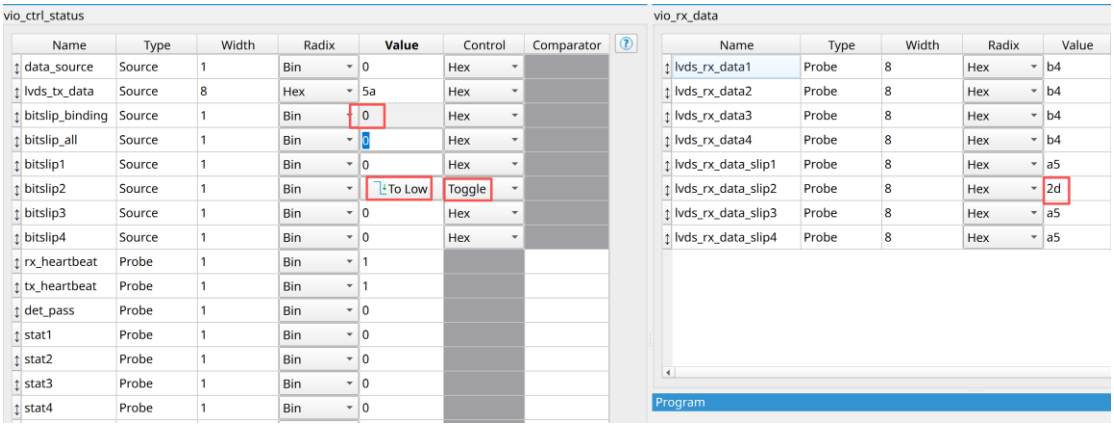
（2）DPA 界面只要把 dpa_reset_n 拉高，就会自动进行 DAP 的操作，当 dpa_all_lock 之后就可以认为 DPA 校准通过。



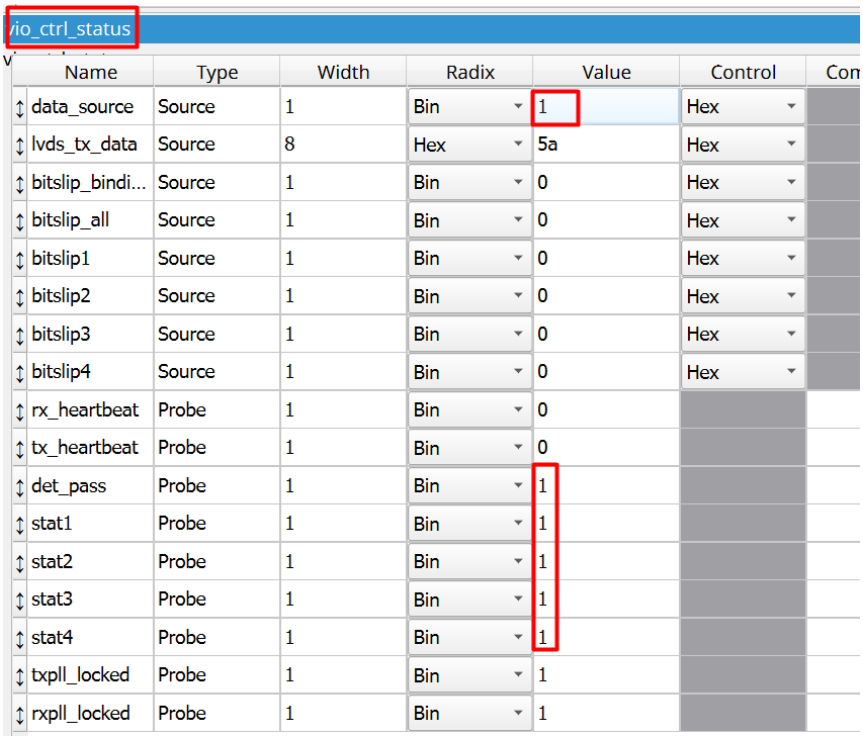
(3)把 bitslip_binding 设置为 1，bitslip_all 的 Control 选择为 Toggle，通过不断点击“To High”和“To Low”来移位数据。移位过程中观察 lvds_rx_data_slipx 来确认数据是否对齐。



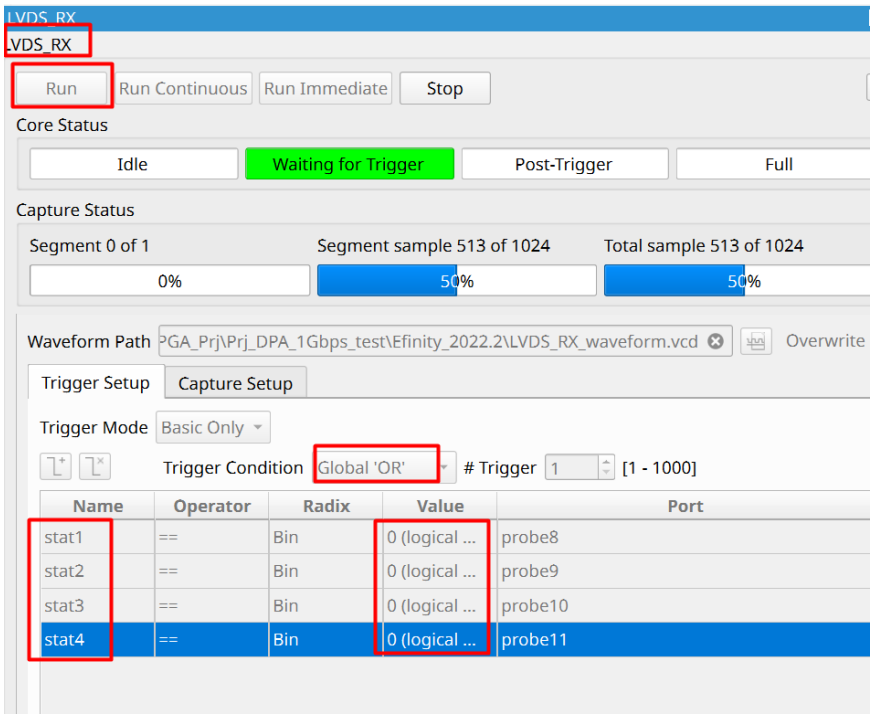
(4)经过步骤（3）之后，如果个别通道没有对齐，把 bitslip_binding 设置为 0，单独调整对应的 blitslip,下图以通道 2 为例。



(4)在 vio_ctrl_status 界面，把 data_source 设置为 0，这是 LVDS TX 的数据由 prbs_gen 模块产生，通过 prbs_detec 模块来进行检测，如果数据没有错相应的 statx 状态为 1，det_pass 为 1 。



当然更保险的办法是在 LVDS_RX 中通过 statx 进行行触发检测，看看 statx 是否有拉低的情况。如果不能触发说明状态稳定。

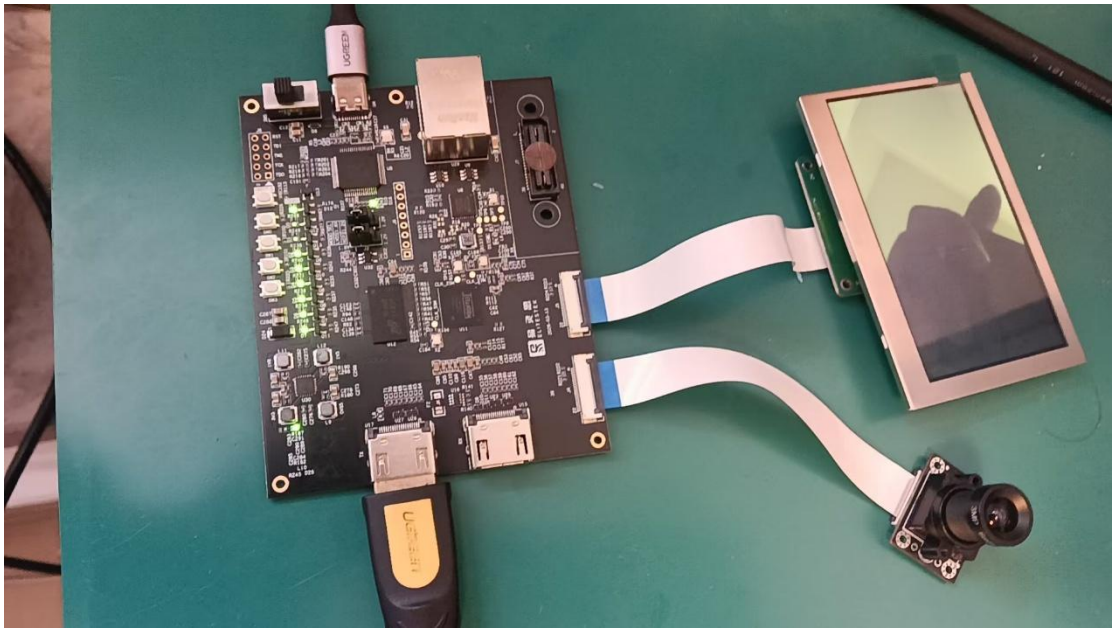


07_mipi_csi2dsi_SC431AI_demo

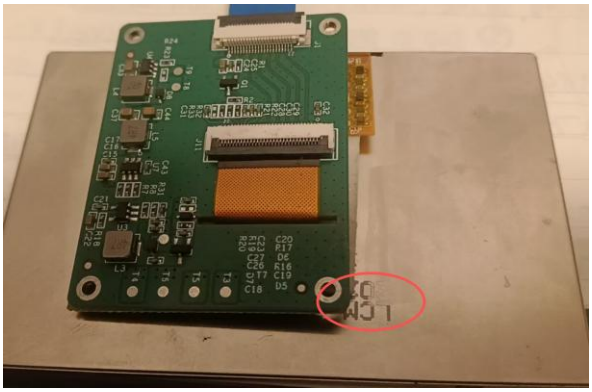
版本说明

rev	说明	软件版本
Ti60f225_csi2dsi_sc431hai_v3.4	(1) 可以同时输出 DSI, (2) 升级软件版本	2025.2.288.4.15
Ti60f225_csi2dsi_sc431hai_v3.4	(1) MIPI 屏的驱动芯片有变化，所以升级了参数。	2025.2.288.4.15

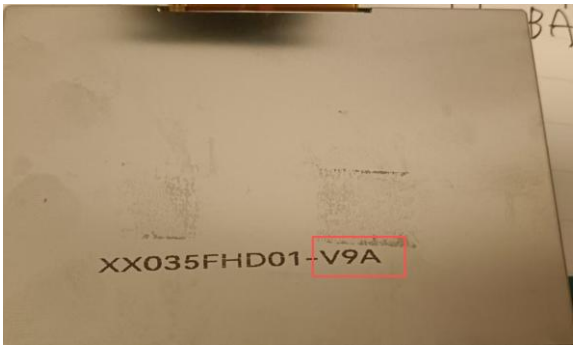
Demo 连接



区分驱动 3.4 与 3.5 的屏



3.4 版本屏



3.5 版本屏

程序变化

（1） 更换了 Panel_1080p_reg.mem 配置参数;

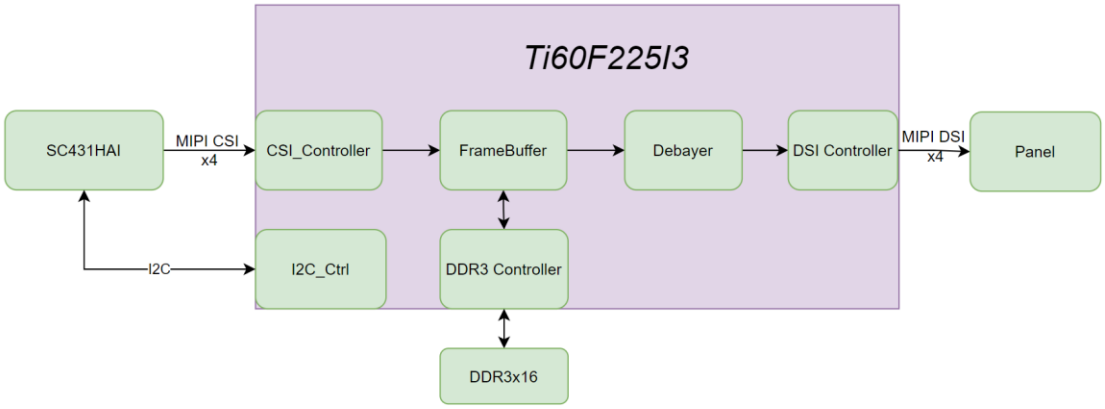
（2） Panel_config 修改如下:

```
panel_config
#(
    .INITIAL_CODE   ("Panel_1080p_reg.mem"),
    .REG_DEPTH      (9'd180)
)
inst_panel_config
```

管脚分配

Interface Name	Pad assigned	
Io_cam_scl	GPIOR_20	
Io_cam_sda	GPIOR_18	
o_cam_rst_p	GPIOR_16	
cam_ck	GPIOT_PN_09	
cam_d0	GPIOT_PN_07	
cam_d1	GPIOT_PN_10	
cam_d2	GPIOT_PN_06	
cam_d3	GPIOT_PN_11	

Demo 说明



（1） CSI RX IP 接收树莓派 sensor IMX219；
（2） I2C_Config 用于配置 sensor 的寄存器；
（3） Framebuffer 用于帧缓存 ；
（4） Axi4FullDeplex 用于实现 AXI 全双工与半双工转换；
（5） Hbram_ctrl 是 Hyperram 的控制器；
（6） Hbram 是 Ti60F100 合封的 hyperram.
（7） Debayer 模块实现 RAW 转 RGB；
（8） Dsi tx 是驱动 panel 的 IP；
（9） Panel_config 用于配置屏的寄存器；

SC431HAI 参数

参数	描述
分辨率	400 万
像素阵列	2568 (H) x 1448 (V)
像素尺寸	2.0 µm x 2.0 µm
镜头光学尺寸	1/3.06”
最大图像传输速率	2560 (H) x 1440 (V) @ 30/60 fps 10-bit 1.2 V
输出接口	8/10-bit 1/2/4-Lane MIPI
输出格式	RAW RGB
CRA	15°
灵敏度	4319 mV/lux·s
动态范围	• 线性模式: 81 dB • HDR 模式: >100 dB
信噪比	38.47 dB
工作温度范围	-30°C ~ +85°C
最佳工作温度范围	-20°C ~ +60°C
电源电压	AVDD = 2.8±0.1 V, DVDD=1.2±0.06 V, DOVDD = 1.8±0.1 V
封装尺寸	CSP, 5.790 mm x 3.554 mm, 35-pin
ESD 等级	• HBM classification: 2 • CDM classification: C3

硬件连接

Sensor 连接转接卡的 J34FPC 座子；Panel 连接转接卡的 J35 FPC 座子

MIPI DSI 使用指导

MIPI Dsi 的使用相比于 CSI 多了一些寄存器控制，生成的 example 直接仿真是可以的，但是修改了参数，一些参数也要做相应的调整。这里我们来做个简单总结 。

（1）MIPI IO 所在的 BANK 电压要配置成 1.2V

（2）输出要接相应 bank 的 REF_RES 管脚。

（3）根据需要设置 IP 关键信号。

名称	参数	说明
MIPI Parallel Clock frequency	50~187.5	指 clk_byte_HS 的时钟频率。
IP core frequency	100	指 mipi_clk 的时钟频率，指定 100MHz
Pack Type 48	Enable, Disable	使能表示打开 48 位的数据类型，比如 RGB888,
Packe Type 64	Enable, Disable	使能表示打开 64 位的数据类型，比如 RGB565. 如果要全部支持可以同时打开 Pack Type48 和 Packe Type 64
Maximum Horizontal Resolution		行像素的最大个数。一定要注意 hactive 的长度，如果设置

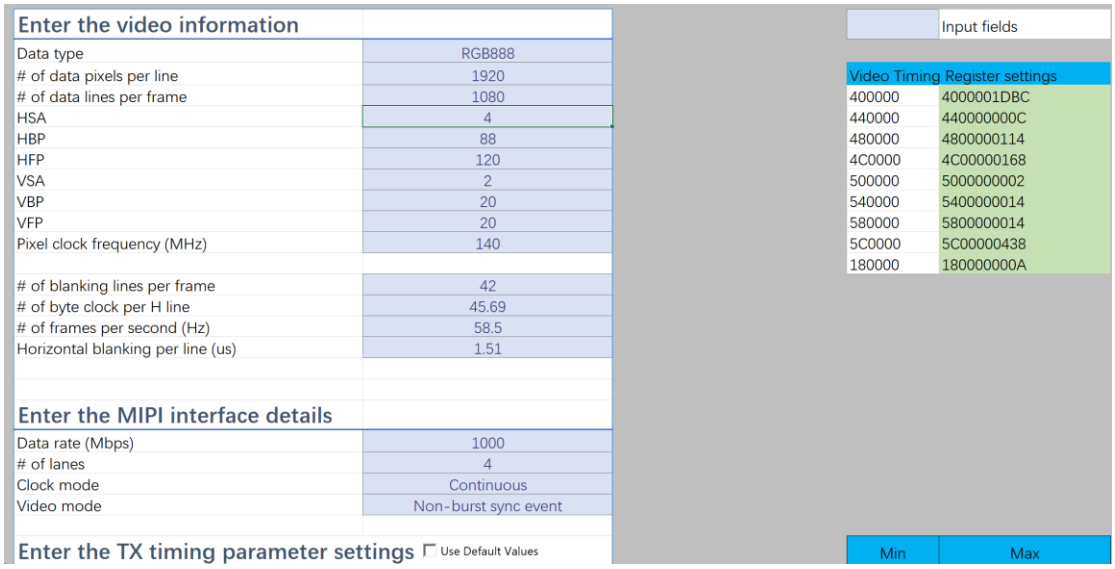
		不正确可能无画面或者画面偏移。
FIFO Pixel Depth Size	最大 4096	FIFO 的 data width 是 64-bit, MIPI IP 里面会缓存 2~3 行才发出去.所以注意这个值要大于一行的一行数据的 2 倍或者 3 倍。
video Transmission Packet Sequences	Non-burst mode with Sync Pulses, Burst mode, Non-burst mode with Sync Events	•Non-Burst Mode with Sync Pulses – 让外设可以精确的重构原始的视频时序，包括同步脉冲宽度。•Non-Burst Mode with Sync Events –与上述类似，但精确重建同步脉冲宽度不是必需的。•Burst mode – RGB 像素包在时间上有压缩，留出更多的时间用于 LP 模式，或者用于复用别的传输。

（4）修改配置参数

视频格式如下，可能根据实际的需要进行修改。

```
parameter   MAX_HRES      = 12'd1920,
parameter   MAX_VRES      = 12'd1080,
parameter   HSP           = 8'd4,
parameter   HBP           = 8'd88,
parameter   HFP           = 8'd120,
parameter   VSP           = 8'd2,
parameter   VBP           = 8'd20,
parameter   VFP           = 8'd20
```

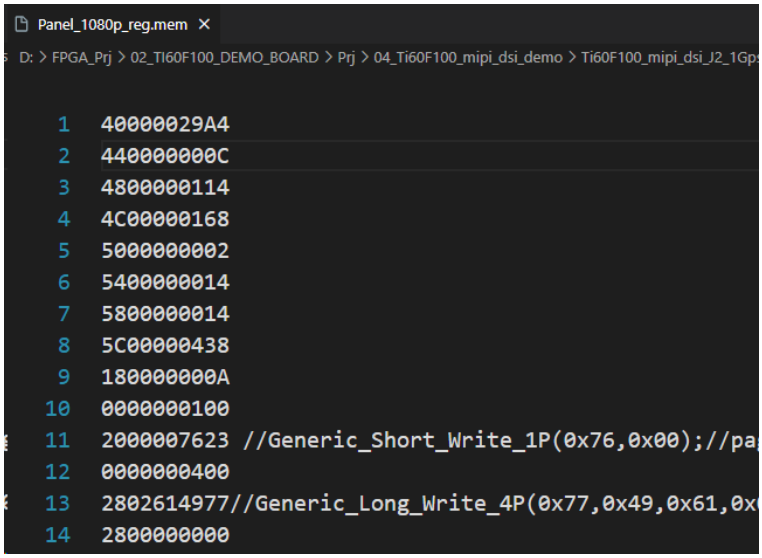
修改参数后要在 Tianium-mipi-utility 文件中验证，确保文件没有错误提示。



Enter the video information	
Data type	RGB888
# of data pixels per line	1920
# of data lines per frame	1080
HSA	4
HBP	88
HFP	120
VSA	2
VBP	20
VFP	20
Pixel clock frequency (MHz)	140
# of blanking lines per frame	42
# of byte clock per H line	45.69
# of frames per second (Hz)	58.5
Horizontal blanking per line (us)	1.51
Enter the MIPI interface details	
Data rate (Mbps)	1000
# of lanes	4
Clock mode	Continuous
Video mode	Non-burst sync event
Enter the TX timing parameter settings	Use Default Values

	Input fields
Video Timing Register settings	
400000	4000001DBC
440000	440000000C
480000	4800000114
4C0000	4C00000168
500000	5000000002
540000	5400000014
580000	5800000014
5C0000	5C00000438
180000	180000000A
	Min / Max

然后要把 Tianium-mipi-utility 文件中 input fields 中的信息替换 Panel_1080p_reg.mem 中的相应信息。这是 IP 需要的视频格式信息。



08_Ti60F225_hardjtag_demo

Soc 入门主请先参考下面公众号的基本使用说明。

Efinity RISC-V IDE 入门使用-4

RISCV 操作常见问题集 - v7

RISCV soft JTAG 调试_v1.2

自定义 RISC V 的 bootloader-v3

版本说明

版本	说明	日期
V1.1	添加 flash x4 demo	2025.2.288.4.15

环境说明

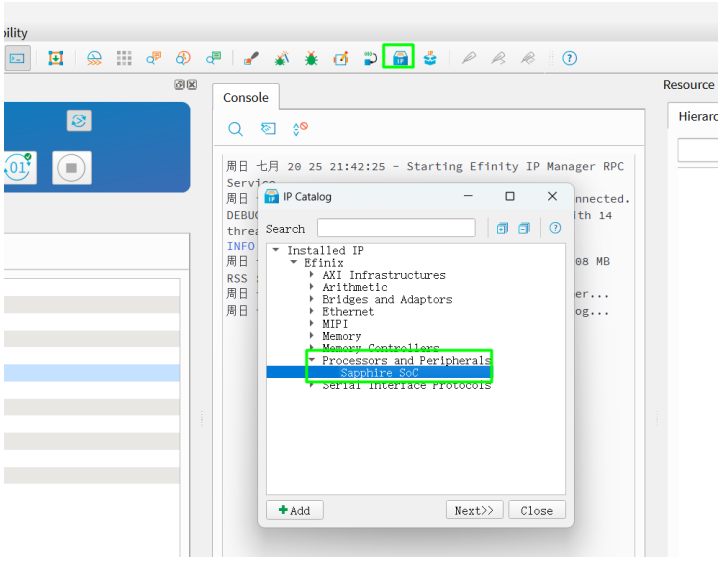
软件版本：v2025.2

Efinity 版本：2025.2

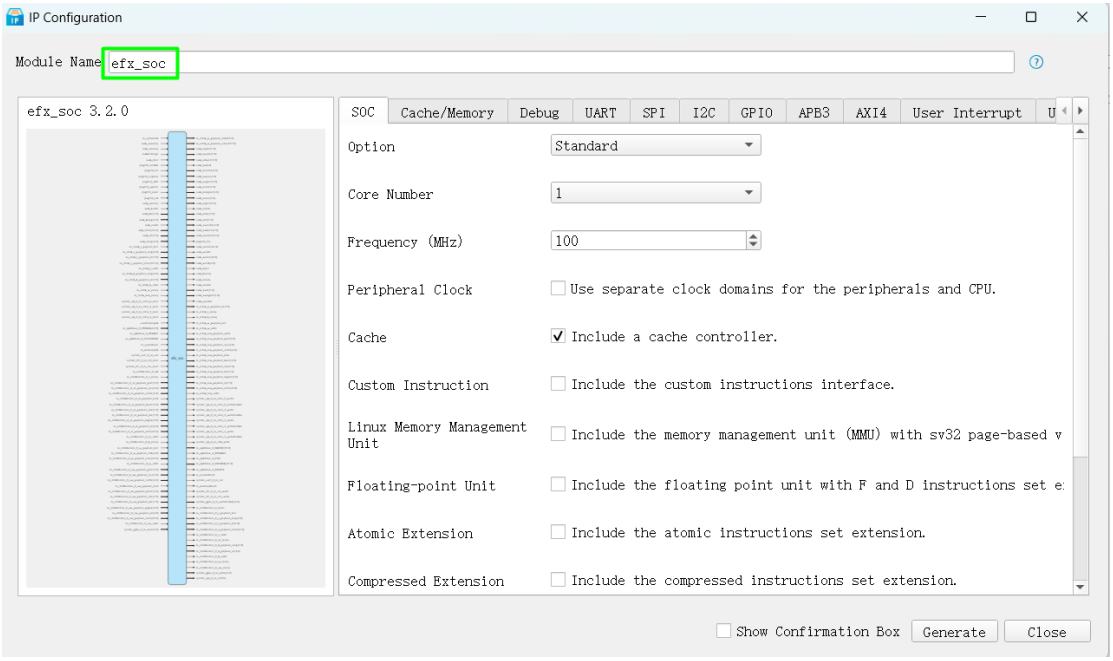
工程说明

(1)逻辑工程配置

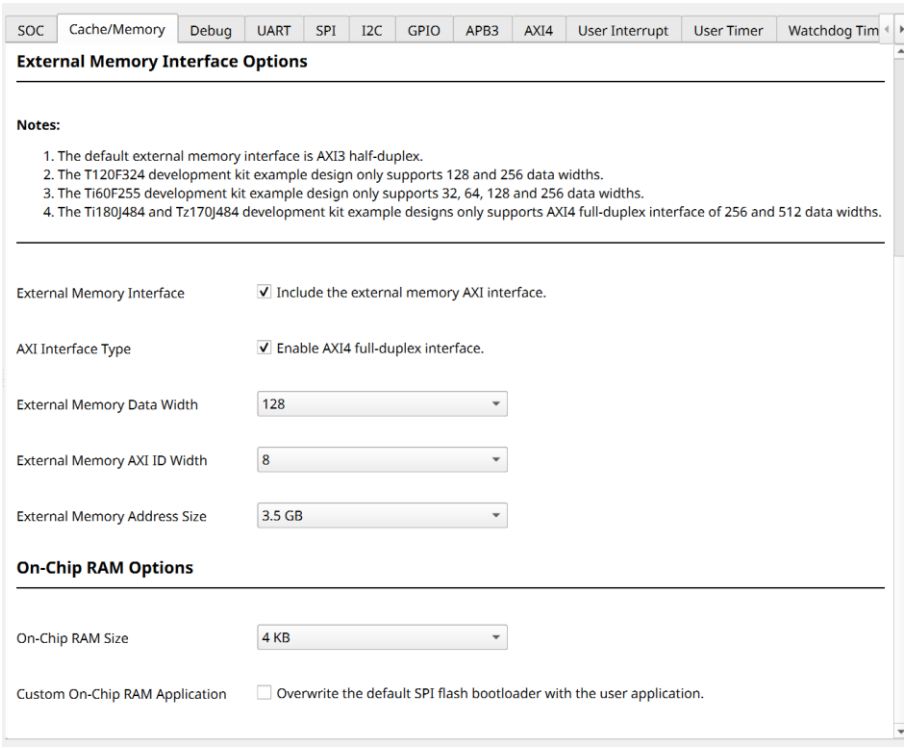
1） 添加 RISCV IP。点击 Open IP Catlog->Processors and Peripherals -> Sapphire Soc



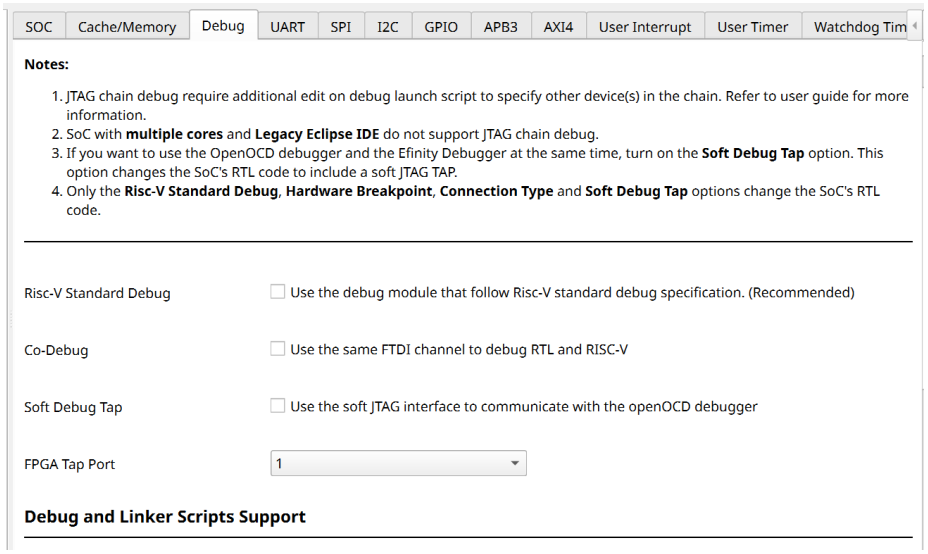
输入 Module Name 为 efx_soc，在使用 softJTAG 的过程中，首先要把 RISCV 的 IP 的调试选项选择 use the soft JTAG interface to communicate with the openOCD debugger 打开，然后点击 Generate 生成 IP.



在 Cache/Memory 选择打开外部存储器 AXI 接口，并且使能双工，其余保持默认配置。



在 Debug 界面使用默认配置。其余接口也保持默认配置。



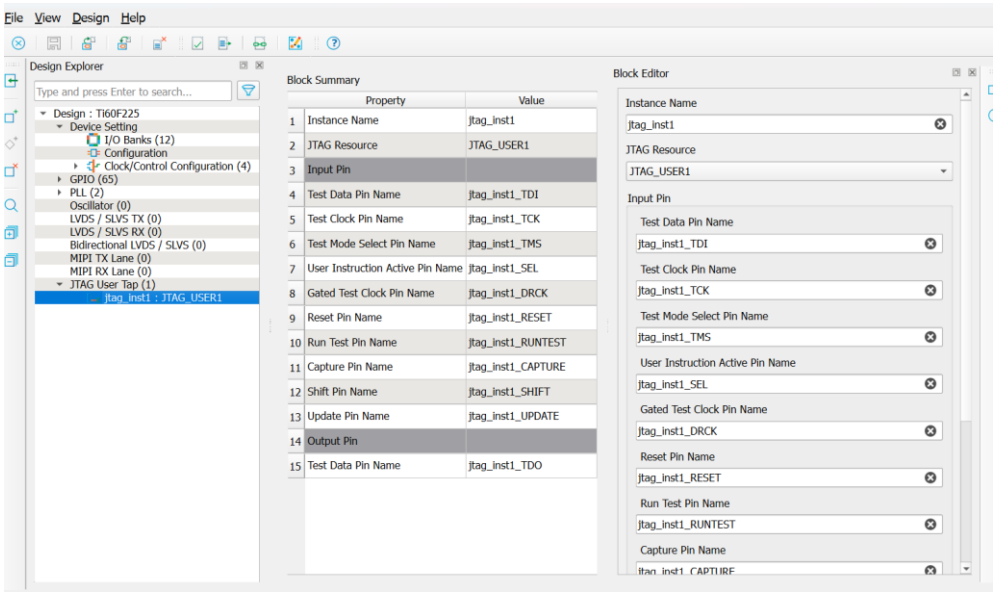
点击右下角的 generate，在 IP 的 efx_soc 目录下会生成 efx_soc_tmpl.v 例化文件，例化内部如下，只需要复制即可以使用。

```
efx_soc u_efx_soc
(
    .io_systemClk ( io_systemClk ),
    .io_ddrA_w_payload_strb ( io_ddrA_w_payload_strb ),
    .io_ddrA_w_payload_data ( io_ddrA_w_payload_data ),
    .io_ddrA_w_payload_id ( io_ddrA_w_payload_id ),
    .io_ddrA_r_payload_last ( io_ddrA_r_payload_last ),
    .io_ddrA_r_payload_resp ( io_ddrA_r_payload_resp ),
    .io_ddrA_r_payload_id ( io_ddrA_r_payload_id ),
    .io_ddrA_r_payload_data ( io_ddrA_r_payload_data ),
    .io_ddrA_r_ready ( io_ddrA_r_ready ),
    .io_ddrA_r_valid ( io_ddrA_r_valid ),
    .io_ddrA_b_payload_resp ( io_ddrA_b_payload_resp ),
    .io_ddrA_b_payload_id ( io_ddrA_b_payload_id ),
    .io_ddrA_b_ready ( io_ddrA_b_ready ),
    .io_ddrA_b_valid ( io_ddrA_b_valid ),
    .io_ddrA_w_payload_last ( io_ddrA_w_payload_last ),
    .io_ddrA_w_ready ( io_ddrA_w_ready ),
    .io_ddrA_w_valid ( io_ddrA_w_valid ),
    .io_ddrA_arw_payload_write ( io_ddrA_arw_payload_write ),
    .io_ddrA_arw_payload_prot ( io_ddrA_arw_payload_prot ),
    .io_ddrA_arw_payload_qos ( io_ddrA_arw_payload_qos ),
    .io_ddrA_arw_payload_cache ( io_ddrA_arw_payload_cache ),
    .io_ddrA_arw_payload_lock ( io_ddrA_arw_payload_lock ),
    .io_ddrA_arw_payload_burst ( io_ddrA_arw_payload_burst ),
    .io_ddrA_arw_payload_size ( io_ddrA_arw_payload_size ),
    .io_ddrA_arw_payload_len ( io_ddrA_arw_payload_len ),
    .io_ddrA_arw_payload_region ( io_ddrA_arw_payload_region ),
    .io_ddrA_arw_payload_id ( io_ddrA_arw_payload_id ),
    .io_ddrA_arw_payload_addr ( io_ddrA_arw_payload_addr ),
    .io_ddrA_arw_ready ( io_ddrA_arw_ready ),
    .io_ddrA_arw_valid ( io_ddrA_arw_valid ),
    .system_spi_0_io_data_0_read ( system_spi_0_io_data_0_read ),
    .system_spi_0_io_data_0_write ( system_spi_0_io_data_0_write ),
```

```
    .system_spi_0_io_data_0_writeEnable ( system_spi_0_io_data_0_writeEnable ),
    .system_spi_0_io_data_1_read ( system_spi_0_io_data_1_read ),
    .system_spi_0_io_data_1_write ( system_spi_0_io_data_1_write ),
    .system_spi_0_io_data_1_writeEnable ( system_spi_0_io_data_1_writeEnable ),
    .system_spi_0_io_data_2_read ( system_spi_0_io_data_2_read ),
    .system_spi_0_io_data_2_write ( system_spi_0_io_data_2_write ),
    .system_spi_0_io_data_2_writeEnable ( system_spi_0_io_data_2_writeEnable ),
    .system_spi_0_io_data_3_read ( system_spi_0_io_data_3_read ),
    .system_spi_0_io_data_3_write ( system_spi_0_io_data_3_write ),
    .system_spi_0_io_data_3_writeEnable ( system_spi_0_io_data_3_writeEnable ),
    .system_spi_0_io_sclk_write ( system_spi_0_io_sclk_write ),
    .userInterruptA ( userInterruptA ),
    .io_apbSlave_0_PADDR ( io_apbSlave_0_PADDR ),
    .io_apbSlave_0_PENABLE ( io_apbSlave_0_PENABLE ),
    .io_apbSlave_0_PRDATA ( io_apbSlave_0_PRDATA ),
    .io_apbSlave_0_PREADY ( io_apbSlave_0_PREADY ),
    .io_apbSlave_0_PSEL ( io_apbSlave_0_PSEL ),
    .io_apbSlave_0_PSLVERROR ( io_apbSlave_0_PSLVERROR ),
    .io_apbSlave_0_PWDATA ( io_apbSlave_0_PWDATA ),
    .io_apbSlave_0_PWRITE ( io_apbSlave_0_PWRITE ),
    .io_asyncReset ( io_asyncReset ),
    .io_memoryClk ( io_memoryClk ),
    .io_systemReset ( io_systemReset ),
    .system_uart_0_io_txd ( system_uart_0_io_txd ),
    .io_memoryReset ( io_memoryReset ),
    .system_uart_0_io_rxd ( system_uart_0_io_rxd ),
    .system_gpio_0_io_writeEnable ( system_gpio_0_io_writeEnable ),
    .system_gpio_0_io_write ( system_gpio_0_io_write ),
    .system_gpio_0_io_read ( system_gpio_0_io_read ),
    .system_spi_0_io_ss ( system_spi_0_io_ss )
);
```

另外 IP 生成时会在工程目录下同时生成一个 embedded_sw 文件夹，里面包括 RISCV 应用所需要的配置信息和测试工程。

Interface IO 连接。SOC 的 JTAG 使用的是 JTAG1，配置如下，其余请参考 interface.

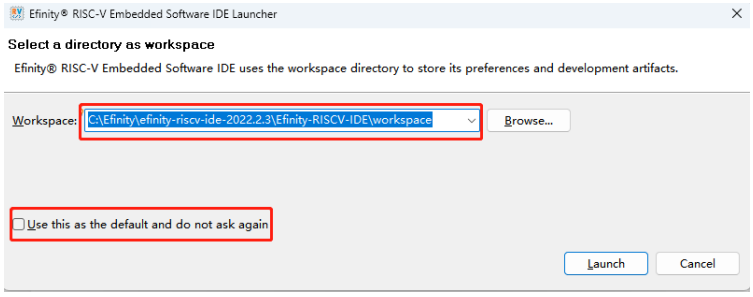


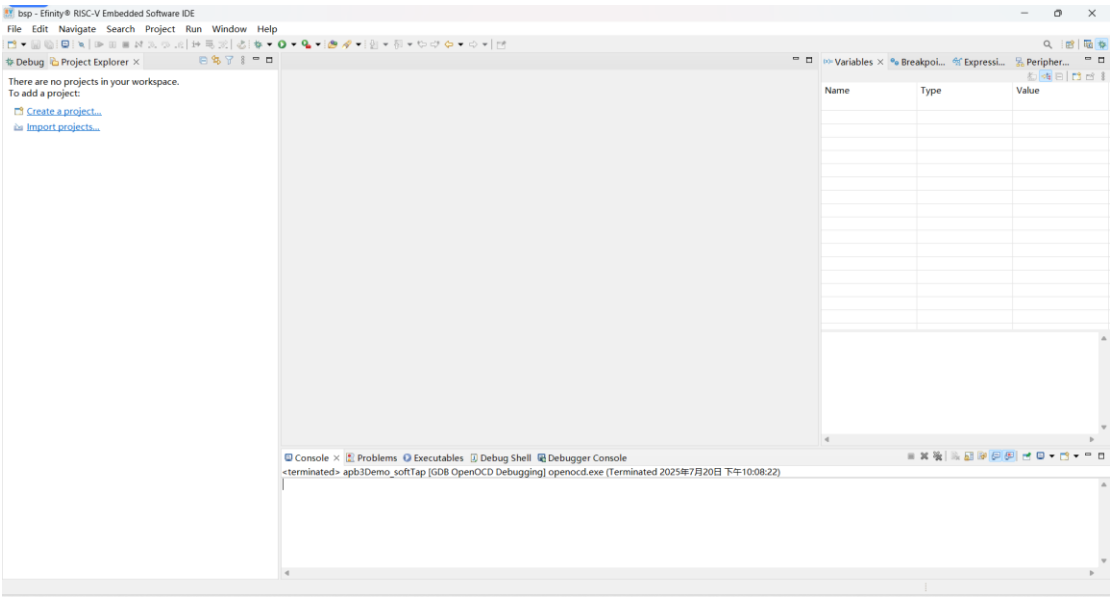
编译工程。工程准备完成后要编译，生成相应的 hex 和 bit 文件。

(2)打开 RSICV IDE 自带工程。

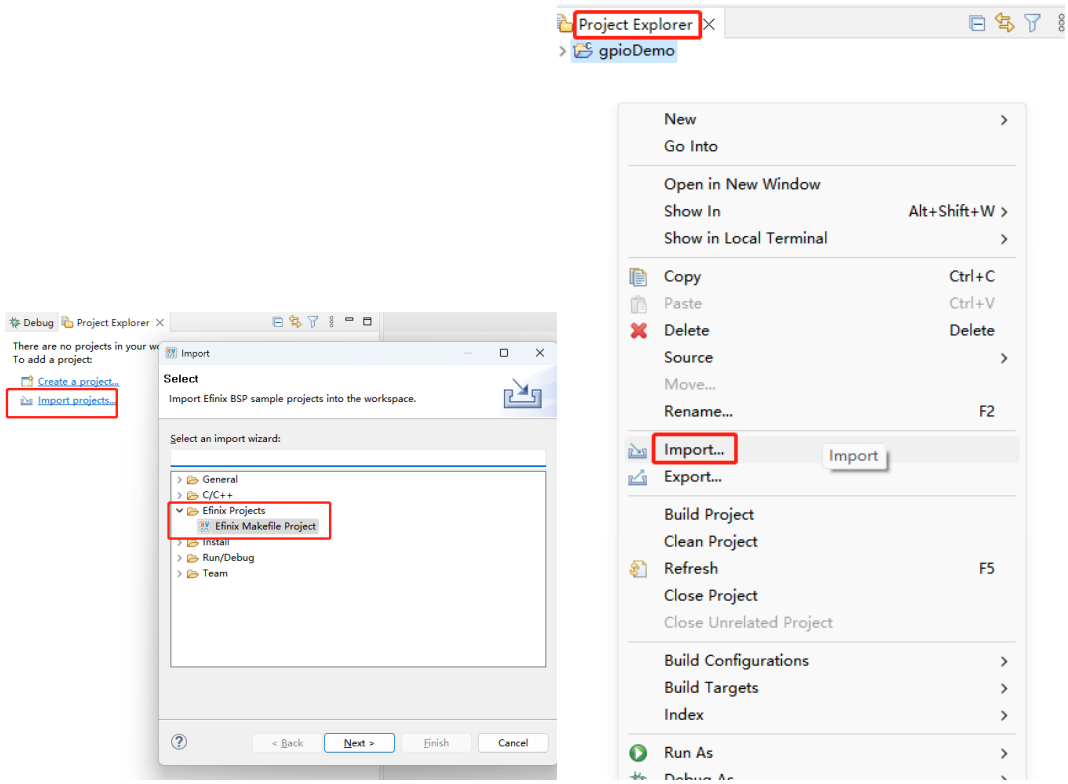
（1）首先打开软件。

step1:选择工程的工作空间。如果工作空间长时间不变可以勾选 Use this as the default and do not ask agin。对于 2025.2 版本可以将该路径可以选择到任何地方。但我们一般指向 IP 生成的 embedded_sw 目录下的 bsp 文件，这样可以方便导入配置信息。

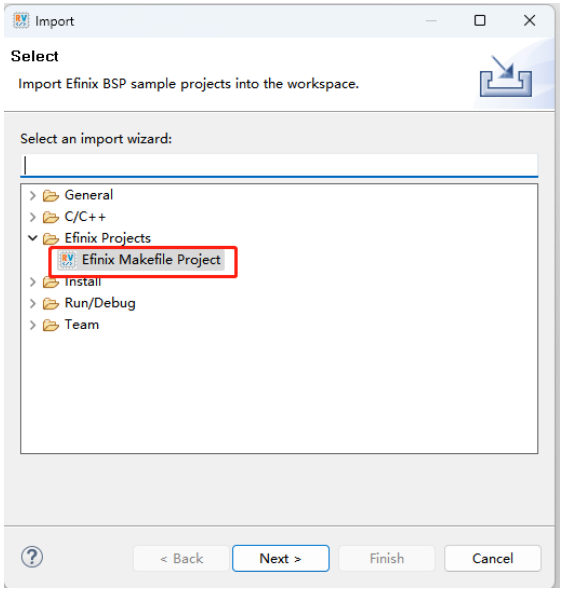




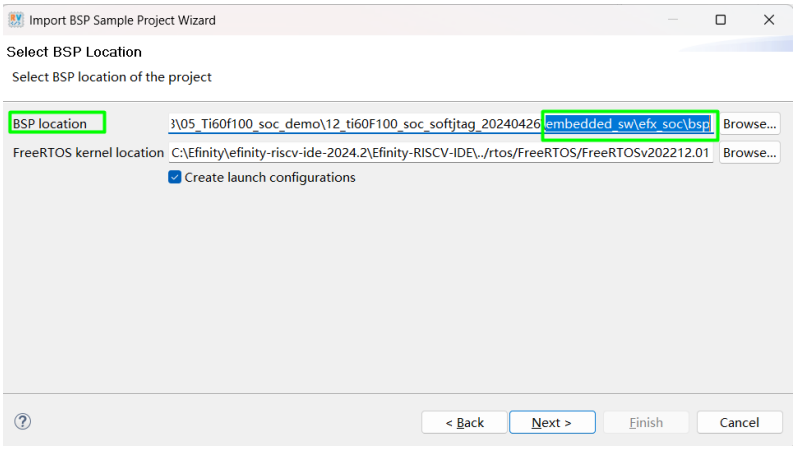
step2: Import 测试工程。选择 Import Projectes... 或者在 Project Explorer 中右击，然后选择 Import... 或者 File -> Import.



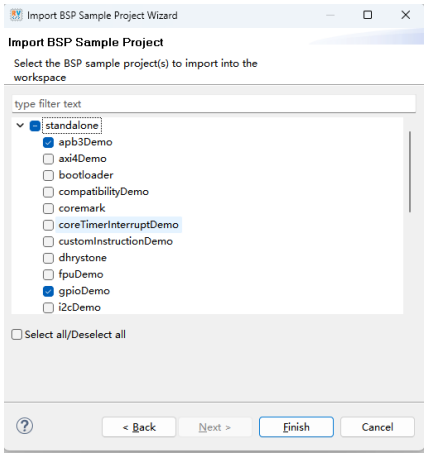
step3:在打开的 Import 对话框中选择 Efinix Projects-> Efinix Makefile Project 可以导入。



step4:选择 bsp 位置。如果是 FreeRTOS 的话，两个都要输入。

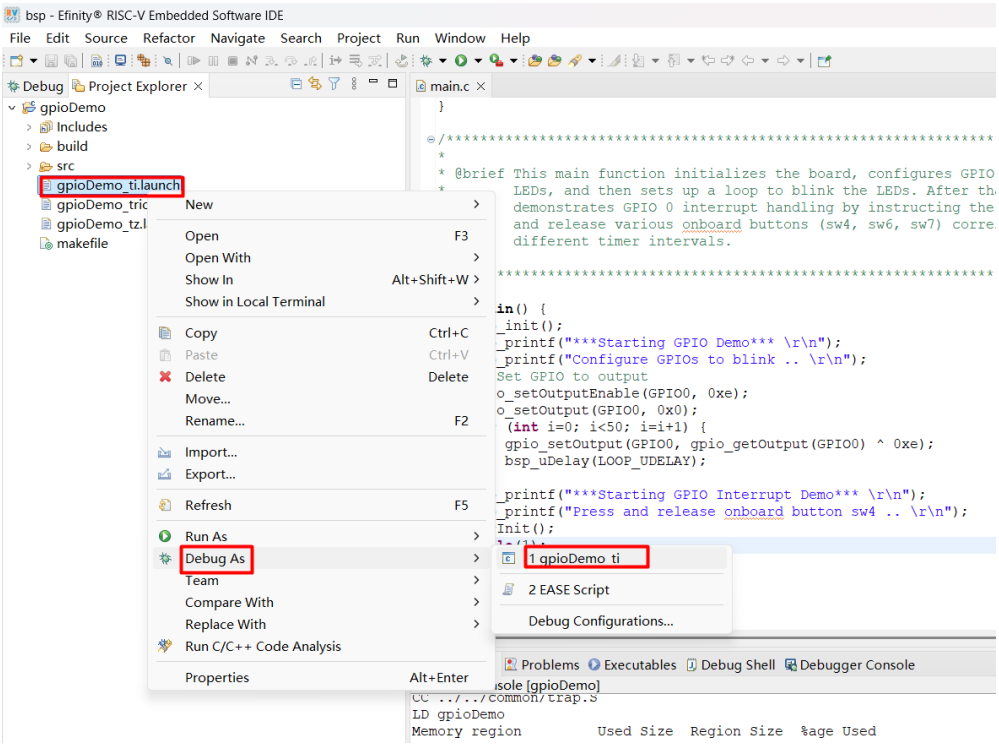


setp5:选择下一步，然后勾选相应工程前面的方框，点击 finish 即可以导入相应的 demo 工程。

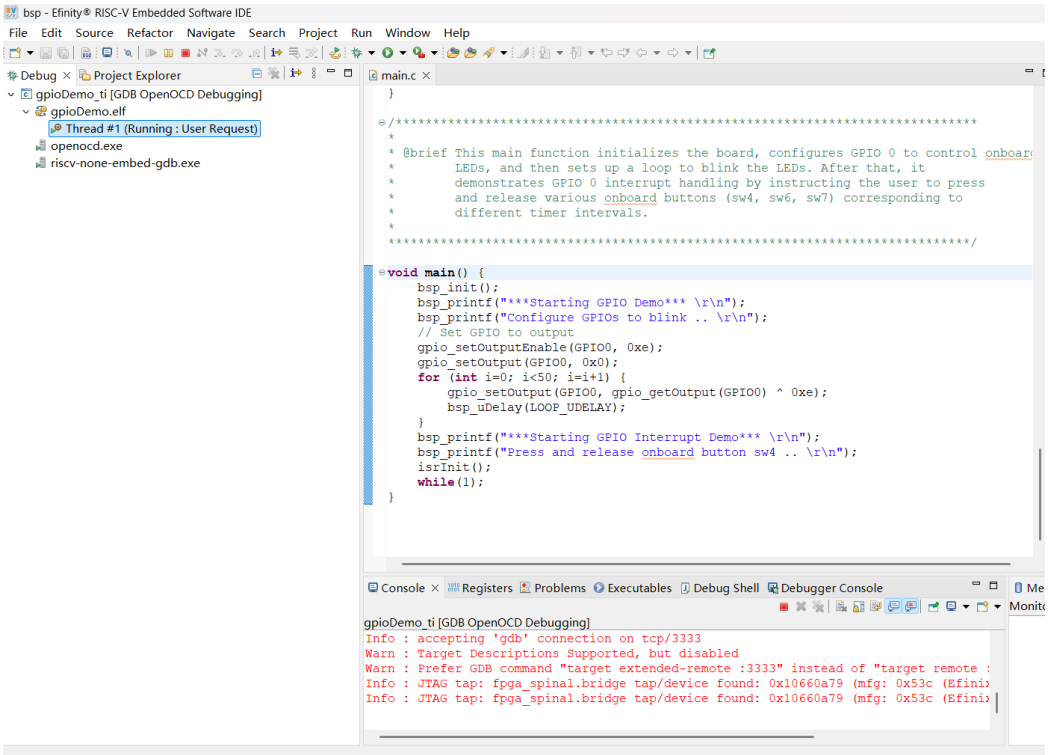


step6:右键选择 build Project.

step7:run 或者 debug。在 debug 之前需要先把逻辑部分的 bit 或者 hex 文件配置到 FPGA，并确保逻辑部分已经运行起来。这里以 gpioDemo 为例。右击 gpioDemo_ti.launch -> Debug As -> gpioDemo_ti,进入 debug 界面

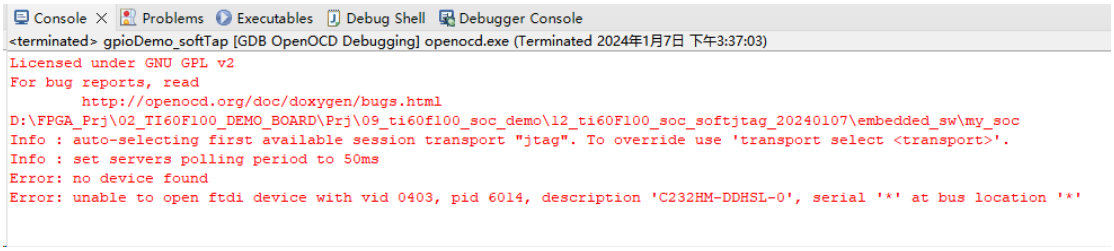


正常进入 debug 界面如下。



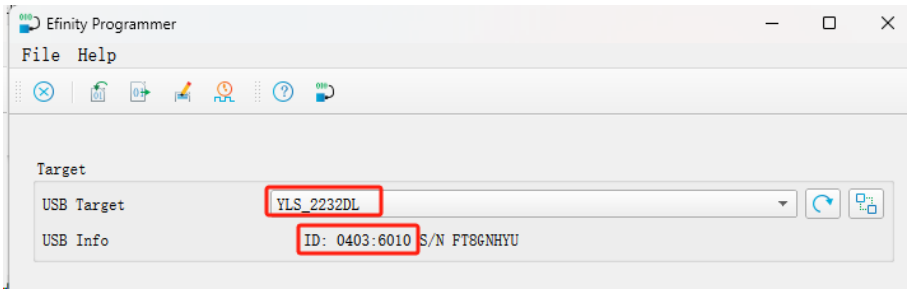
Debug 界面

第一次 debug 可能会出现以下错误提示。



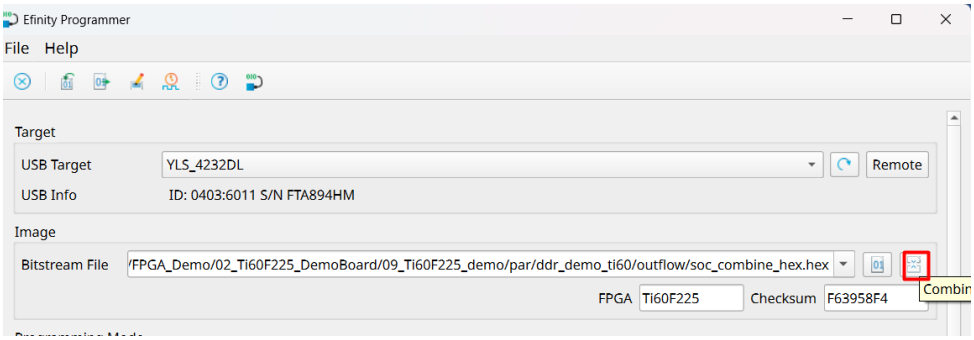
需要修改路径"…\embedded_sw\my_soc/bsp/efinix/EfxSapphireSoc/openocd/*ftdi_ti.cfg*"文件下的配置。与之与使用的下载器对应。

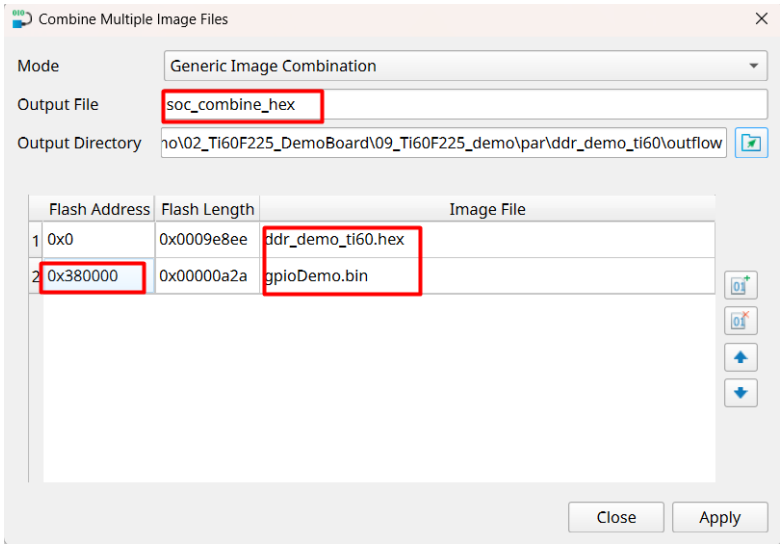
```
if { $LEGACY_IDE == 0 } {
    adapter driver ftdi
    //ftdi device_desc "YLS_4232DL"
    ftdi vid_pid 0x0403 0x6011
    ftdi channel 1
```



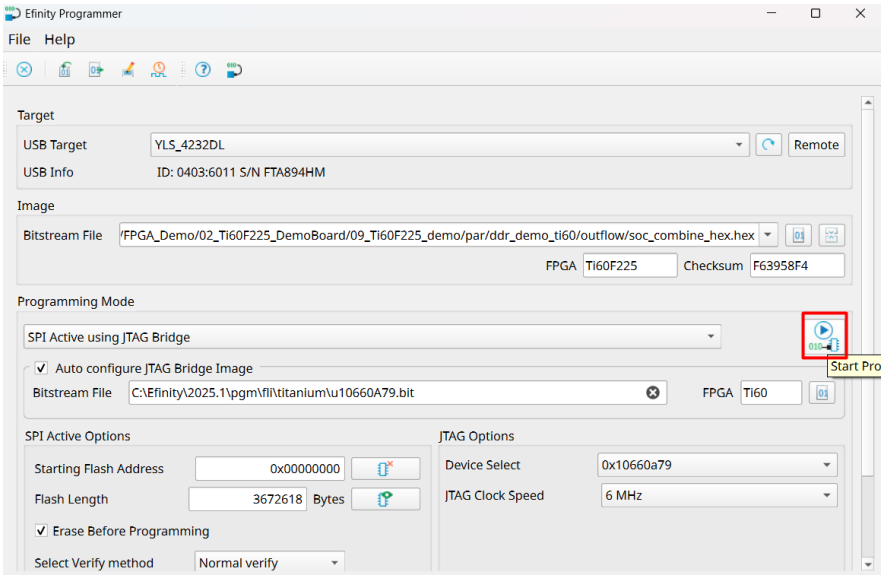
(3)合成固化文件

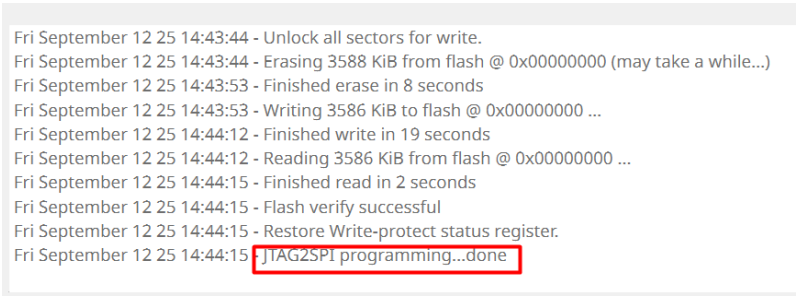
程序调试完成后，需要考虑程序固化到 flash 的问题。
这里以 soc 的 gpiodemo 为例，把逻辑部分 hex 文件和 gpioDemo.bin 合成一个 soc_combine_hex.hex 文件。在 Programmer 点击"Combine Multiple Image Files"打开文件合成界面。注意逻辑部分地址必须指定为 0，而 soc 部分在下图中存储的地址为 0x380000，这个地址是由 bootloader 来指定的，如果没有修改 bootlaoder 地址默认为 0x38000.





固化程序。在 programmer 中选择 soc_combine_hex.hex 文件，Programming Mode 选择“SPI Active using JTAG Bridge”。然后点击”Start Program”，当提示“JTAG2SPI programming···done“说明固化完成。



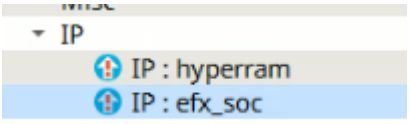


程序烧写设置界面

程序烧写完成之后，对 demo 板重新上电，可以看到 led 灯闪烁。

(4)IP 更新注意事项

（1）在用新版本的 Efinity 打开老版本的工程时，会提示需要升级，IP 界面提示如下:



当升级 IP 时报

（1）Error [GENERATE_IP-ERROR] Build script failed in build_command, 'java' 不是内部或外部命令，也不是可运行的程序或批处理文件。

说明需要安装 java8，java8 在我们提供的软件安装包中 jre-8u381-windows-x64.exe，注意，

（1）java8 安装之后要重启 efinity 再次更新才能成功。

（2）IP 升级时要关闭 RISCV IDE 软件，因为很多时候 RISCV IDE 的路径指向 IP 的 softwave 路径。

（3）IP 之后之后再次 debug 时可能需要重新修改 cfg 文件的，就是以下三个参数，否则再次 debug 的时候会指示错误。

```
ftdi device_desc "Quad RS232-HS"
ftdi vid_pid 0x0403 0x6011
ftdi channel 0
```

09_Ti60F225_co_debug_demo

并发调试（Enable Concurrent Debugging）

自 Efinity RISC-V 嵌入式软件 v2025.2 版本起，新增了一种调试流程，称为「并发调试」（concurrent debugging）。在并发调试模式下，可通过单个 FTDI JTAG 接口，同时进行 RTL 调试与 RISC-V 内核调试，当然两者使用不同的 USER TAP.

你需要安装以下软件：

- Efinity 软件 v2025.2 版本
- Efinity RISC-V 嵌入式软件集成开发环境（IDE）v2025.2 版本

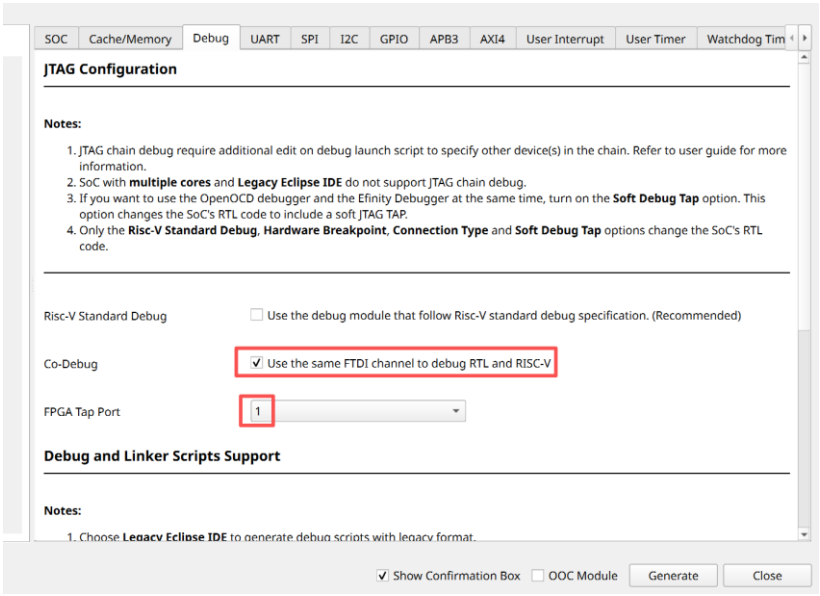
若要使用并发调试功能，请遵循以下步骤：

1. 在「**HBR**」选项卡中，启用 efx_soc IP 配置里的「**联合调试（Co-Debug）**」选项。
2. 在 Efinity 调试向导（Efinity Debug Wizard）中，配置信号并编译工程。
3. 下载 bit 文件。
4. 启动 RISC-V 嵌入式软件 IDE，导入目标工程。
5. 右键单击「group.launch」文件，选择「运行 / 调试（Run/Debug）」。
6. 打开调试器（Debugger），先启用「共享（Shared）」选项，再连接 connect debugger
7. 使用调试器执行 RTL 调试。

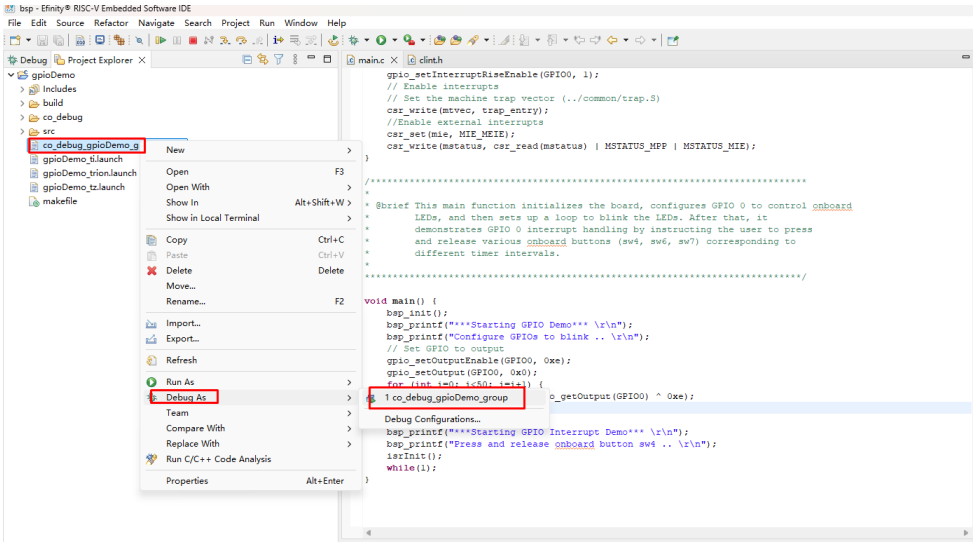
你可同时进行两类调试操作，二者互不干扰。

注意：联合调试（Co-Debug）仅支持 FPGA 的 JTAG 用户测试访问端口（JTAG User TAP interface）。启用联合调试后，「软件调试端口（Soft Debug Tap）」选项将隐藏；反之，启用软件调试端口时，联合调试选项会隐藏。

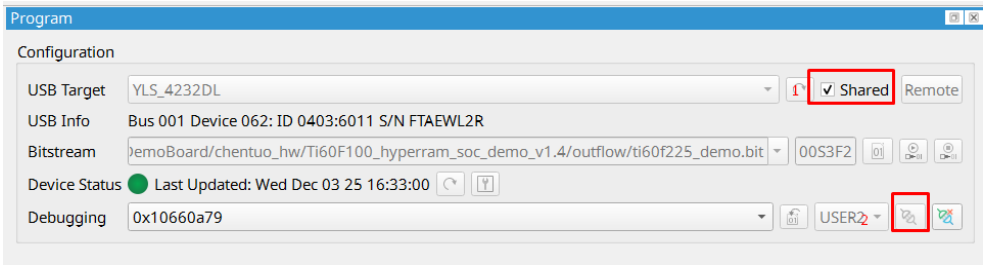
Step1:



Step5:



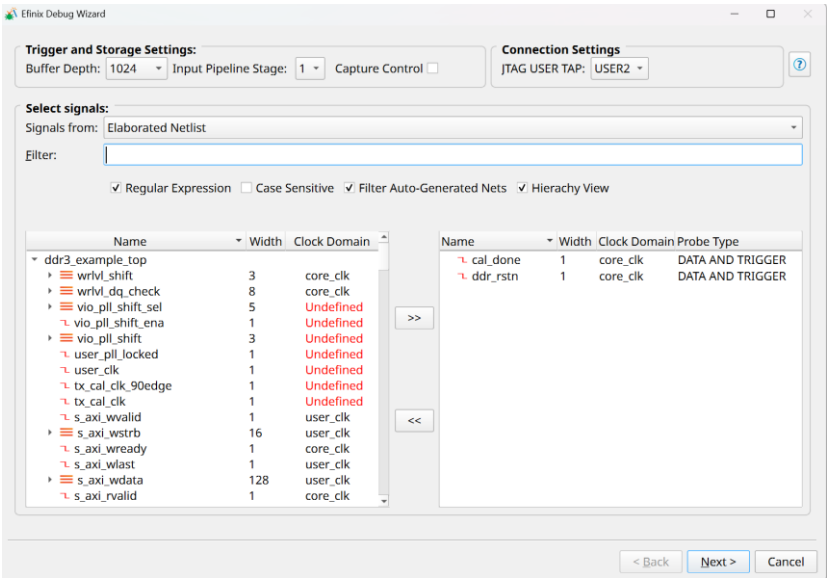
Step7:



测试说明

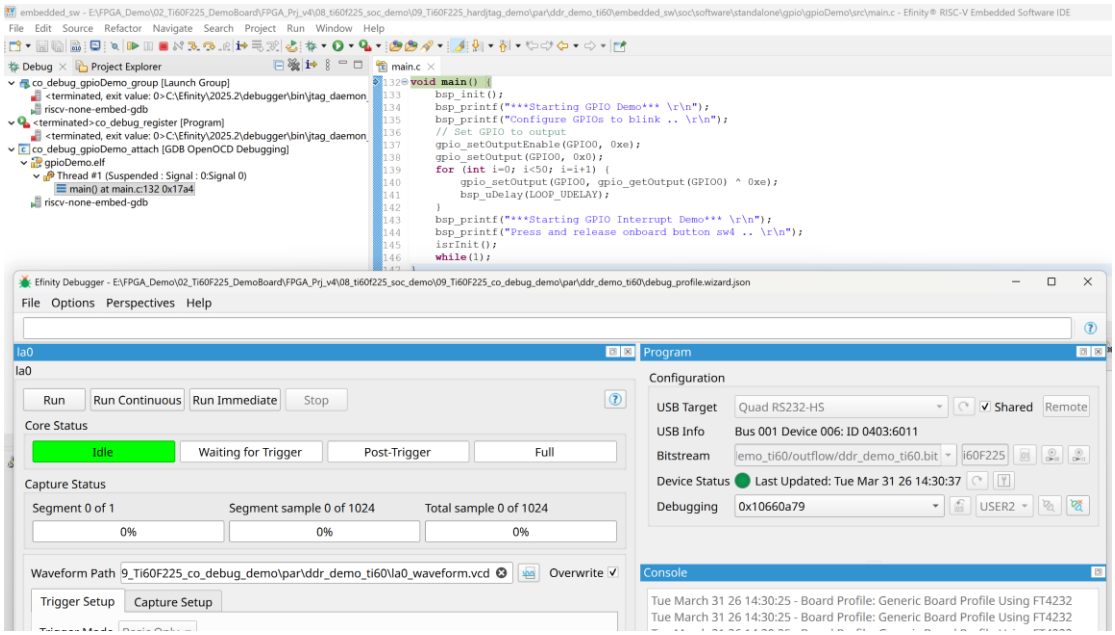
联合调试，勾选“Use the same FTDI channel to debug RTL and RISC-V”，另外注意 FPGA Tap Port 选择的是 1。

在 Debug Wizard 中选择的 JTAG USER TAP 为“USER2”



确认好上面的选项之后，生成 IP 再重新编译工程。

效果测试：



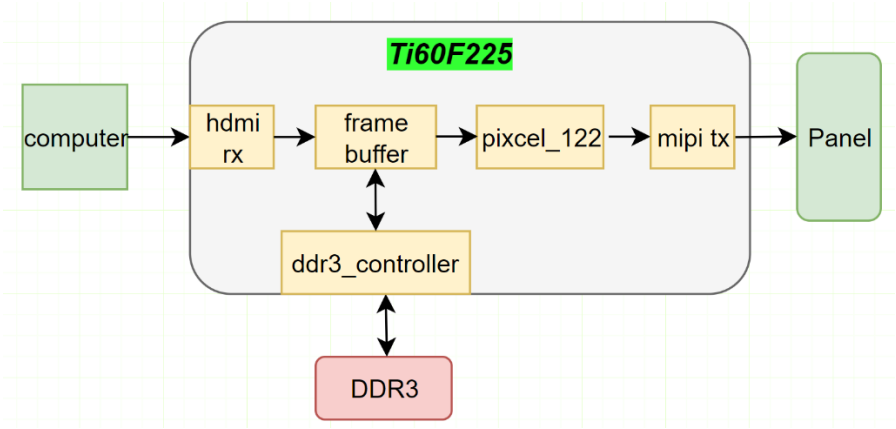
09_Ti60F225_hdmi2dsi_demo

版本说明

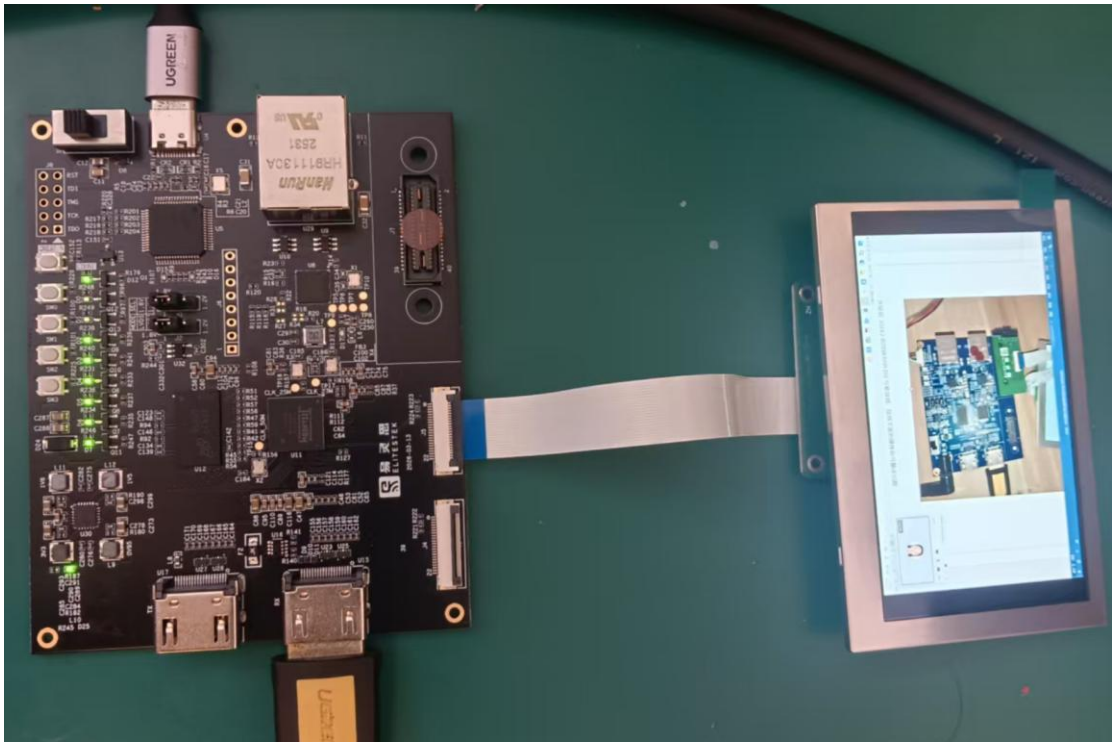
版本	说明	Efinity 版本
Ti60F225_hdmi2dsi_J7_1Gpsb_V10		2025.2.288.4.15

该 demo 实现从 HDMI 输入到 MIPI DSI 输出。

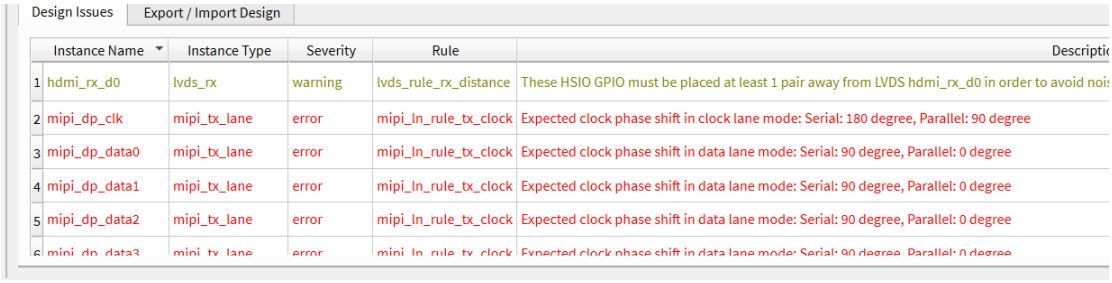
（1） 工程框图



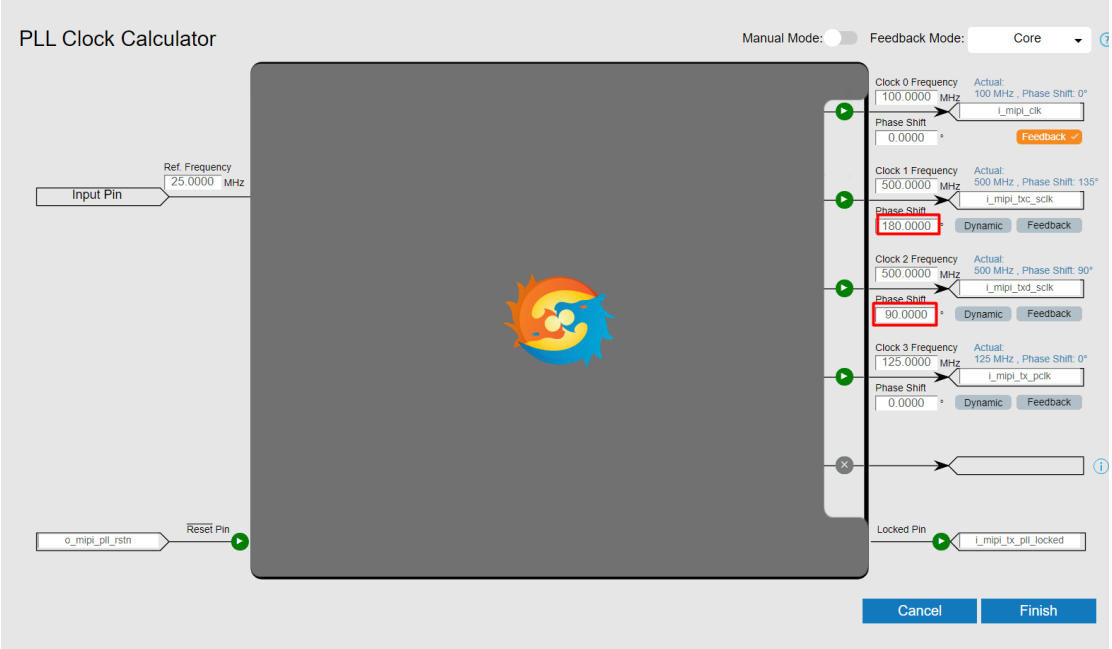
硬件连接如下：



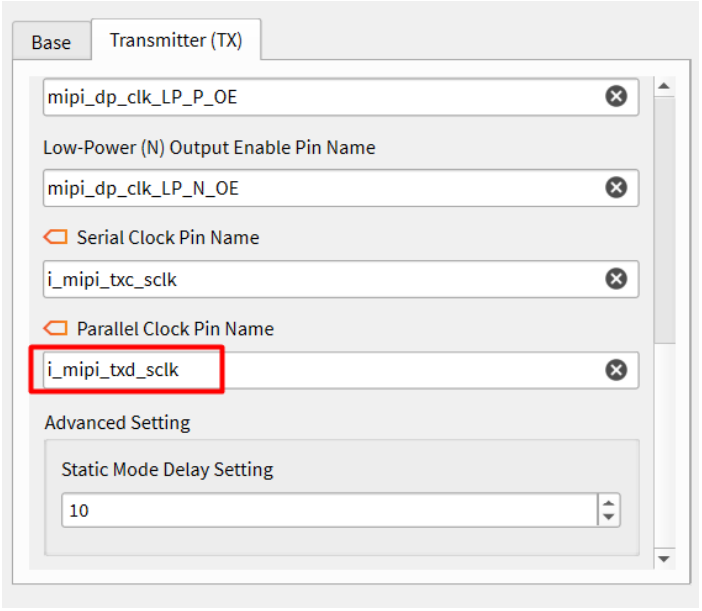
升级到 2024.2 的时钟 MIPI DSI 可能报错，按照下面的操作即可解决问题



（1）把时钟相位按下面修改



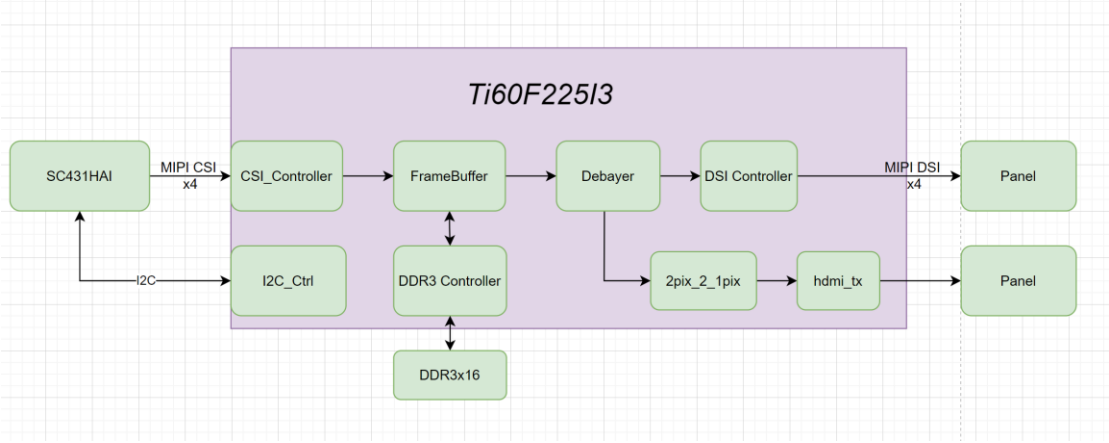
（2）把 mipi tx 时钟的 Parallel Clock 修改为 i_mipi_txd_sclk。



10_Ti60f225_csi2hdmi_demo

软件说明	版本说明	
Ti60f225_sc431hai2hdmi_v0	2025.2.288.4.15	(1)hdmi 的 vs,hs 进行了交换
Ti60f225_sc431hai2hdmi_v3		
Ti60f225_sc431hai2hdmi_v4	2025.2.288.4.15	（1）图像做镜像与翻转； （2）优化了 rgb_gain 模块 （3）去掉了 dsi 输出部分 （4）去除部分无用代码 （5）增加了曝光时间；增加了 ANA 增益控制寄存器 （6）MIPI CSI RX 版本从 5.8 升级到了 5.15
Ti60f225_sc431hai2hdmi_v5	2026.1.132.2.19	（1）efinity 版本升级； （2）升级 CSI RX IP 未升级，因为升级之后 vs 解析不出来。 （3）删除 vga_gen.v 模块
Ti60f225_sc431hai2hdmi_v7	2026.1.132.2.19	（1）升级了 framebuffer_axi 接口

一、Demo 框图



(1) Csi_rx_controller 模块。MIPI CSI 控制器;

(2) I2c_master_ctrl_top 模块。通过 I2C 配置 sc431hai 的参数配置;

(3) Sensor_clipper 模块。把信号源剪裁到 1920x1080 分辨率;

(4) Frame_buffer 模块。视频帧缓存模块，用于实现视频读写到 ddr3;

(5) Ddr3_top 模块。Ddr3 控制器模块;

(6) Debyer_top 模块。对 RAW 数据进行 debyer 处理;

(7) Dsi_tx IP 模块。用于实现 DSI 数据发送;

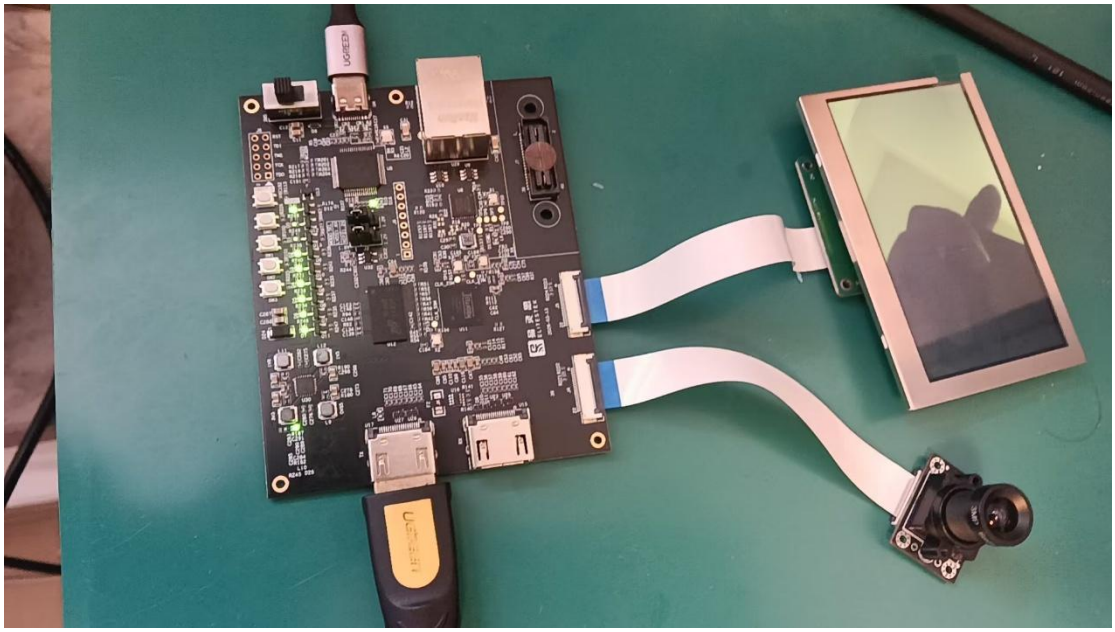
(8) Panel_config 模块，用于发送 panel 的配置数据;

(9) Dvi_encoder 模块。HDMI 发送 IP;

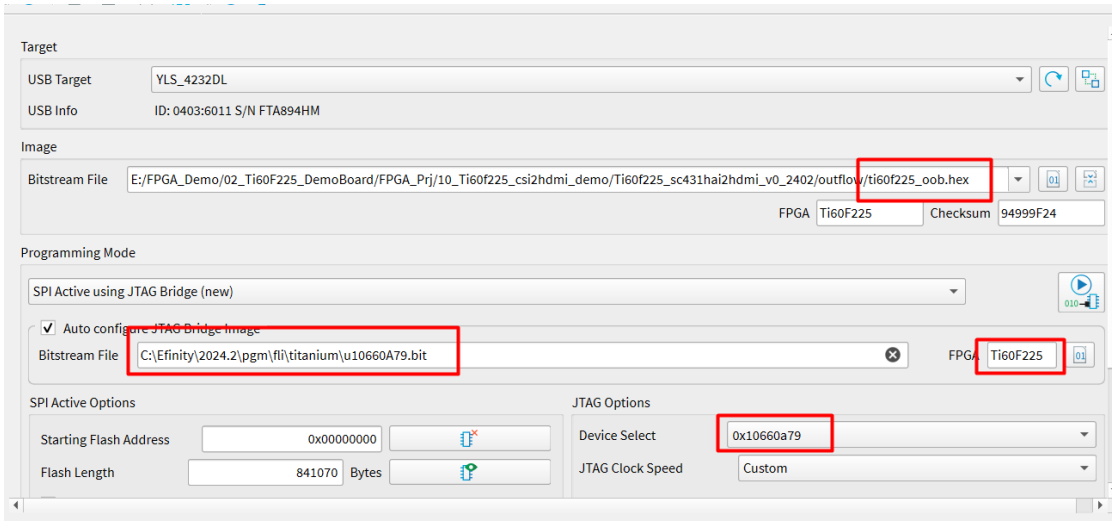
环境准备

（1）确保 J2,J3 选择电压为 1.2V

硬件测试



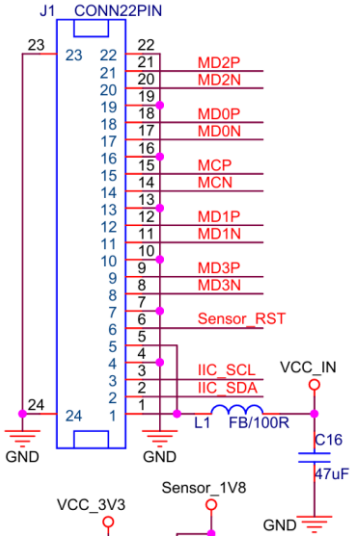
烧写程序，然后重启电源查看结果 。

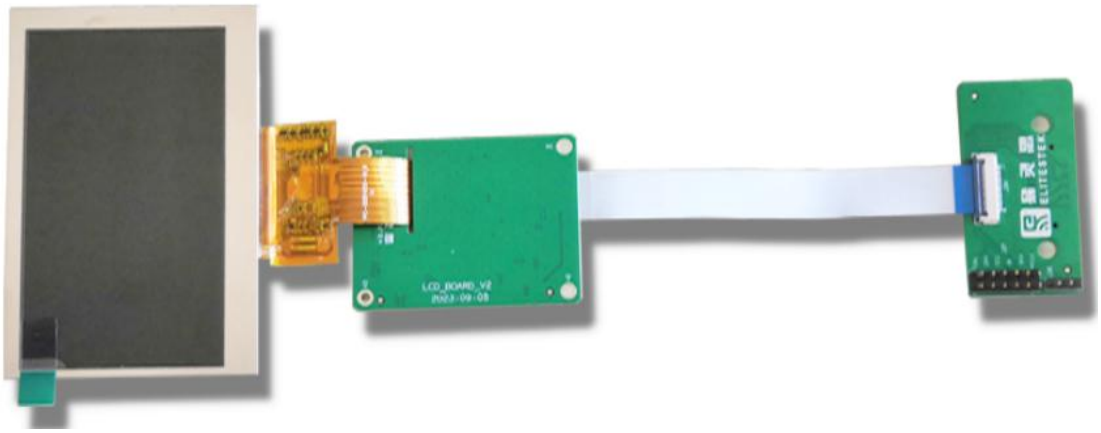


（1） SC431HAI



参数	描述
分辨率	400 万
像素阵列	2568 (H) x 1448 (V)
像素尺寸	2.0 µm x 2.0 µm
镜头光学尺寸	1/3.06”
最大图像传输速率	2560 (H) x 1440 (V) @ 30/60 fps 10-bit 1.2 V
输出接口	8/10-bit 1/2/4-Lane MIPI
输出格式	RAW RGB
CRA	15°
灵敏度	4319 mV/lux·s
动态范围	• 线性模式: 81 dB • HDR 模式: >100 dB
信噪比	38.47 dB
工作温度范围	-30°C ~ +85°C
最佳工作温度范围	-20°C ~ +60°C
电源电压	AVDD = 2.8±0.1 V, DVDD=1.2±0.06 V, DOVDD = 1.8±0.1 V
封装尺寸	CSP, 5.790 mm x 3.554 mm, 35-pin
ESD 等级	• HBM classification: 2 • CDM classification: C3





信号指示

LED	说明

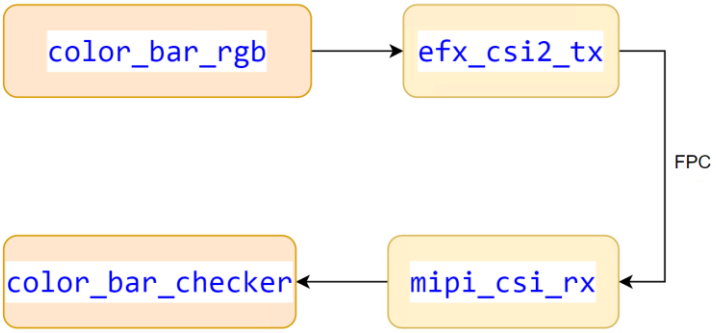
LED0(D0)	D0 一秒闪烁一次，说明 DDR3，cal_done 为高； 如果 D0 不闪烁说明 cal_done 校准有问题；
LED1(D1)	D1 一秒闪烁一次，说明 sensor 接收到稳定的视频信号； 如果 D1 不闪烁说明没有接收到稳定的视频信号；

V4 版本的增益配置如下：

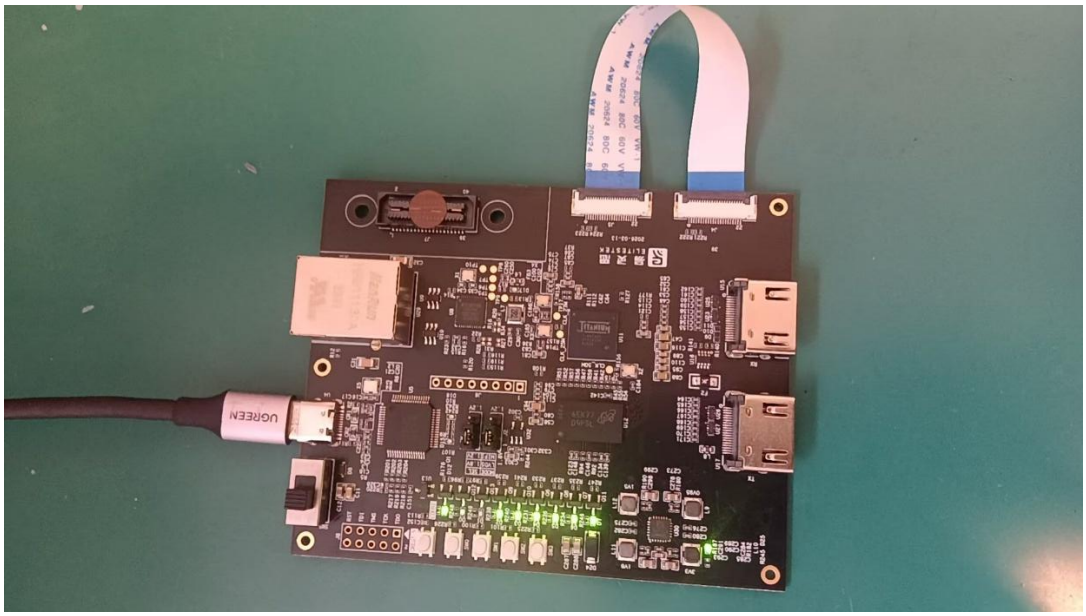
Name	Type	Width	Radix	Value	Control	Comparator
r_gain	Source	16	Dec	00280	Hex	
g_gain	Source	16	Dec	00255	Hex	
b_gain	Source	16	Dec	00350	Hex	
gamma_sel	Source	1	Bin	0	Hex	

11_mipi_csi_loopback

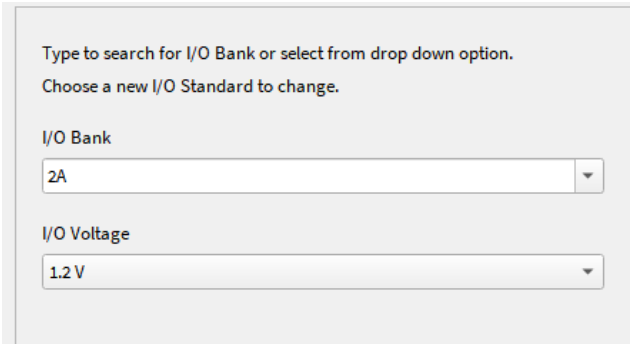
Demo 框图



硬件连接



MIPI 所在的 Bank 为 2A,2B.所以在 interface designer 中要把 bank 2A,2B 要设置成 1.2V

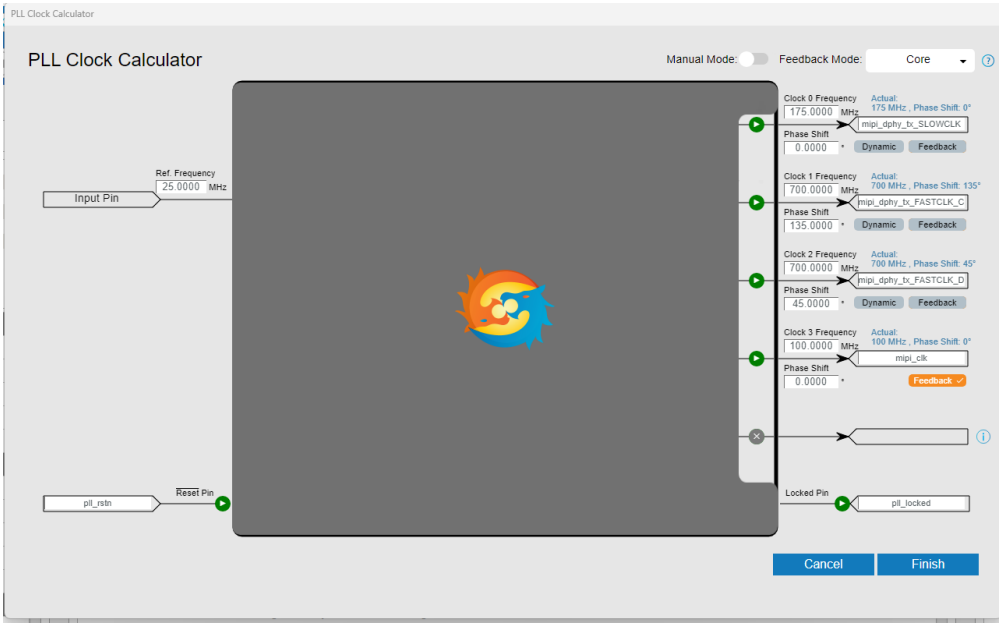


LED 指示灯的作用

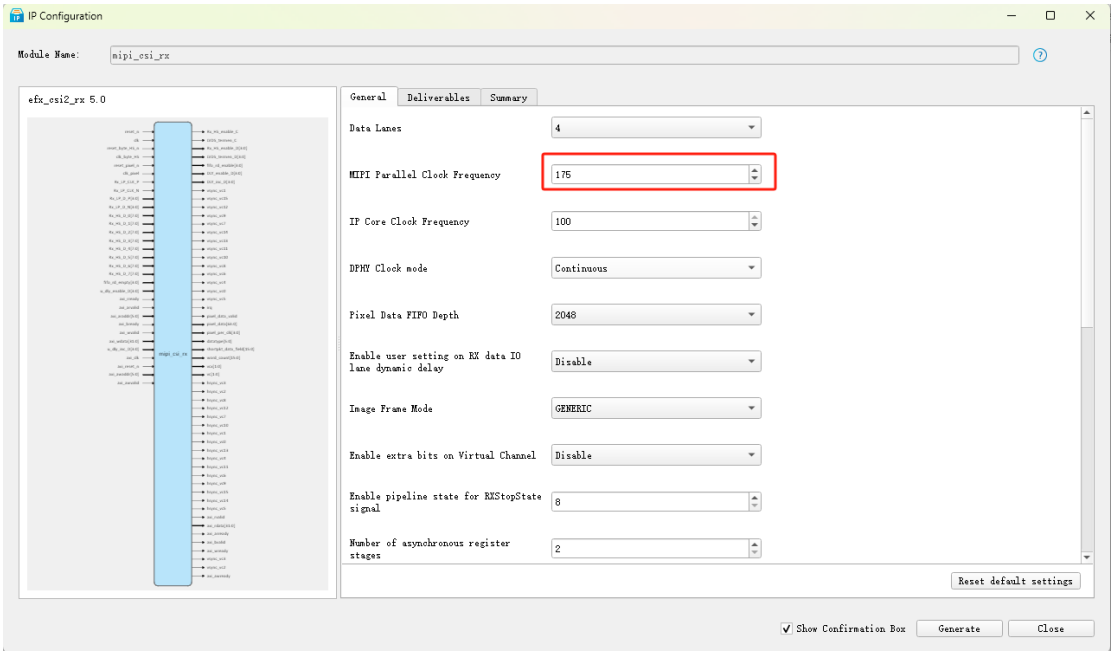
GPIO	例化	丝印	说明
GPIOR_27	O_LED[0]	D0	MIPI DPHY 接收时钟计数指示灯
GPIOR_24	O_LED[1]	D1	接收 fail 指示灯
GPIOL_P_12	O_LED[2]	D2	接收 pass 指示灯
GPIOL_P_02	O_LED[3]	D3	接收视频帧计数指示灯
GPIOL_P_08	CLK_25M		

MIPI 自环速率配置。

Demo 工程设置 MIPI 速率是 1.4Gbps.所以 MIPI TX PLL 的配置如下,



CSI2_RX 和 CSI2_TX IP 的 MIPI Parallel Clock Frequency 频率都修改成 175M。



MIPI CSI TX 的时钟修改

mipi_dphy_tx_SLOWCLK :175M

mipi_dphy_tx_FASTCLK_C : 700M

mipi_dphy_tx_FASTCLK_D： 700M

mipi_clk ： 100M

四、评估传输参数

（1）根据时钟要求计算 pixel 时钟支持的范围。可能自环的时候这两个时钟会有一些矛盾，但是相差不大的话，经过确认是没有问题的。另外我们也相信能够发送成功的，接收也可以。

MIPI TX 要求：

PIX_CLK_MHZ <= (DATARATE_MPBS * NUM_DATA_LANE) / PACK_BIT。

MIPI RX 要求：

PIX_CLK_MHZ >= (DATARATE_MPBS * NUM_DATA_LANE) / PACK_BIT

MIPI 接口参数如下：

MIPI Data Lane = 4

MIPI Data Rate = 1400Mbps

视频格式如下：

```
    parameter PIXEL_BIT       = 24,
    parameter PACK_BIT        = 48,
    parameter HSA             = 5,
    parameter HBP             = 5,
    parameter HFP             = 1024,
    parameter HACT_CNT        = 1920,
    parameter VSA             = 1,
    parameter VBP             = 1,
    parameter VFP             = 100,
    parameter VACT_CNT        = 1080,
    parameter HS_BYTECLK_MHZ = 100,
    parameter DATATYPE        = 6'h24     //24 - RGB888
```

按照上面的公式计算像素时钟。

DATARATE_MPBS　= 1400;

NUM_DATA_LANE = 4;

PACK_BIT = 48;

PIX_CLK_MHZ <= 800 * 2/ 48 = 116.67 //像素时钟频率

是这样就行了吗？当然不是，还要确认修改成两个 lane 之后原来的数据带宽是否还能传输呢？要用评估工具进行评估保证 PASS。

这里 Horizontal blanking per line(us) = (5+5+1024)*(1000/66)ns = 15.51us

Enter the video information	
Data type	RGB888
# of data pixels per line	1920
# of data lines per frame	1080
# of blanking lines per frame	102
# of frames per second (Hz)	8
Horizontal blanking per line (us)	51.70
Pixel clock frequency (MHz)	30
Enter the MIPI interface details	
Data rate (Mbps)	800
# of lanes	2
Clock mode	Continuous
Enter the TX timing parameter settings	☑ Use Default Values
THS-PREPARE (ns)	50
THS-ZERO (ns)	262
THS-TRAIL (ns)	1220
THS-EXIT (ns)	500
TCLK-POST (ns)	455
TCLK-TRAIL (ns)	80
TCLK-PREPARE (ns)	70
TCLK-ZERO (ns)	400
TCLK-PRE (ns)	10
TLPX (ns)	100
Results	
Minimum horizontal blanking test	PASS
Minimum line rate test	PASS
Minimum bandwidth test	PASS

XLanFPGA技术交流

CSI soft IP timing

Figure 2: Video Timing Waveform (Horizontal)

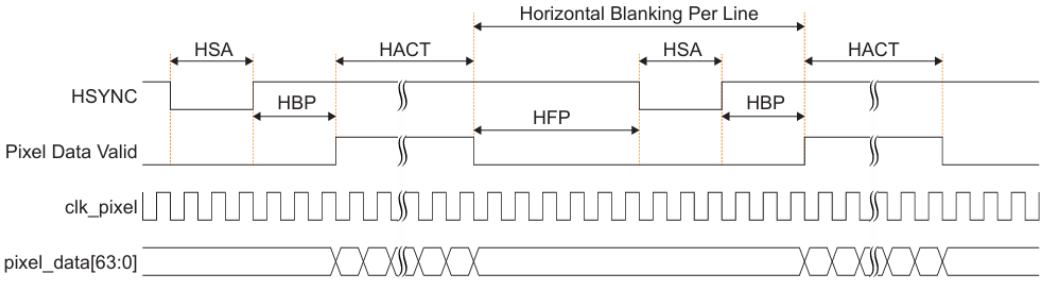
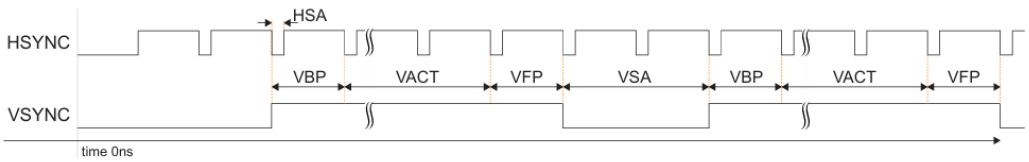
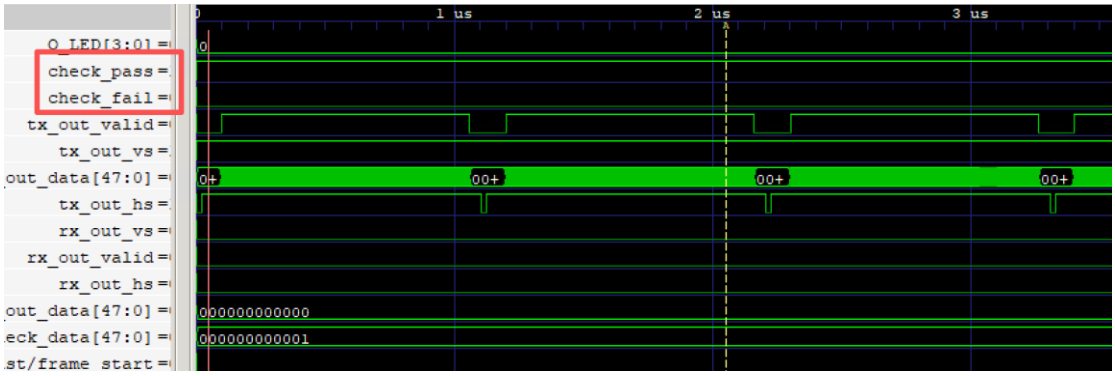


Figure 3: Video Timing Waveform (Vertical)



demo 上我们选择 50M.要计算视频带宽是否能传输，即要考虑公式也要考虑 utility 表。OK ， PASS.

通过 Debug 也可以通过触发 check_pass 来确认数据接收是否正确。



12_mipi_dsi_loop_demo

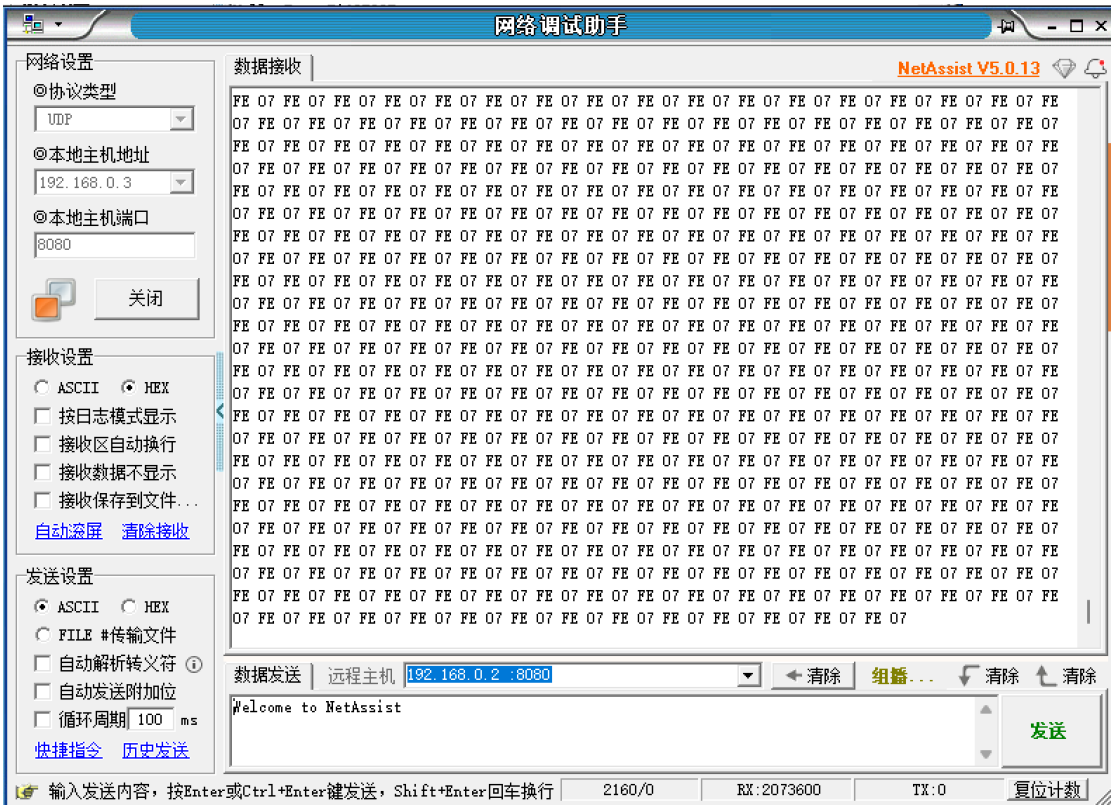
应客户要求，做一个测试程序

13_carrychain_prj_demo

验证逻辑锁定功能

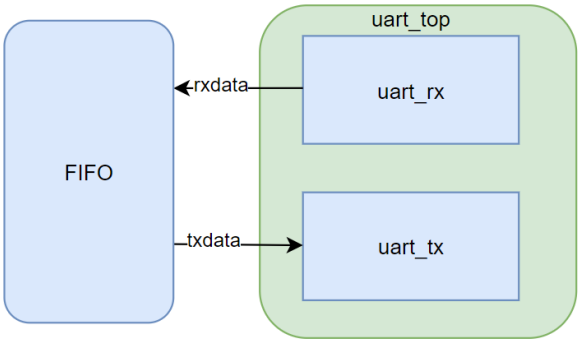
14_Ti60f225_sc431hai2ge_demo

该 demo 实现 sensor 一帧数据采集，通过以太网上传到网络高度助手，方便通过软件分析 RAW 数据，进行 ISP 处理。



17_Ti60F225_uart_demo

一、框图



二、管脚分配

GPIO : Instance View

			all	all
Instance	Package Pin	Resource	I/O Bank	Alt Conn
clk_sel	A2	GPIOL_N_18	1B	None
gpio_clk_25m	J2	GPIOL_P_08	1A	GCLK,RCLK
gpio_clk_27m	H2	GPIOL_P_10	1B	GCLK,RCLK
gpio_clk_50m	R13	GPIOR_29	BR	PLL_CLKIN
led[0]	G3	GPIOL_P_12	1B	None
led[1]	N2	GPIOL_P_02	1A	None
led[2]	G4	GPIOL_N_12	1B	None
led[3]	E1	GPIOL_P_15	1B	VREF
rxd	R4	GPIOL_02	BL	None
txd	R14	GPIOR_28	BR	None

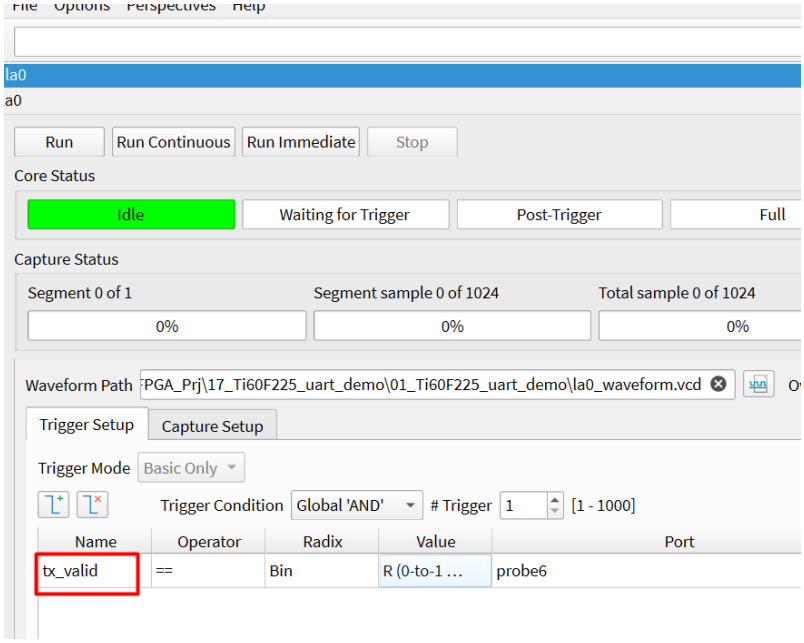
从管脚分配上可以看到 RXD,TXD 在 TR bank

二、程序说明

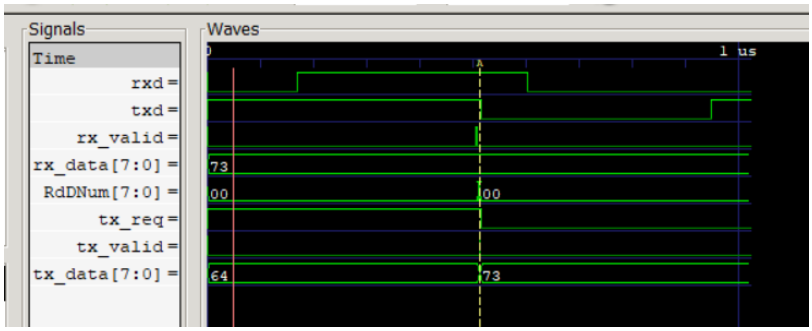
Uart 的 rxd 和 txd 连接到了 LED 上，所以当正常收发时会看到两个灯有闪烁，另外两个灯设置为 1，正常运行是暗。如下图是一张快速连续收发数据的效果，正常收发时 D3,D4 会闪烁，D5,D6 固定为暗。

```
  assign led[1:0] = {rxd,txd};
  assign led[3:2] = 2'b11;
```

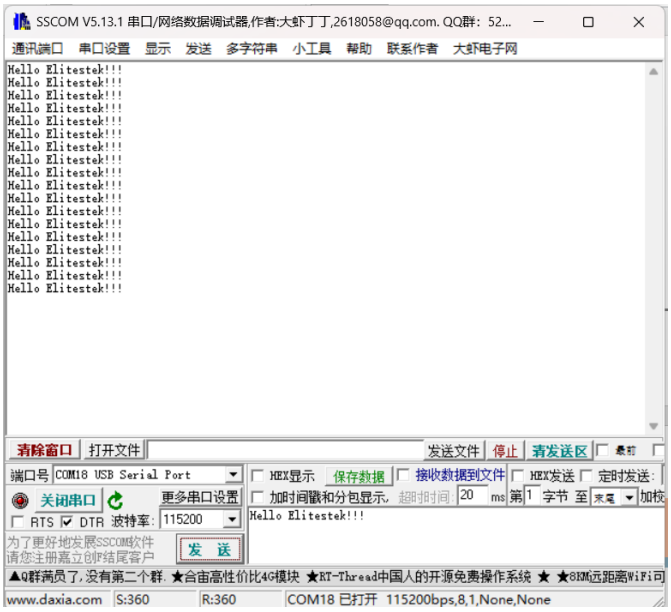
程序添加了 debug,可以设置触发条件为 tx_valid



在收发数据时可以看到 txd 数据实际已经发出



测试效果如下:


SSCOM V5.13.1 串口/网络数据调试器,作者:大虾丁丁,2618058@qq.com. QQ群：52...
通讯端口 串口设置 显示 发送 多字符串 小工具 帮助 联系作者 大虾电子网
Hello Elitestek!!!
清除窗口
打开文件
发送文件
停止
请发送区
端口号 COM18 USB Serial Port
HEX显示
保存数据
接收数据到文件
HEX发送
定时发送:
关闭串口
更多串口设置
加时间戳和分包显示,
超时时间
20
ms
第
1
字节 至
末尾
加校
RTS
DTR
波特率:
115200
为了更好地发展SSCOM软件
请您注册嘉立创P结尾客户
发 送
▲Q群满员了,没有第二个群. ★合宙高性价比4G模块 ★RT-Thread中国人的开源免费操作系统 ★ ★8KM远距离WiFi可
www.daxia.com
S:360
R:360
COM18 已打开 115200bps,8,1,None,None